Department of Computer Science and Engineering

Unit 3

Raster Graphics Algorithms

By

Dr. Srividya M S

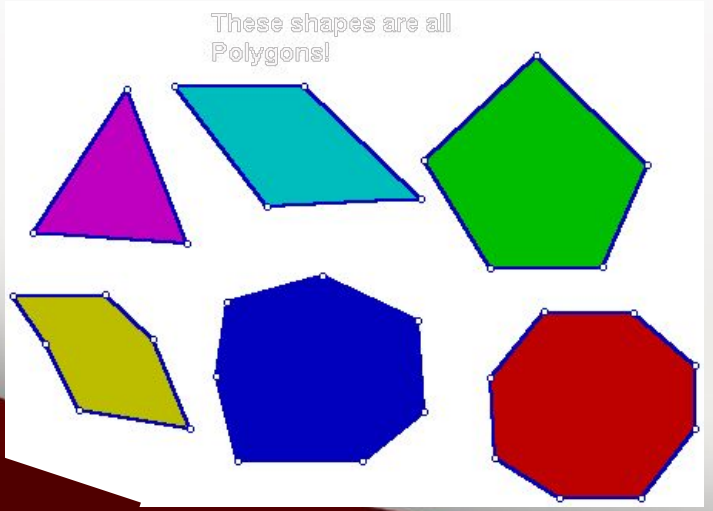
Computer Graphics

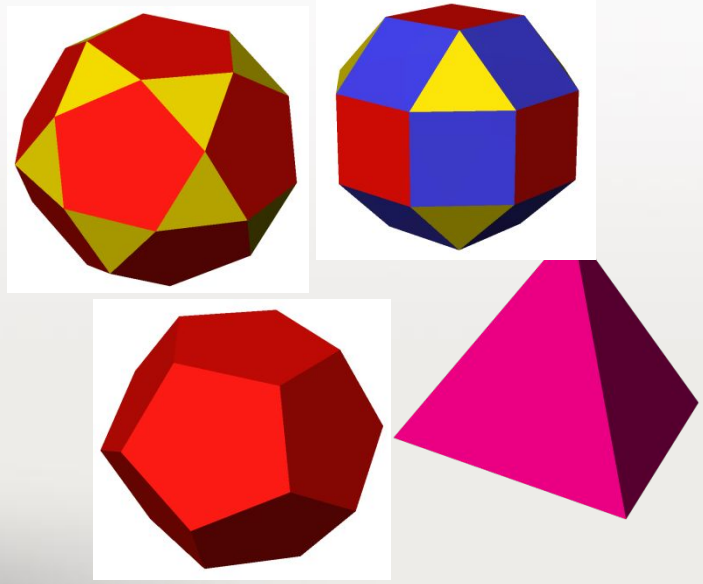
Objectives

- Introduce the elements of geometry
  - Points
  - Lines
- Introduce Three-dimensional Primitives.
- Develop mathematical operations among them in a coordinate-free manner.

Basic Elements

- CG has a set of ***geometric objects*** such as **lines, polygons and polyhedra.**
- In geometry, polyhedron (plural polyhedra) is a geometric solid in 3D with flat faces and straight edges.



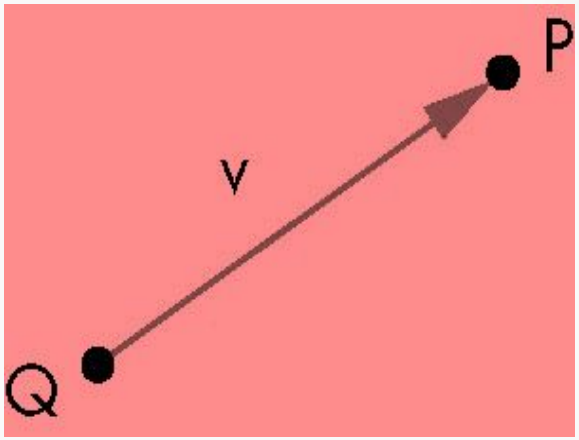


Points

- **Fundamental** geometric object.
- In 3D geometric system, it is a **location in** space.
- **Property of point :** ***its location.***
- A mathematical point has ***no size*** , ***no shape.***
- Useful in specifying geometric objects but are not sufficient.
- We need real numbers to specify the distance between two points.

Vectors

- A vector is a quantity with two attributes
  - Magnitude (length)
  - Direction (orientation)
- ***Examples***
  - Force
  - Velocity



- A vector **does not have a fixed location** in space.
- In CG, a *directed line segment* has both magnitude(length) and direction(orientation) and hence a *vector.*

unit 3

Geometric Lines

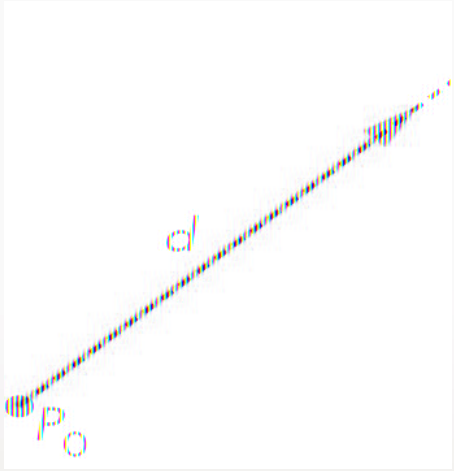
Lines

***Point + Vector = line* in an affine space.**

***Point - Point = line* in an affine space.**

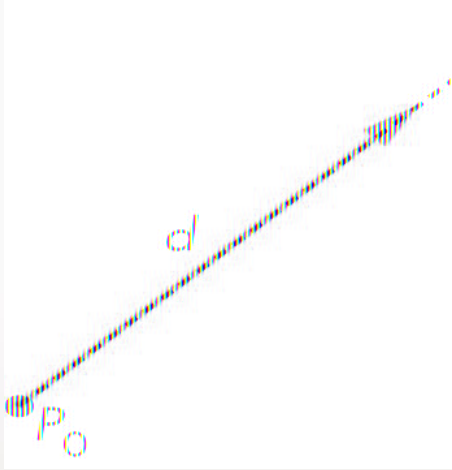
Consider all points of the form $P(\alpha) = P_0 + \alpha d$, where P_0 is an arbitrary point, d is an arbitrary vector, and α is a scalar(can vary).

$P(\alpha)$ is a point for any value of α. For geometric vectors, these points lie on a line, as shown.



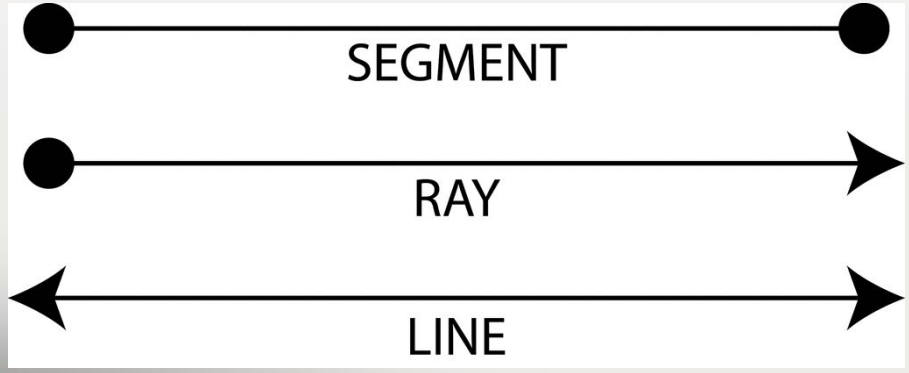
Lines

- This is **parametric form of the line**, because we generate points on the line by varying the parameter *α*.
- For $\alpha = 0$, the line passes through the point P_0 and, as α is increased, all the points generated lie in the direction of the vector *d*.
- If α is nonnegative value, ray emanating from P_0 and going in the direction of d is obtained.



Lines

- **Line**
  - Infinitely long in both the directions.
- **Line segment**
  - Finite piece of line between 2 points.
- **Ray**
  - Infinitely long in one direction.



LINE-DRAWING ALGORITHMS

A straight-line segment in a scene is defined by the coordinate positions for the endpoints of the segment.

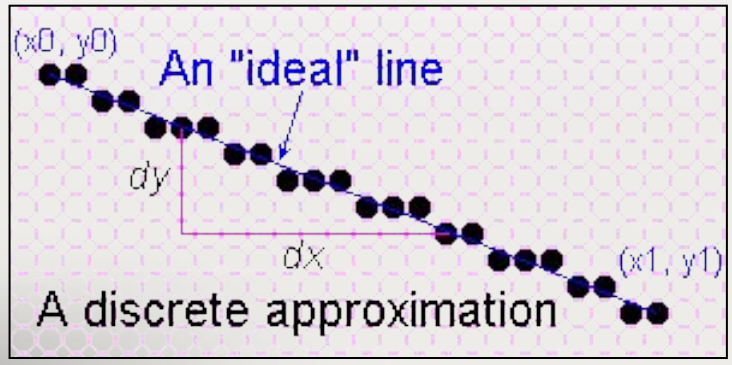
1. To display the line on a raster monitor, the graphics system must
   - first project the endpoints to integer screen coordinates and
   - determine the nearest pixel positions along the line path between the two endpoints.
2. Then the line color is loaded into the frame buffer at the corresponding pixel coordinates.
3. Reading from the frame buffer, the video controller plots the screen pixels.
   - This process digitizes the line into a set of discrete integer positions that, in general, only approximates the actual line path.

LINE-DRAWING ALGORITHMS

- On raster systems, lines are plotted with pixels, and step sizes in the horizontal and vertical directions are constrained by pixel separations.

- “Sample" a line at discrete positions and determine the nearest pixel to the line at sampled position.
  - Sampling is measuring the values of the function at equal intervals

- A line is sampled at unit intervals in one coordinate and the corresponding integer values nearest the line path are determined for the other coordinate.



What is an *ideal* line

- Must appear straight and continuous
  - Only possible with axis-aligned and 45° lines
- Must interpolate both defining end points
- Must have uniform density and intensity
  - Consistent within a line and over all lines
  - What about anti-aliasing ?
    - Aliasing is the jagged edges on curves and diagonal lines in a bitmap image.
    - Anti-aliasing is the process of smoothing out those jaggies.
      - Graphics software programs have options for anti-aliasing text and graphics.
      - Enlarging a bitmap image accentuates the effect of aliasing.
- Must be efficient, drawn quickly
  - Lots of them are required

Simple Line

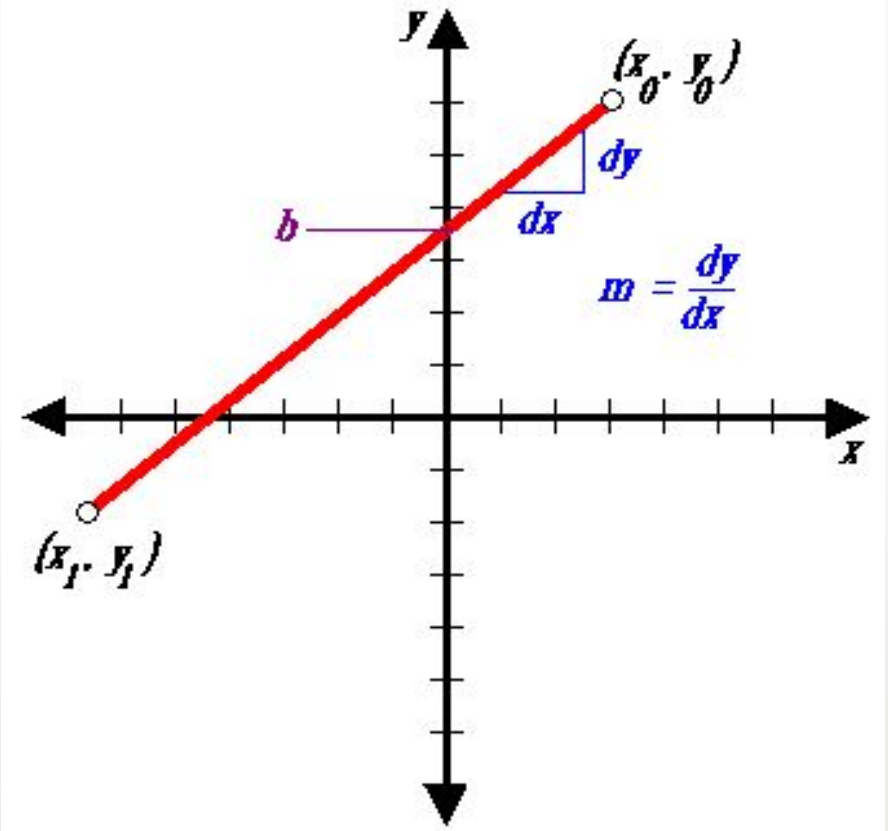
*The **Cartesian slope-intercept equation** for a straight line is :*

$$y = mx + b$$

with *m* as the slope of the line and ***b*** as the ***y*** intercept.

Simple approach:

- increment x, solve for y



Line-Drawing Algorithms:

DDA
and

Bresenham's Line Drawing Algorithm

DDA: Digital Differential Analyzer

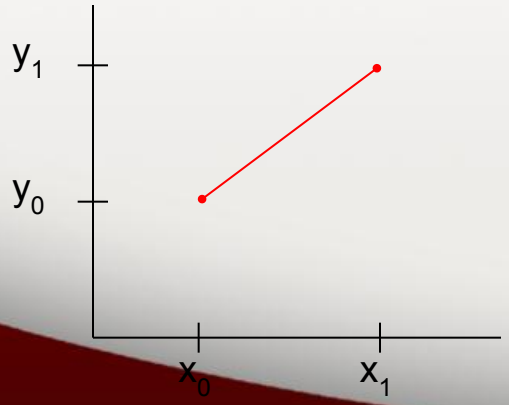
Algorithms for displaying lines are based on the Cartesian slope-intercept equation

$y = m.x + b$

where m and b can be calculated from the line endpoints: $m = (y_1-y_0) / (x_1-x_0)$

$b = y_0 - m.\ x_0$

For any x interval δx along a line the corresponding y interval $\delta y = m.\delta x$



DDA Algorithm

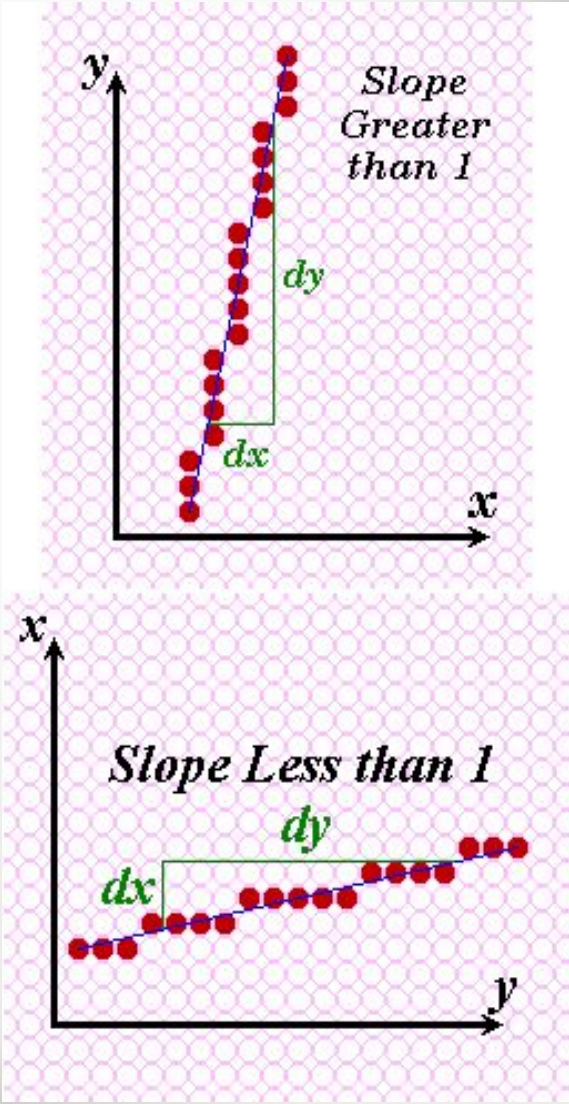
- The digital differential analyser (DDA) is a scan-conversion line algorithm based on using δx or δy.

- A line is sampled at unit intervals in one coordinate (x) and the corresponding integer values nearest the line path are determined for the other coordinate (y).

Line with positive slope

- If m>1,
  - Sample at unit y intervals (dy=1)
  - Compute successive x values as
    - $x_{k+1}=x_k+1/m$ $0<=k<=y_{end}-y_0$
    - Increment k by 1 for each step
    - Round x to nearest integer value.

- If m<=1,
  - Sample at unit x intervals (dx=1)
  - Compute successive y values as
    - $y_{k+1}=y_k+m$ $0<=k<=x_{end}-x_0$
    - Increment k by 1 for each step
    - Round y to nearest integer value.



```
inline int round (const float a)  { return int (a + 0.5); }

void lineDDA (int x0, int y0, int xEnd, int yEnd)
{
    int dx = xEnd - x0,  dy = yEnd - y0,  steps,  k;
    float xIncrement, yIncrement, x = x0, y = y0;

    if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
    else
        steps = fabs (dy);
    xIncrement = float (dx) / float (steps);
    yIncrement = float (dy) / float (steps);

    setPixel (round (x), round (y));
    for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel (round (x), round (y));
    }
```

m<=1

then xincrement becomes 1

steps: This is the number of pixels to plot, which is set to the larger of abs(dx) or abs(

DDA algorithm

- Need a lot of floating point arithmetic.
  - 2 ‘round’s and 2 adds per pixel.

- Is there a simpler way ?
- Can we use only integer arithmetic ?
  - Easier to implement in hardware.

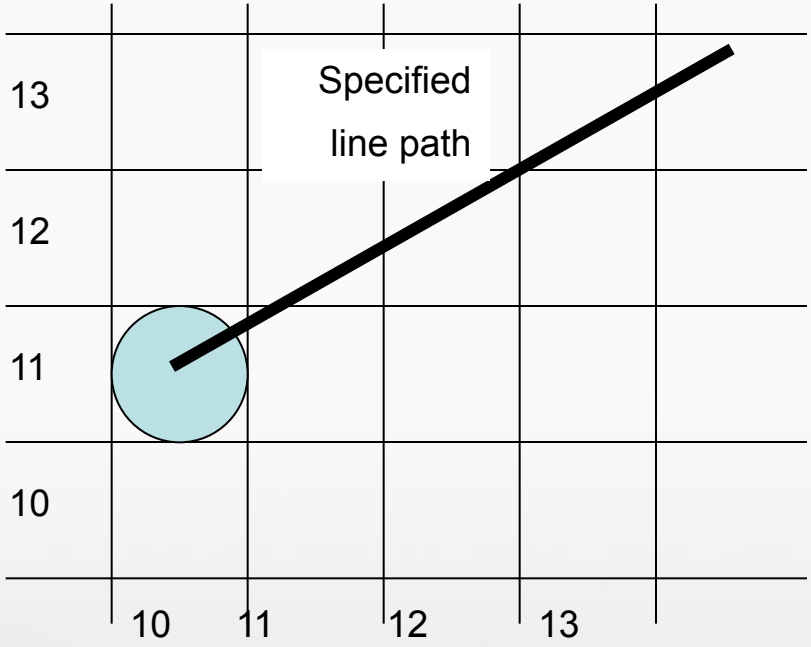
Bresenham's line algorithm

- Accurate and efficient
- Uses only incremental integer calculations

The method is described for a line segment with a positive slope less than one

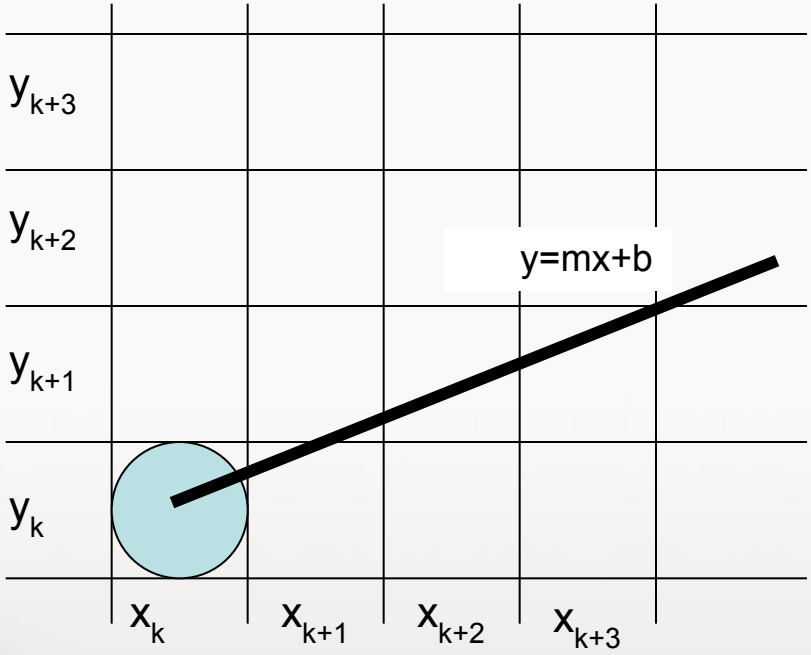
The method generalizes to line segments of other slopes by considering the symmetry between the various octants and quadrants of the xy plane

Bresenham's line algorithm



- Decide what is the next pixel position
  - (11,11) or (11,12)

Illustrating Bresenham's Approach



- For the pixel position $x_{k+1}=x_k+1$, which one we should choose:
- (x_{k+1},y_k) or (x_{k+1}, y_{k+1})

Bresenham's line algorithm

Advantage:

1. It involves only integer arithmetic, so it is simple.
2. It avoids the generation of duplicate points.
3. It can be implemented using hardware because it does not use multiplication and division.
4. It is faster as compared to DDA (Digital Differential Analyzer) because it does not involve floating point calculations like DDA Algorithm.

Disadvantage:

1. This algorithm is meant for basic line drawing only Initializing is not a part of Bresenham's line algorithm. So to draw smooth lines, you should look into a different algorithm.

Bresenham's Approach

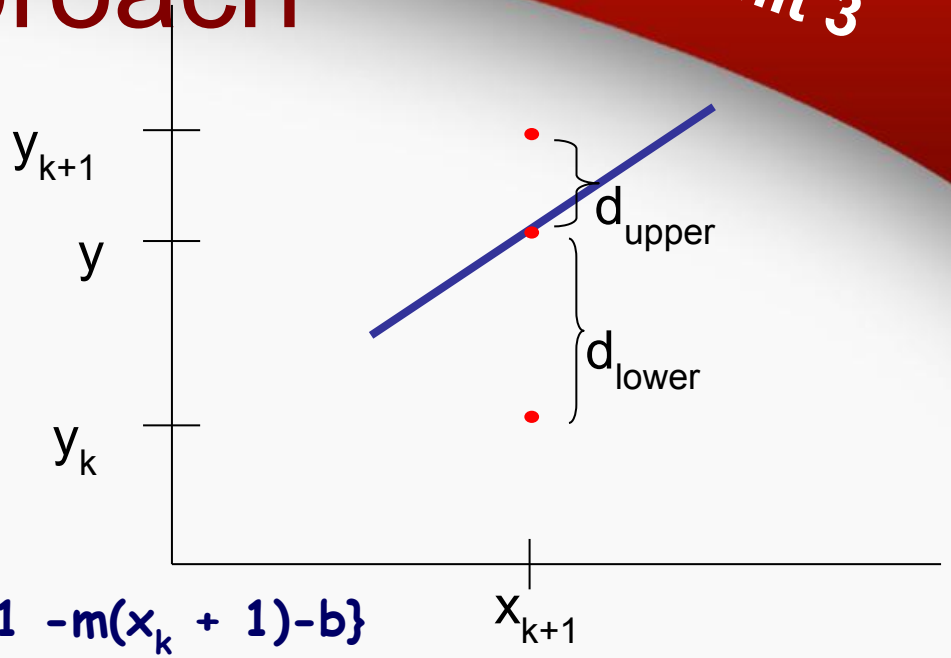
$y = m(x_k + 1) + b$

$d_{lower} = y - y_k$

$= m(x_k + 1) + b - y_k$

$d_{upper} = (y_k + 1) - y$

$= y_k + 1 - m(x_k + 1) - b$



$d_{lower} - d_{upper} = m(x_k + 1) + b - y_k - \{y_k + 1 - m(x_k + 1) - b\}$

$= 2m(x_k + 1) - 2y_k + 2b - 1$

Multiplying both side by (dx)

$dx(d_{lower} - d_{upper}) = 2dy(x_k + 1) - 2dx(y_k) + 2dx(b) - dx$

Now we need to find decision parameter P_k

$P_k = dx(d_{lower} - d_{upper})$ and, $C = 2dy + 2dx(b) - dx$ which is constant

The Decision Parameter

So new equation is.

$P_K = 2dy(x_k) - 2dx(y_k) + C$ ……[1]

Now our next parameter will be

$P_{K+1} = 2dy(x_{k+1}) - 2dx(y_{k+1}) + C$

Subtracting P_k from P_{k+1}

$P_{k+1} - P_k = 2dy(x_{k+1} - x_k) - 2dx(y_{k+1} - y_k) + C - C$

Note: here x is always incrementing so we can write x_{k+1} as $x_k + 1$

$P_{k+1} - P_k = 2dy(x_k - x_k + 1) - 2dx(y_{k+1} - y_k)$

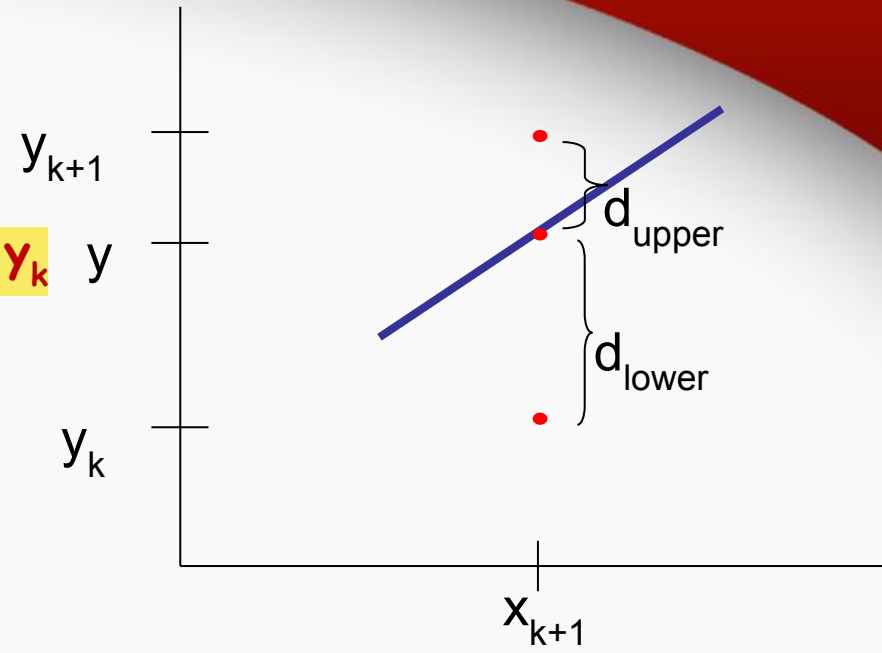
$P_{k+1} = P_k + 2dy - 2dx(y_{k+1} - y_k)$ .....[2]

Successive decision parameter

$P_{k+1} = P_k + 2dy - 2dx(y_{k+1} - y_k)$.....(2)

when $P_k < 0$ then $(d_{lower} - d_{upper}) < 0$

So $d_{lower} < d_{upper}$ then we will write y_{k+1} as y_k



Then our formula will be:

$P_{k+1} = P_k + 2dy - 2dx(y_k - y_k)$

$P_{k+1} = P_k + 2dy$

And when $P_k > 0$ then $(d_{lower} - d_{upper}) > 0$

So $d_{lower} > d_{upper}$ then we will write y_{k+1} as $y_k + 1$

At that time our formula will be:

$P_{k+1} = P_k + 2dy - 2dx(y_k + 1 - y_k)$

$P_{k+1} = P_k + 2dy - 2dx$

Initial decision parameter

For Initial decision parameter

From 1st equation

$P_K = 2dy(x_k)-2dx(y_k) + C$

$P_K = 2dy(x_k)-2dx(y_k)+2dx(b)-dx+2dy$

By using cartesian line equation

$y_k=m(x_k)+b$

$b=y_k-m(x_k)$

$P_0 = 2dy(x_k)-2dx(y_k)+2dx(y_k-m(x_k))-dx+2dy$ (By using $m=dy/dx$)

$= 2dy(x_k)-2dx(y_k)+2dxy_k-2dyx_k+2dy-dx$

$P_0 = 2dy-dx$

Bresenham's Line-Drawing Algorithm for | m | < 1

1. Input the twoline endpoints and store the left endpoint in (x_0, y_0).

2. Load (x_0, y_0) into the frame buffer; that is, plot the first point.

3. Calculate constants Δx, Δy, $2\Delta y$, and $2\Delta y - 2\Delta x$, and obtain the starting value fo the decision parameter as

   $p_0 = 2\,\Delta y - \Delta x$

4. At each x_k along the line, starting at k = 0, perform the following test:

   If $p_k < 0$, the next point to plot is (x_{k+1}, y_k) and

   $p_{k+1} = p_k + 2\Delta y$

   Otherwise, the next point to plot is (x_{k+1}, y_{k+1}) and

   $p_{k+1} = p_k + 2\Delta y - 2\Delta x$

5. Repeat step 4 Δx -1 times.

Trivial Situations: Do not need Bresenham

- $m = 0 \Rightarrow$ horizontal line
- $m = \pm 1 \Rightarrow$ line $y = \pm x$
- $m = \infty \Rightarrow$ vertical line

Example

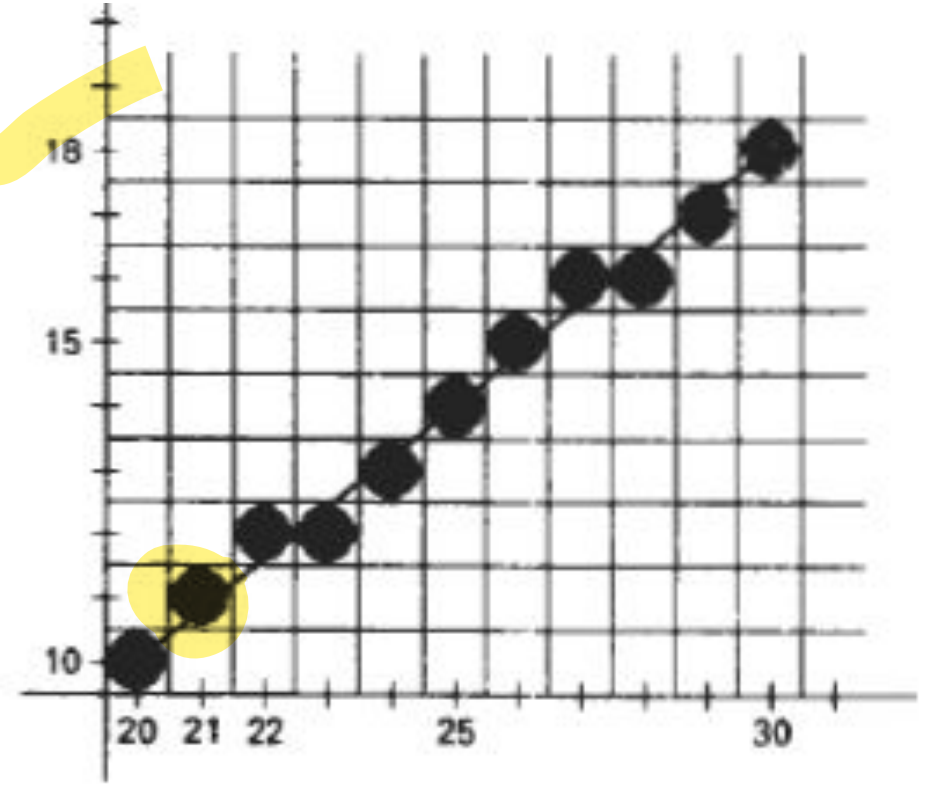
- Draw the line with endpoints (20,10) and (30, 18).
  - Δx=30-20=10, Δy=*18-10*=8,
  - $p_0 = 2\Delta y - \Delta x$=16-10=6
  - *2*Δy=*16*, and *2*Δy - 2Δx=-4
- Plot the initial position at (20,10), then

If $\mathbf{p_k < 0}$, the next point to plot is (x_{k+1}, y_k) and $p_{k+1}=p_k + 2\Delta y$

Otherwise, the next point to plot is (x_{k+1}, y_{k+1}) and $p_{k+1}=p_k + 2\Delta y - 2\Delta x$

k	p_k	(x_{k+1}, y_{k+1})	k	p_k	(x_{k+1}, y_{k+1})
0	6	(21, 11)	5	6	(26, 15)
1	2	(22, 12)	6	2	(27, 16)
2	−2	(23, 12)	7	−2	(28, 16)
3	14	(24, 13)	8	14	(29, 17)
4	10	(25, 14)	9	10	(30, 18)

k	p_k	(x_{k+1}, y_{k+1})	k	p_k	(x_{k+1}, y_{k+1})
0	6	(21, 11)	5	6	(26, 15)
1	2	(22, 12)	6	2	(27, 16)
2	−2	(23, 12)	7	−2	(28, 16)
3	14	(24, 13)	8	14	(29, 17)
4	10	(25, 14)	9	10	(30, 18)



Example

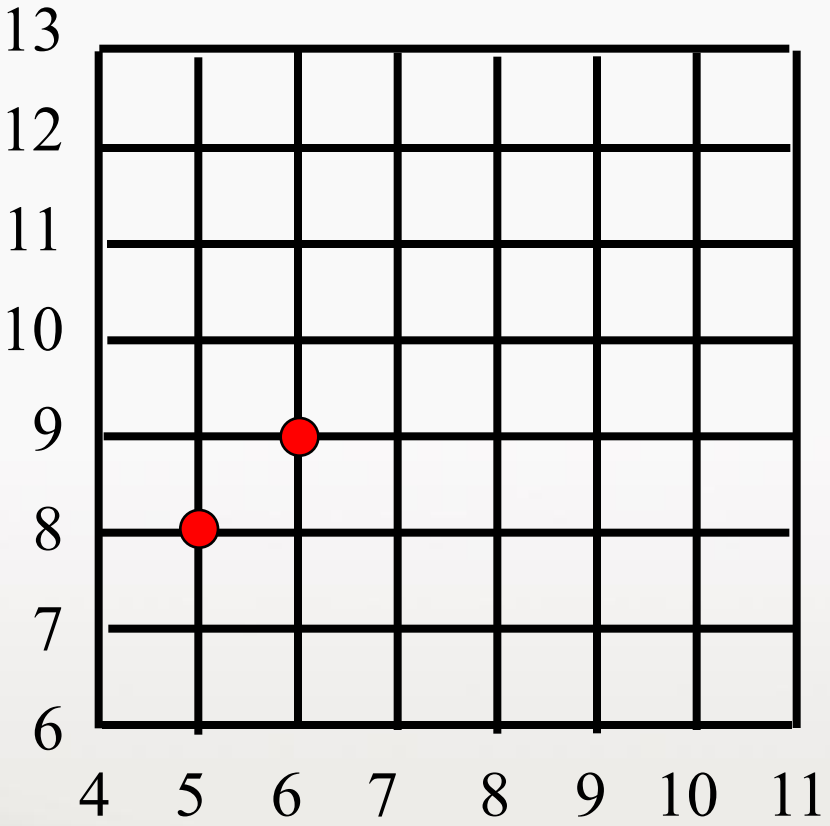
- Line end points: $(x_0, y_0) = (5,8); \quad (x_1, y_1) = (9,11)$
- Deltas: $dx = 4; dy = 3$

initially $p(5,8) = 2(dy)\text{-}(dx)$

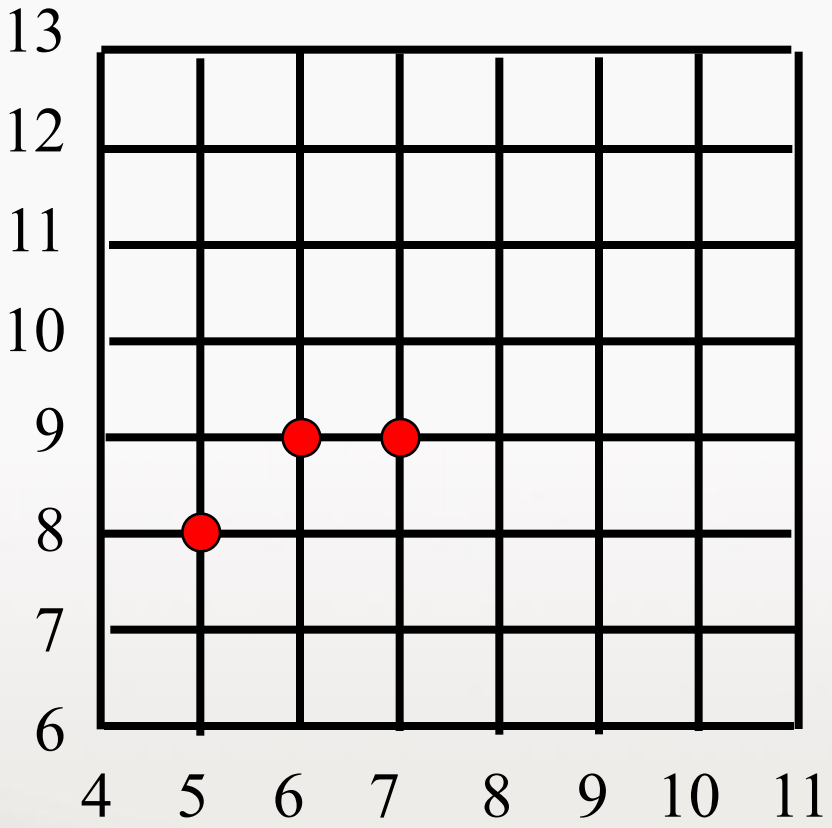
$$= 6 - 4 = 2 > 0$$

$$p = 2 \quad \Rightarrow \quad NE$$

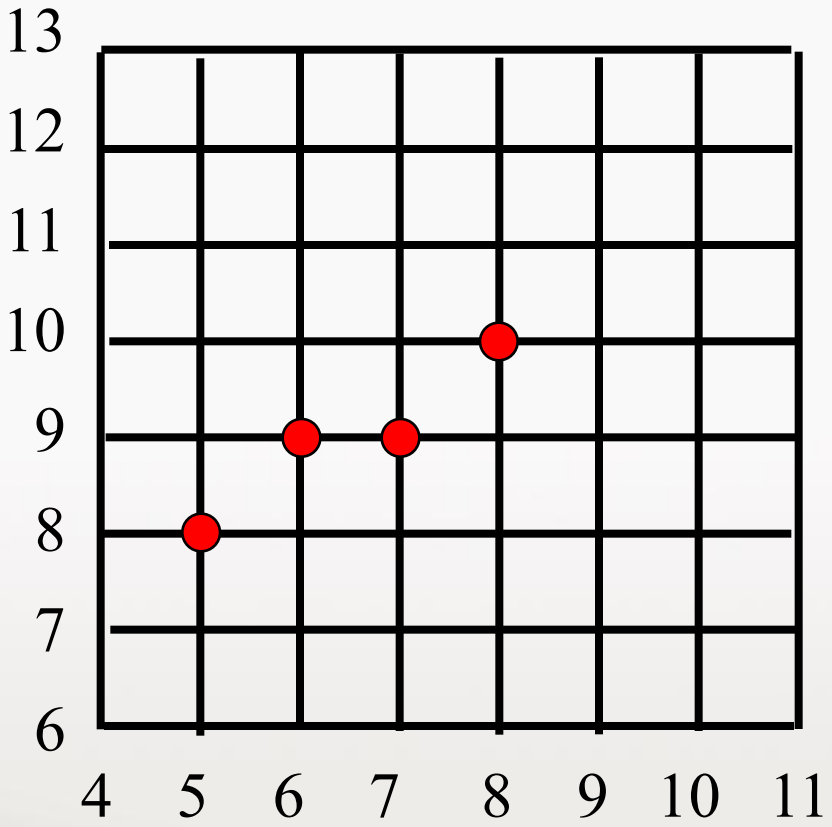
Graph



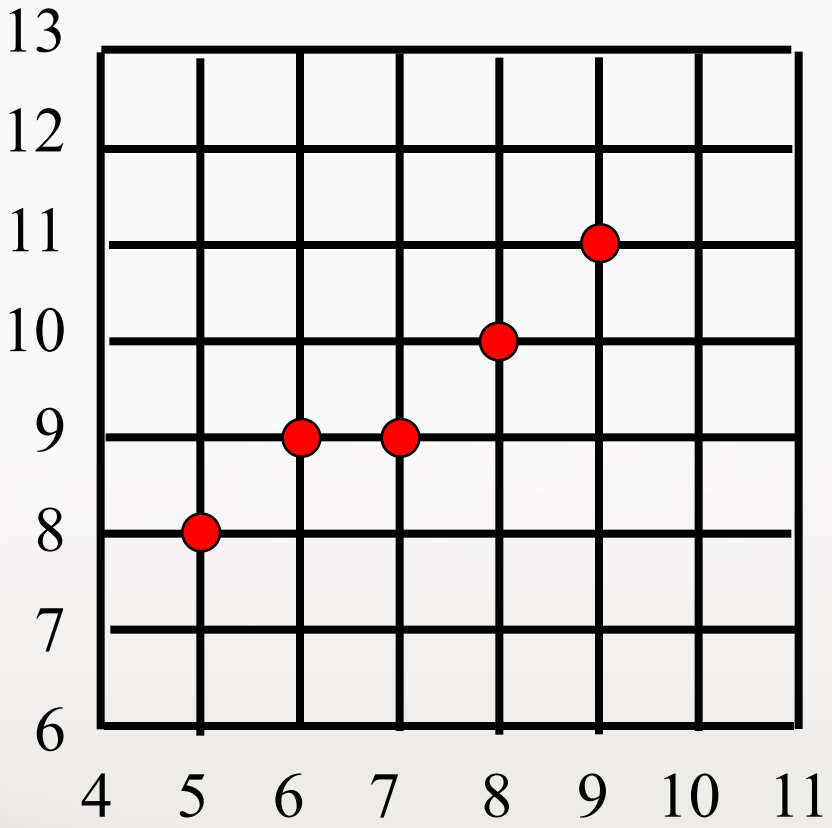
Continue the process...



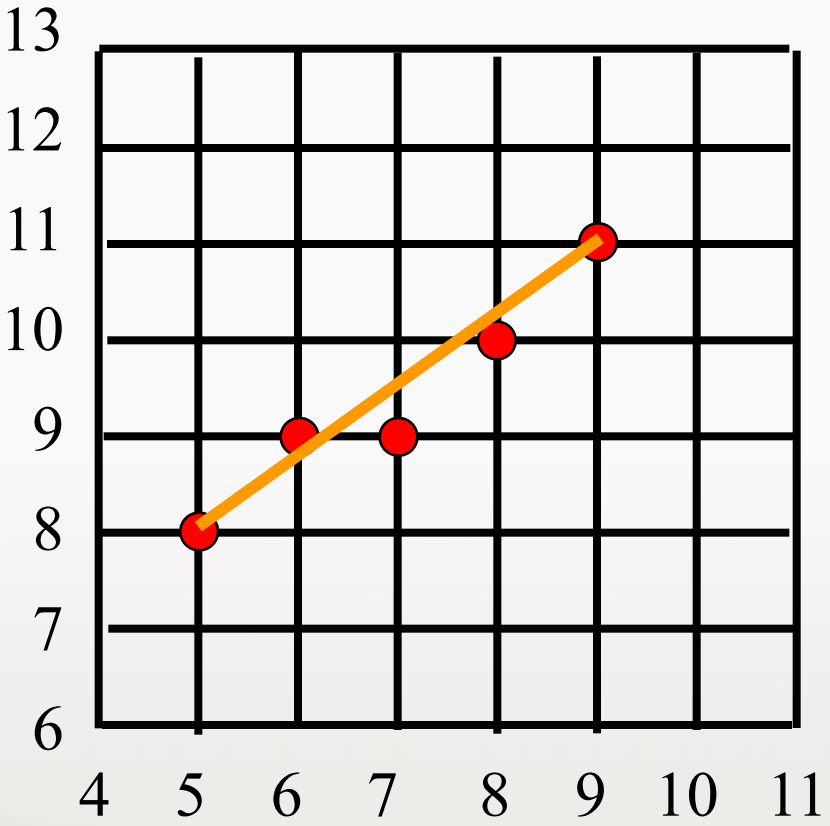
Graph



Graph

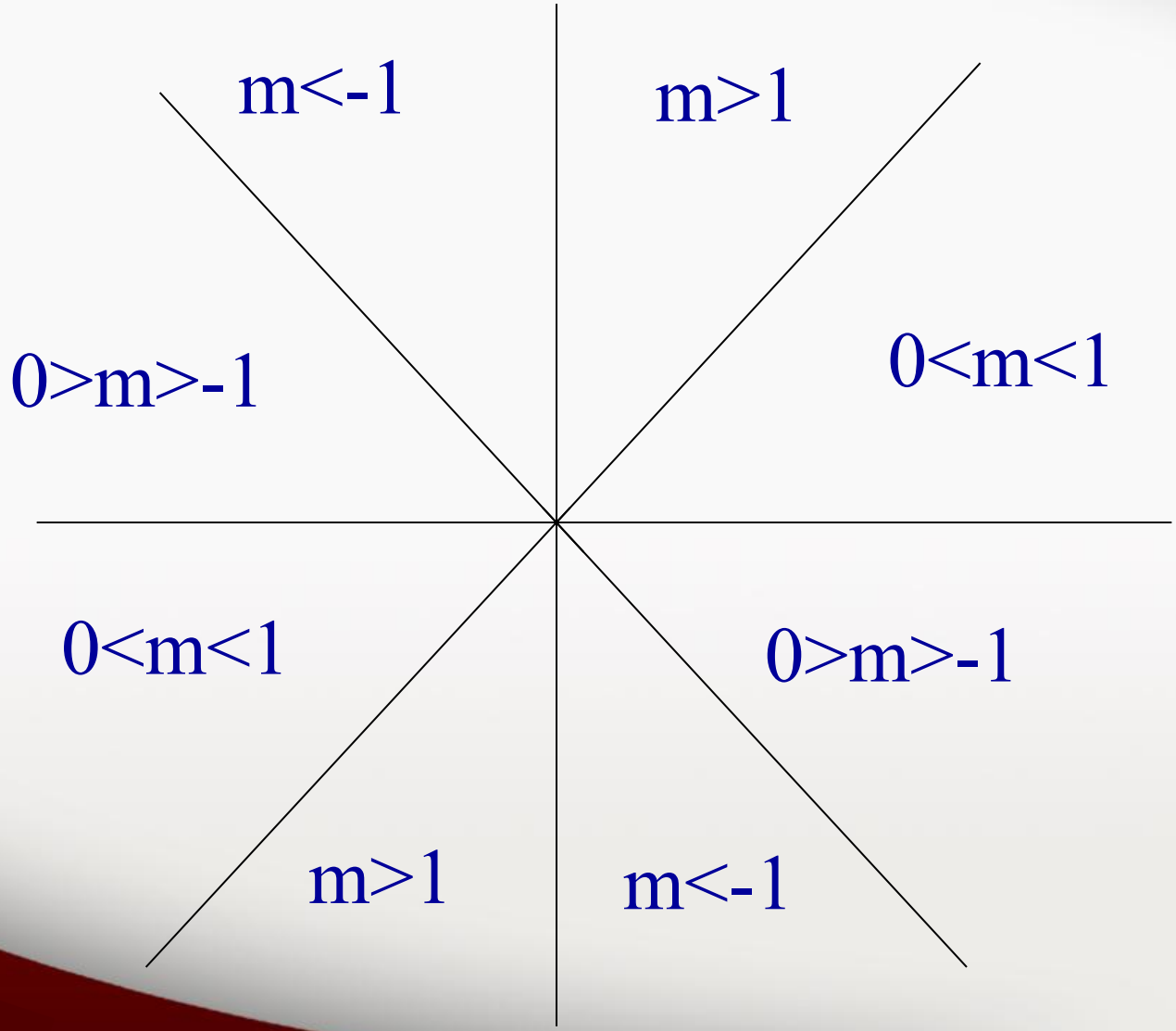


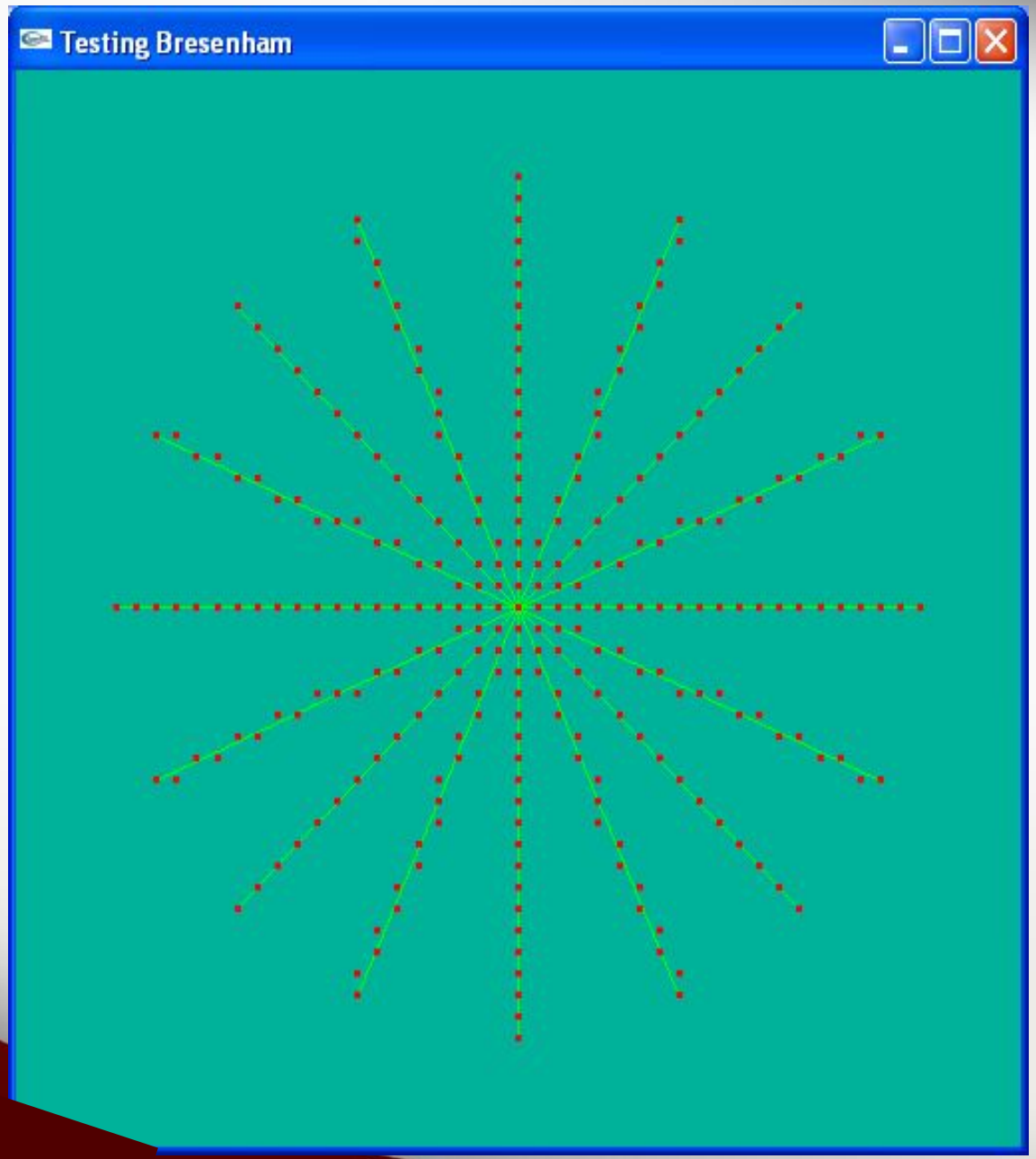
Graph



```
/*  Bresenham line-drawing procedure for |m| < 1.0.  */
void lineBres (int x0, int y0, int xEnd, int yEnd)
{
   int dx = fabs (xEnd - x0),  dy = fabs(yEnd - y0);
   int x, y, p = 2 * dy - dx;
   int twoDy = 2 * dy,  twoDyMinusDx = 2 * (dy - dx);

   /* Determine which endpoint to use as start position.  */
   if (x0 > xEnd) {
      x = xEnd;    y = yEnd;   xEnd = x0;
   }
   else {
      x = x0;     y = y0;
   }
   setPixel (x, y);

   while (x < xEnd) {
      x++;
      if (p < 0)
         p += twoDy;
      else {
         y++;
         p += twoDyMinusDx;
      }
      setPixel (x, y);
   }
```

Line-drawing algorithm should work in every octant, and special cases





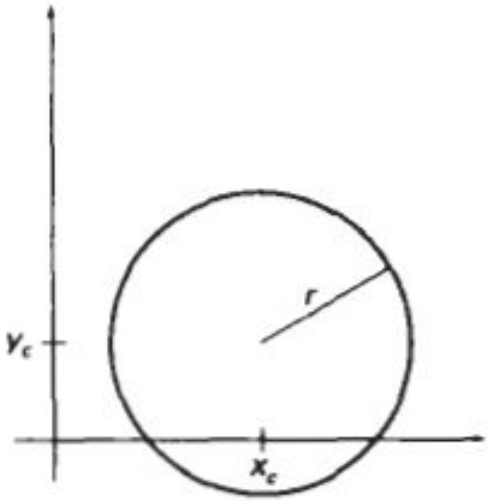
Simulating the Bresenham algorithm in drawing 8 radii on a circle of radius 20

Horizontal, vertical and ±45° radii handled as special cases

Scan Converting Circles

$$(x - x_c)^2 + (y - y_c)^2 = R^2$$

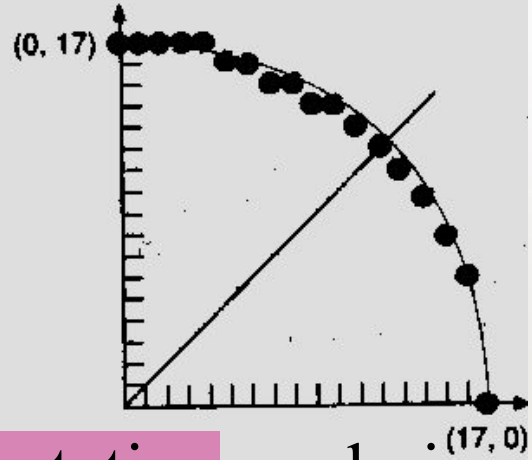
$$y = y_c \pm \sqrt{R^2 - (x - x_c)^2}$$



- Explicit: y = f(x)

$$y = \pm\sqrt{R^2 - x^2}$$

We could draw a quarter circle by incrementing x from 0 to R in unit steps and solving for +y for each step.

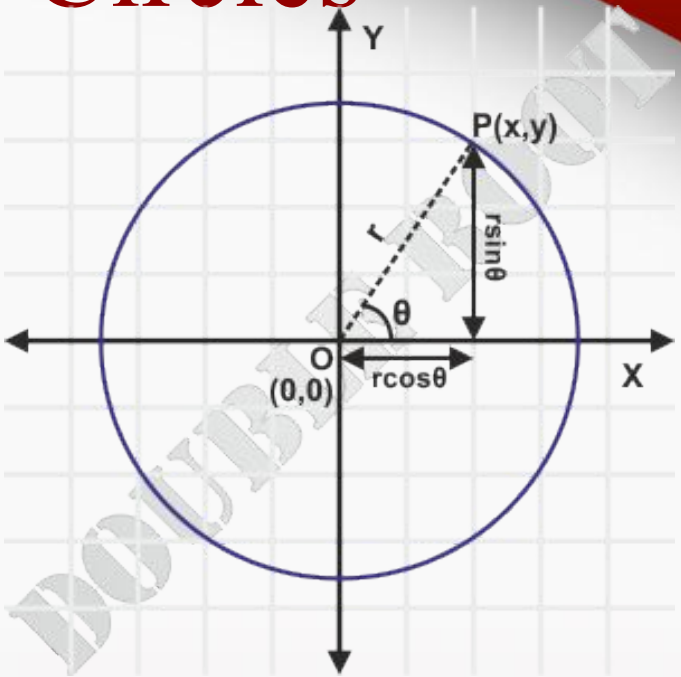


Method needs lots of computation, and gives non-uniform pixel spacing

Scan Converting Circles

Parametric Equation for Circle:

$$x = R\cos\theta$$
$$y = R\sin\theta$$



Draw quarter circle by stepping through the angle from 0 to 90

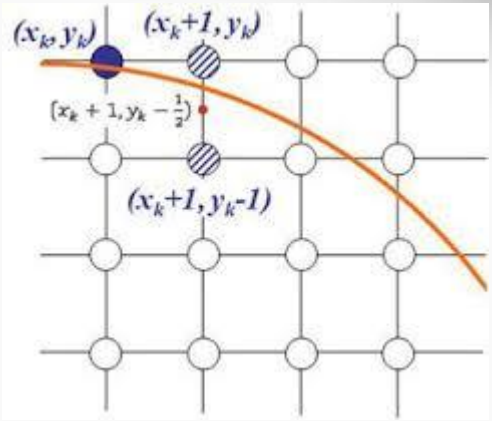
- avoids large gaps but still unsatisfactory
- How to set angular increment

Computationally expensive trigonometric calculations

Scan Converting Circles

- Implicit: $f(x,y) = x^2+y^2-R^2$

If $f(x,y) = 0$ then it is on the circle.
$f(x,y) > 0$ then it is outside the circle.
$f(x,y) < 0$ then it is inside the circle.



Try to adapt the Bresenham midpoint approach

Again, exploit symmetries

Generalising the Bresenham midpoint approach

- Set up decision parameters for finding the closest pixel to the cırcumference at each sampling step
  - Avoid square root calculations by considering the squares of the pixel separation distances
- Use direct comparison without squaring.

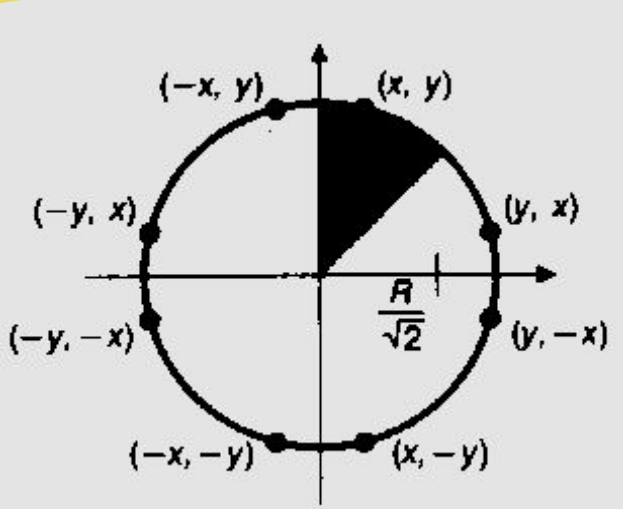
  Adapt the midpoint test idea: test the halfway position between pixels to determine if this midpoint is inside or outside the curve

  This gives the **midpoint algorithm for circles**

  Can be adapted to other curves: conic sections

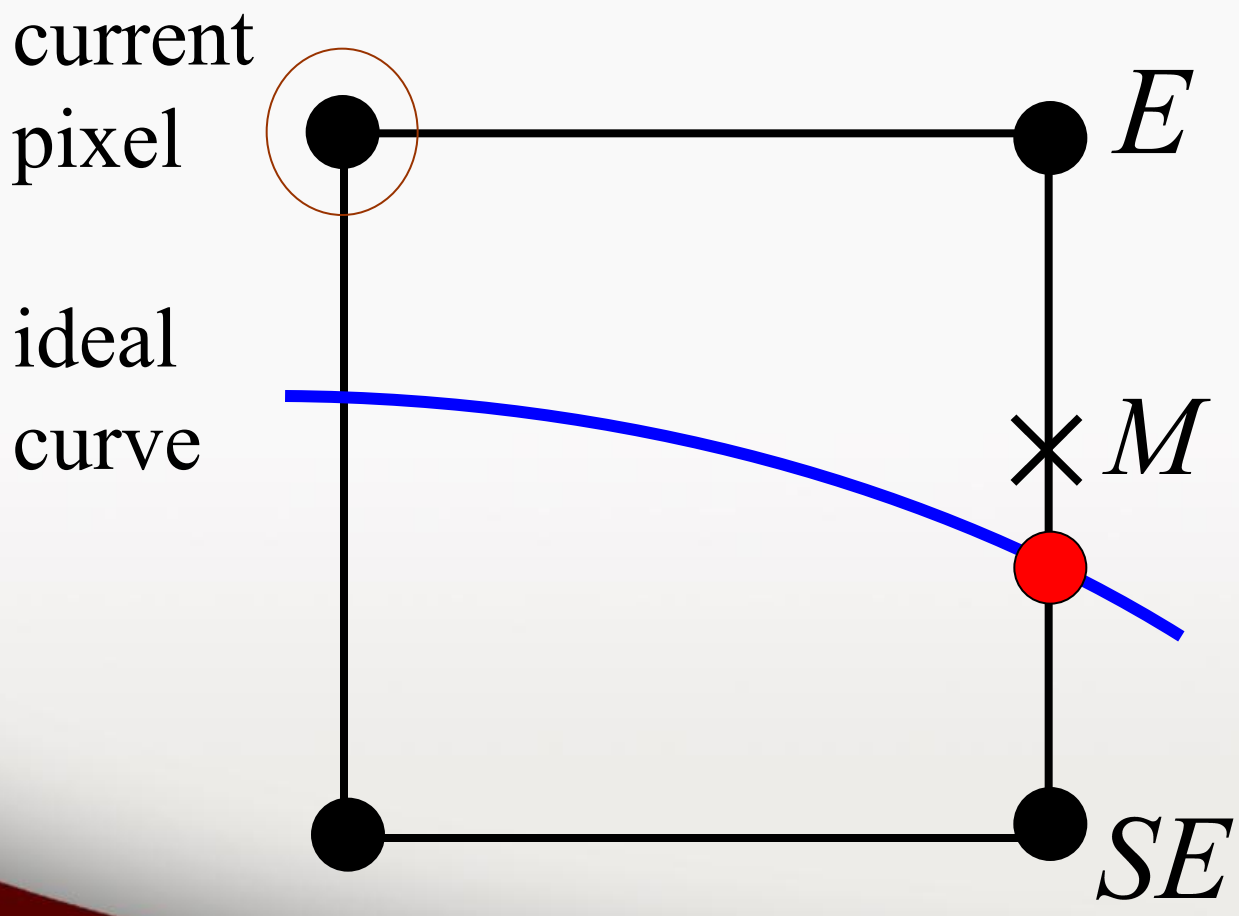
8-way Symmetry of a Circle

If a point (x, y) is found in one octant, the points in the other seven octants can be determined simply by swapping and/or negating the coordinates 12.

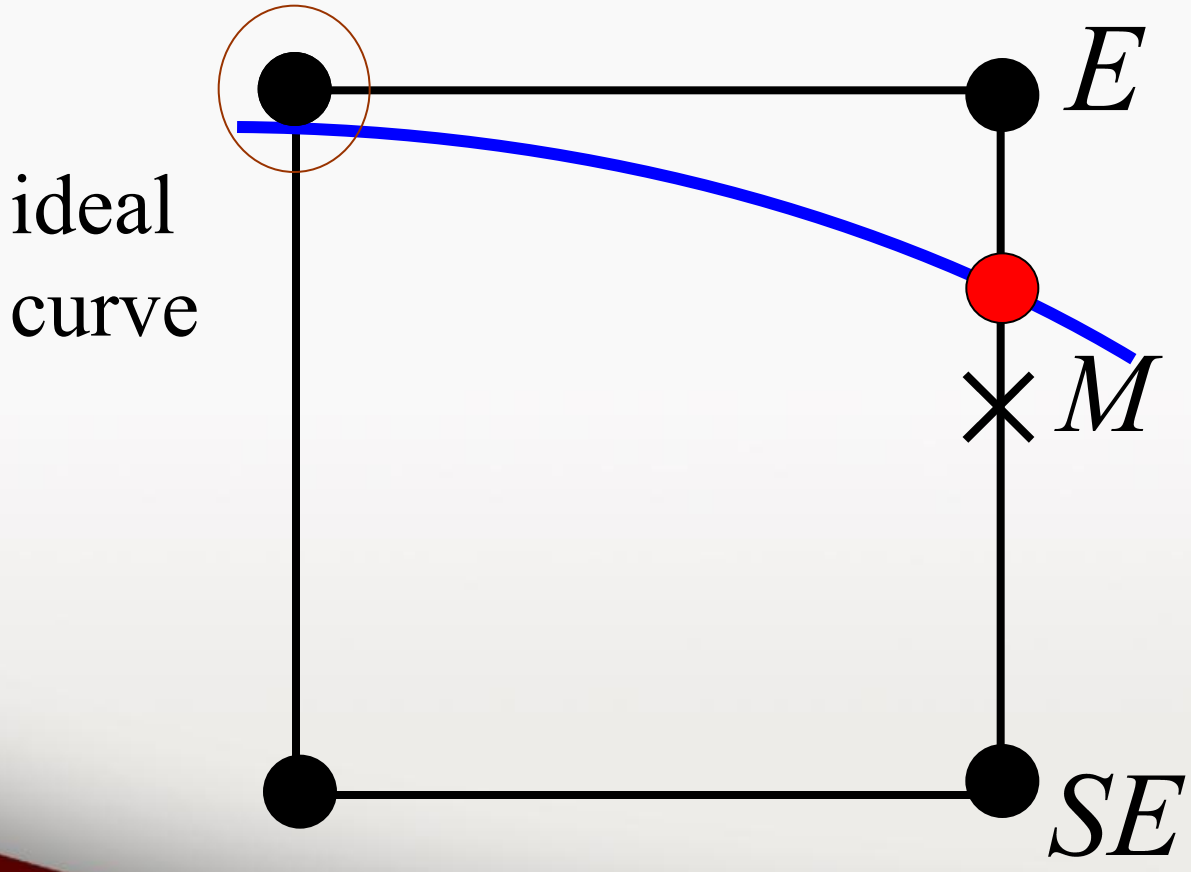



```
void CirclePoints (int x, int y, int value)
{
    WritePixel (x, y, value);
    WritePixel (y, x, value);
    WritePixel (y, −x, value);
    WritePixel (x, −y, value);
    WritePixel (−x, −y, value);
    WritePixel (−y, −x, value);
    WritePixel (−y, x, value);
    WritePixel (−x, y, value);
}   /* CirclePoints */
```

$$F(M) \square 0 \Rightarrow SE$$



$$F(M) < 0 \Rightarrow E$$



Midpoint Circle Algorithm

Assuming we have just plotted the pixel at (x_k, y_k) so we need to choose between (x_k+1, y_k) and (x_k+1, y_k-1)

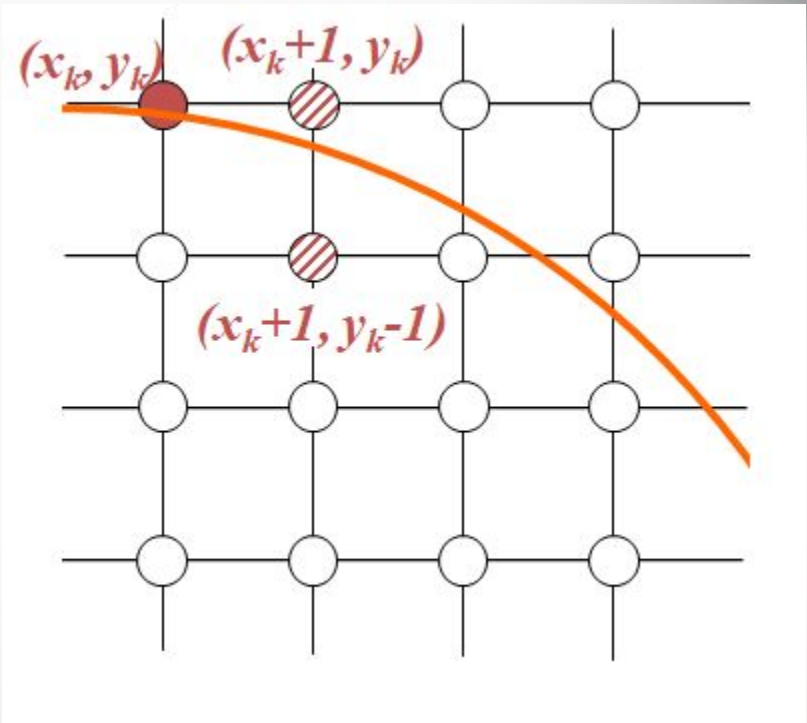
Our **decision variable** can be defined as:

$$p_k = f_{circ}(x_k + 1, y_k - \tfrac{1}{2})$$

$$= (x_k + 1)^2 + (y_k - \tfrac{1}{2})^2 - r^2$$

If $p_k < 0$, the midpoint is inside the circle and the pixel at **y_k is closer to the circle.**

Otherwise **the midpoint is outside and y_k-1 is closer.**

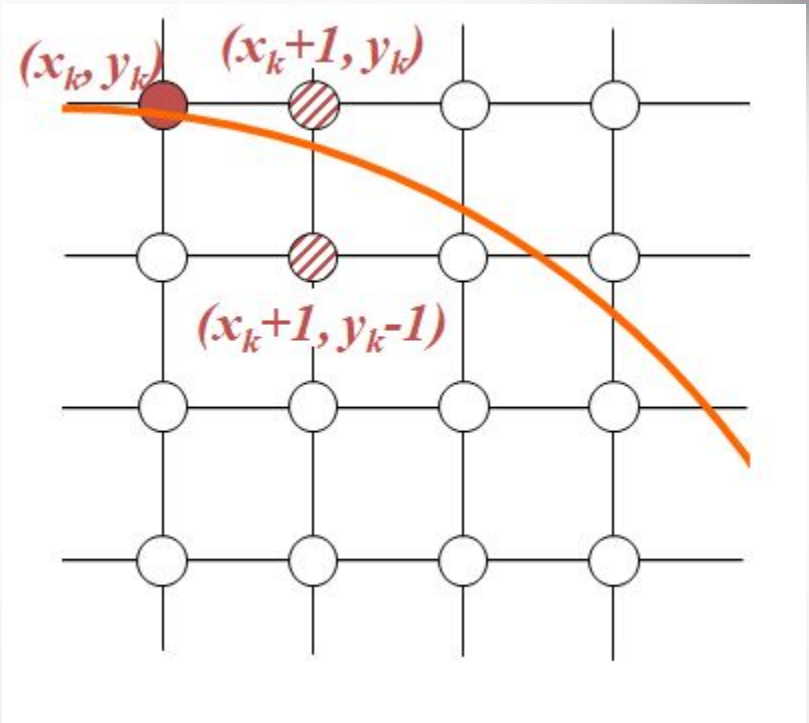


Midpoint Circle Algorithm

Let $p_k = d$

$d_{old} = F(M)$

$d_{old} = F(X_k + 1, Y_k - ½)$



If $d<0$, choose E

$x = x+1$ which gives d_{new}

$d_{new} = (X_k + 1) + 1, (Y_k - ½)$

$(\Delta d)_E = d_{new} - d_{old}$

$F(X_k + 2, Y_k - ½) - F(X_k + 1, Y_k - ½)$

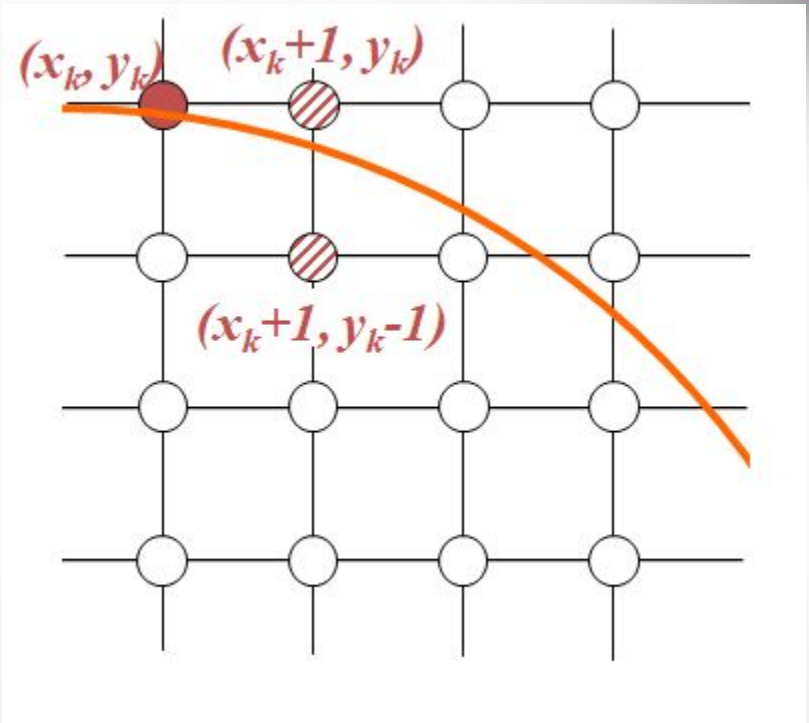
$(\Delta d)_E = 2X_k + 3$

Midpoint Circle Algorithm

Let $p_k = d$

$d_{old} = F(M)$

$d_{old} = F(X_k + 1, Y_k - ½)$



If d>=0, choose SE

x=x+1

y=y-1 which gives d_{new}

$d_{new} = (X_k + 1) + 1, (Y_k - ½) - 1$

$(\Delta d)_{SE} = d_{new} - d_{old}$

$F(X_k + 2, Y_k - 3/2) - F(X_k + 1, Y_k - ½)$

$(\Delta d)SE = 2X_k - 2Y_k + 5$

Initialisation

$x_0 = 0, \quad y_0 = r$

Initial decision variable found by evaluating circle function at first midpoint test position

$$p_0 = F_{circ}(1, r - 1/2)$$

$$= 1 + (r - 1/2)^2 - r^2$$

$$= \frac{5}{4} - r$$

For integer radius r p_0 can be rounded to $p_0 = 1 - r$ since all increments are integer.

Midpoint Circle Algorithm

1. Input radius r and circle center (x_c, y_c), and obtain the first point on the circumference of a circle centered on the origin as

$$(x_0, y_0) = (0, r)$$

2. Calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

3. At each x_k position, starting at $k = 0$, perform the following test: If $p_k < 0$, the next point along the circle centered on $(0, 0)$ is (x_{k+1}, y_k) and

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise, the next point along the circle is $(x_k + 1, y_k - 1)$ and

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$.

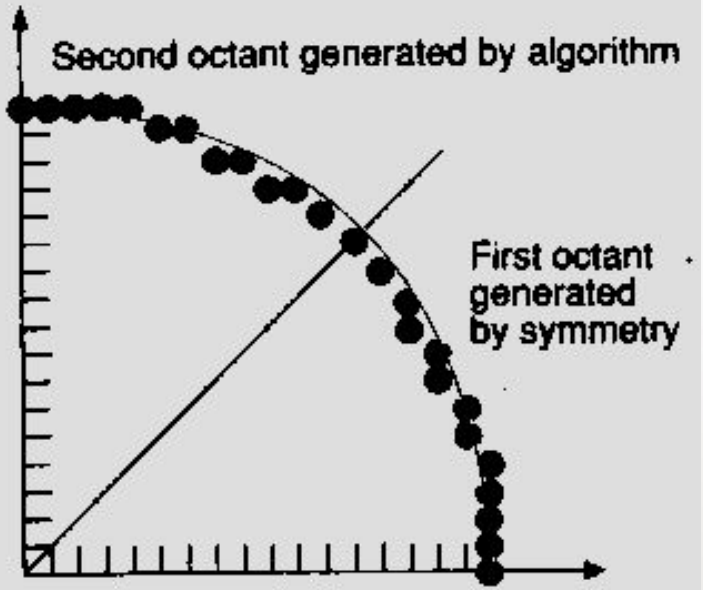
4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values:

$$x = x + x_c \qquad y = y + y_c$$

6. Repeat steps 3 through 5 until $x \geq y$.

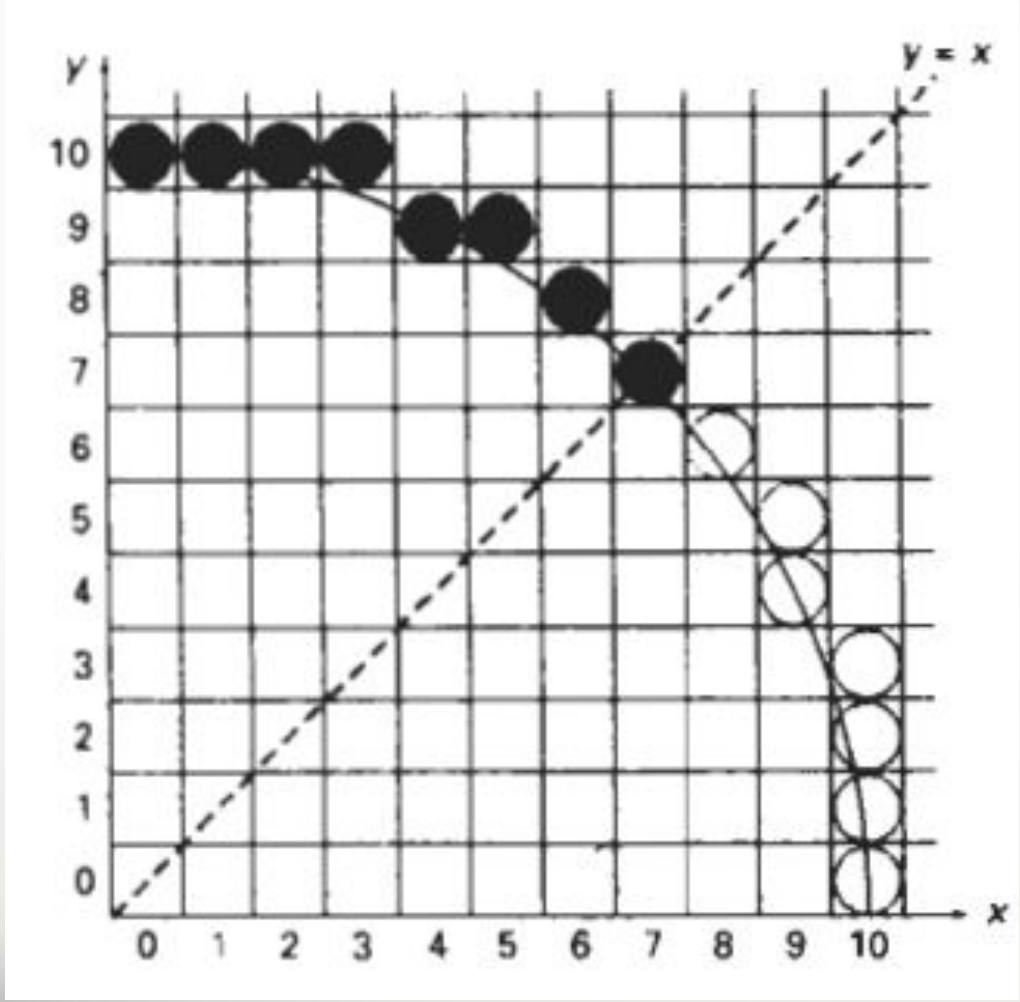
Midpoint Circle Algorithm (cont.)

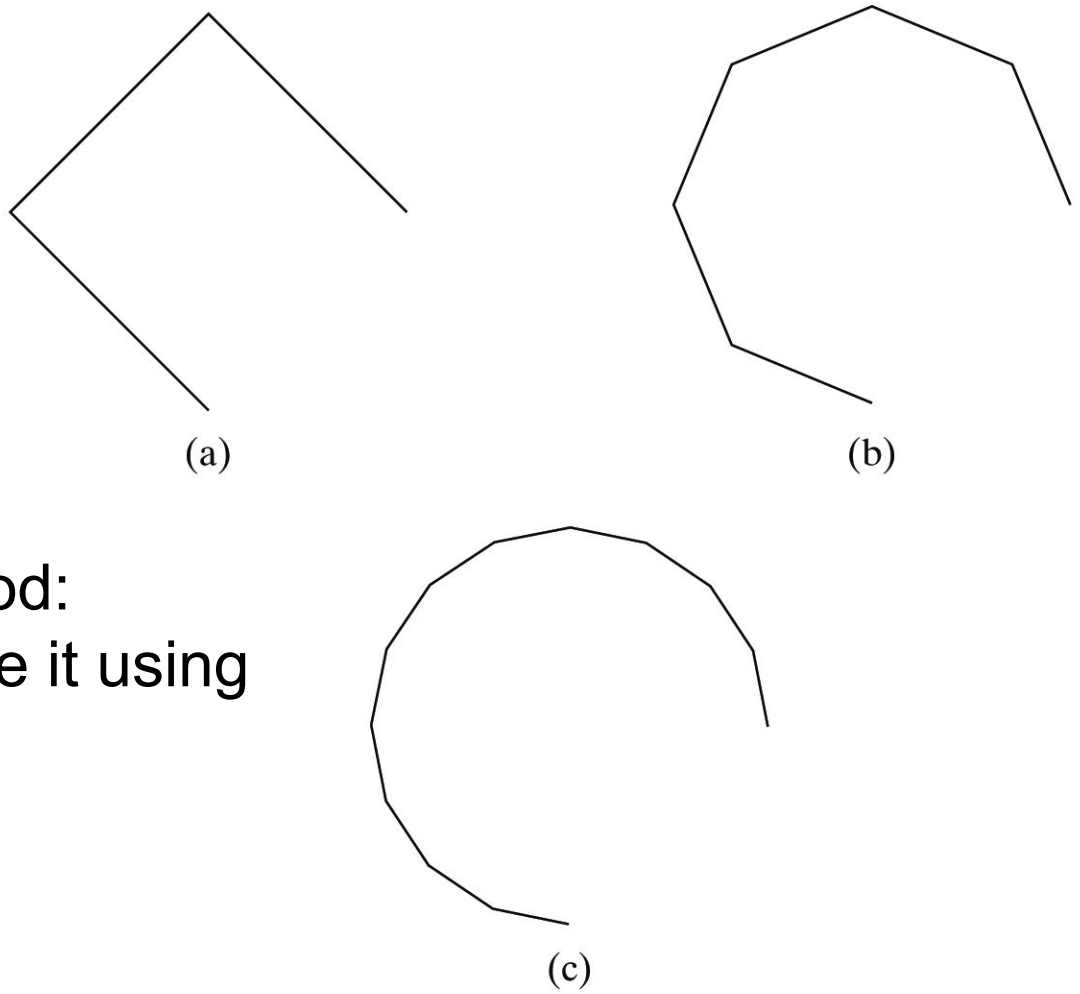
```
void MidpointCircle (int radius, int value)
/* Assumes center of circle is at origin. Integer arithmetic only */
{
    int x = 0;
    int y = radius;
    int d = 1 - radius;
    CirclePoints (x, y, value);

    while (y > x) {
        if (d < 0)              /* Select E */
            d += 2 * x + 3;
        else {                  /* Select SE */
            d += 2 * (x - y) + 5;
            y--;
        }
        x++;
        CirclePoints (x, y, value);
    }  /* while */
}  /* MidpointCircle */
```



Example

- r=10





Another method:

Approximate it using a polyline.

Figure 3-15

A circular arc approximated with (a) three straight-line segments, (b) six line segments, and (c) twelve line segments.

Midpoint Ellipse algorithm

Ellipse function can be defined as:

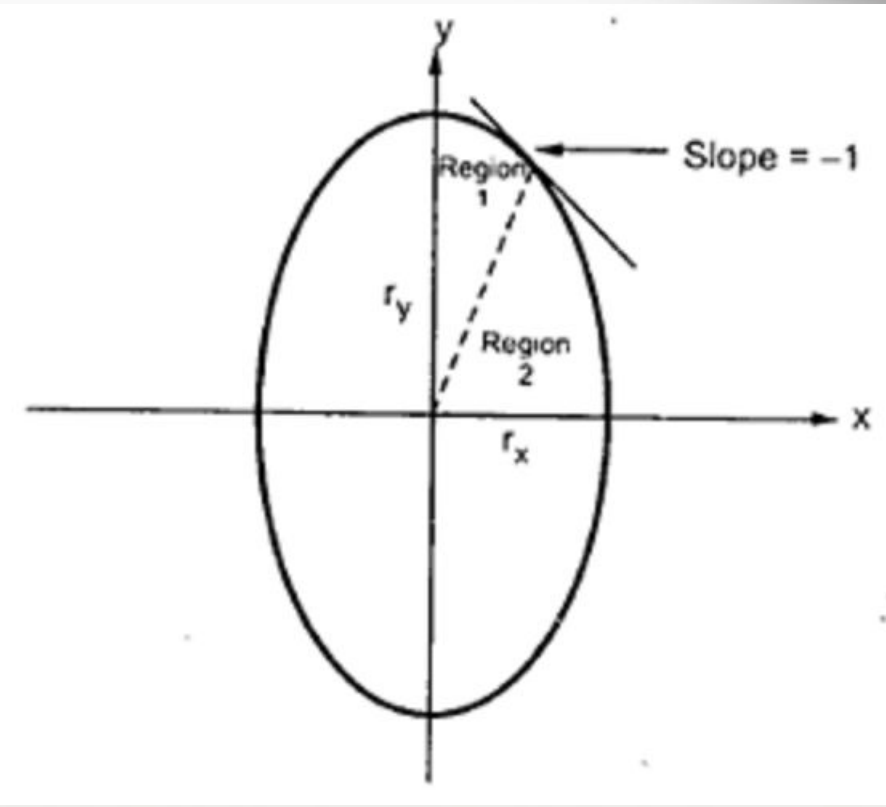
$f_{ellipse}(x,y)=r_y^2x^2+r_x^2y^2-r_x^2r_y^2$

According to this there are some properties which have been generated that are:

$f_{ellipse}(x,y)<0$ which means **(x,y)** is inside the ellipse boundary.

$f_{ellipse}(x,y)>0$ which means **(x,y)** is outside the ellipse boundary.

$f_{ellipse}(x,y)=0$ which means **(x,y)** is on the ellipse boundary.

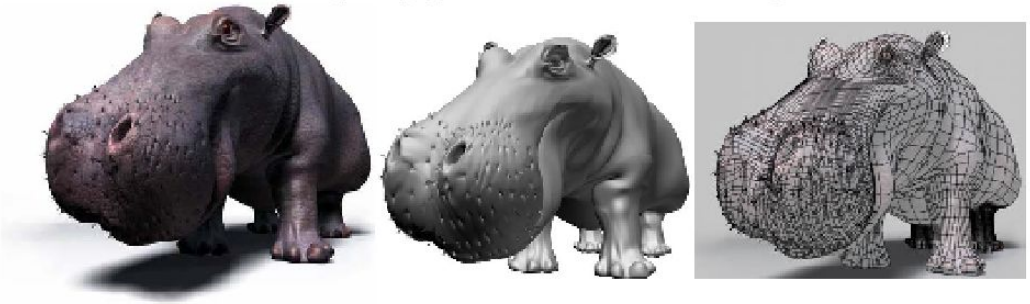


1. Take the input and ellipse centre and obtain the first point on an ellipse centered on the origin as a $(x,y_0)= (0,r_y)$.
2. Now calculate the initial decision parameter in region 1 as:
   $\mathbf{p1_0=r_y^2+1/4r_x^2-r_x^2r_y}$
3. At each $\mathbf{x_k}$ position in region **1** perform the following task. If $\mathbf{p1_k<0}$ then the next point along the ellipse centered on **(0,0)** is $\mathbf{(x_k+1,y_k)}$.
   i.e. $\mathbf{p1_{k+1}=p1_k+^2r_y^2x_{k+1}+r_y^2}$
   Otherwise the next point along the circle is $(x_{k+1},y_k -1)$
   i.e. $\mathbf{p1_{k+1}=p1_k+2r_y^2x_{k+1} – 2r_x^2y_{k+1}+r_y^2}$
4. Now, again calculate the initial value in region 2 using the last point $\mathbf{(x_0,y_0)}$ calculated in a region 1 as : $\mathbf{p2_0=r_y^2(x_0+1/2)^2+r_x^2(y_0-1)^2-r_x^2r_y^2}$
5. At each $\mathbf{y_k}$ position in region 2 starting at $\mathbf{k=0}$ perform the following task.
   If $\mathbf{p2_k<0}$ the next point along the ellipse centered on **(0,0)** is $\mathbf{(x_k , y_{k-1})}$
   i.e. $\mathbf{p2_{k+1}=p2_k-2r_x^2y_{k+1}+r_x^2}$
   Otherwise the next point along the circle will be $\mathbf{(x_{k+1},y_k -1)}$
   i.e. $\mathbf{p2_{k+1} =p2_k+2r_y^2x_{k+1} -2r_x^2y_{k+1}+r_x^2}$
6. Now determine the symmetric points in another three quadrants.
7. Plot the coordinate value as: $\mathbf{x=x+x_c}$, $\mathbf{y=y+y_c}$
8. Repeat the steps for region 1 until $\mathbf{2r_y^2x>=2r_x^2y}$.

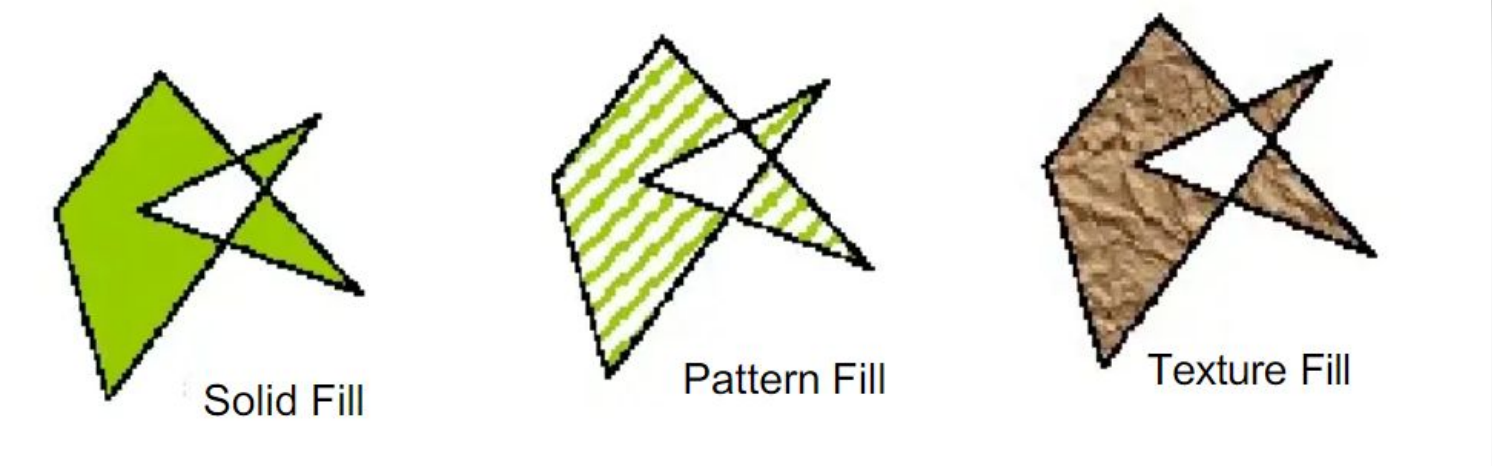
Fill area : an area that is filled with solid colour or pattern

Polygon Fill Areas

- Most library routines require that a fill area be specified as a polygon
  - OpenGL only allows ***convex*** polygons
- Non-polygon (curved) objects can be approximated by polygons
  - Surface tessellation, polygon mesh, triangular mesh



How do we Fill



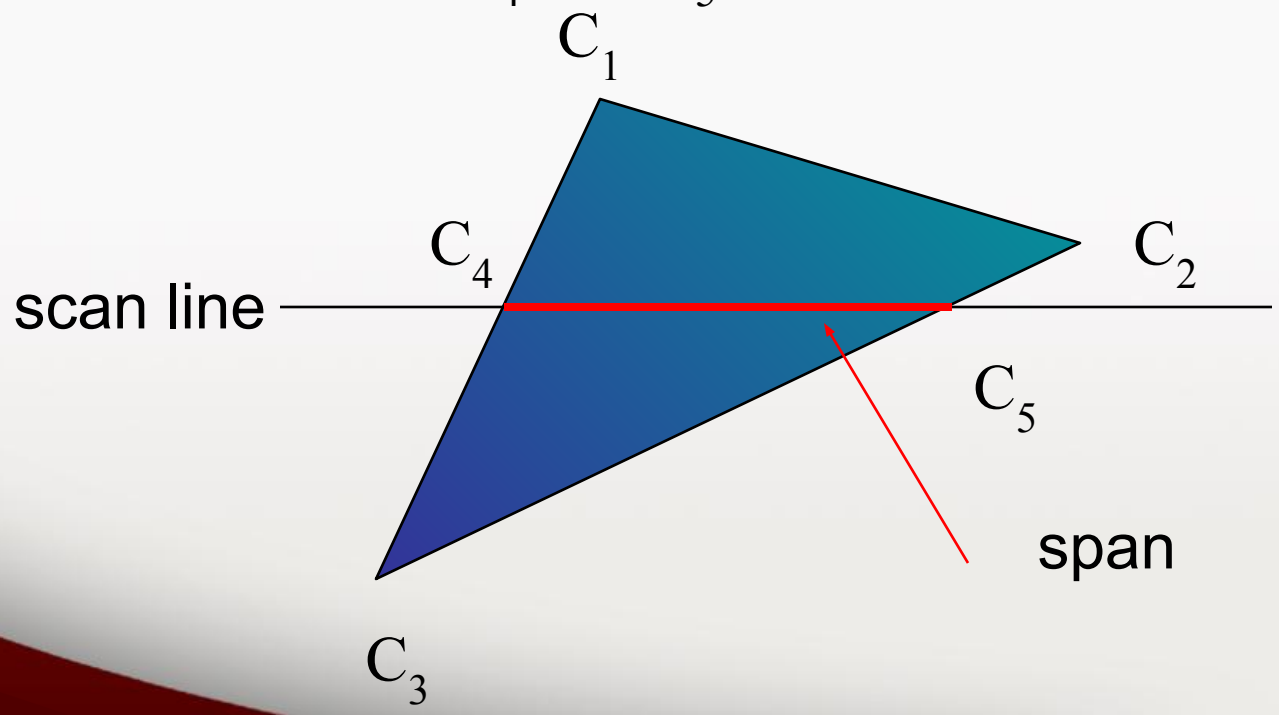
Some requirements are there:

- The shape must be closed
- It must have a well defined inside and outside

- A test for determining if the point is inside
- A procedure for identifying the points inside

Using Interpolation

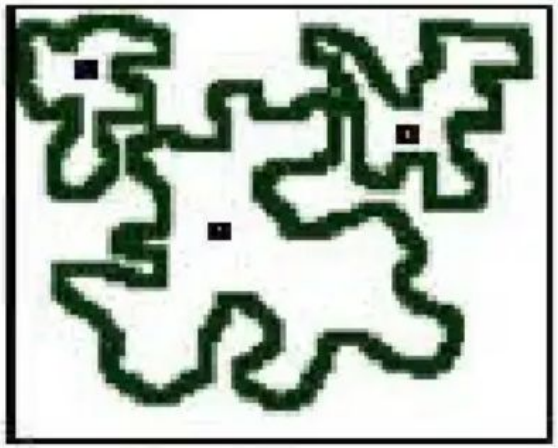
C_1 C_2 C_3 specified by **glColor** or by vertex shading
C_4 determined by interpolating between C_1 and C_3
C_5 determined by interpolating between C_2 and C_3
interpolate between C_4 and C_5 along span



Digital Images

Original Image

Pink pixels have been filled with vellow



Digital outline and seed points

Filled outlines

Filled Area Primitives

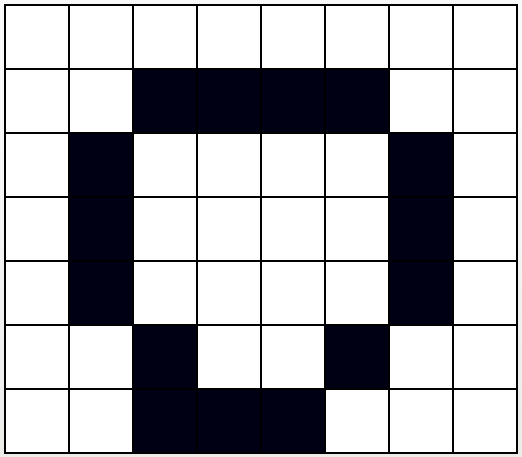
- Area filling is a method or process that helps us to fill an object, area, or image
- Pixels appear inside the polygon with any color other than the background color
- There are two algorithms or methods used to fill the polygon.
  - **Seed Fill Algorithm**
  - **Scan Line Algorithm**

Seed Fill Algorithm

- In this method, we will select a seed or starting point inside the boundary.
- This is further divided into two parts.
  1. Flood-fill Algorithm
  2. Boundary-fill Algorithm

Flood Fill Algorithm

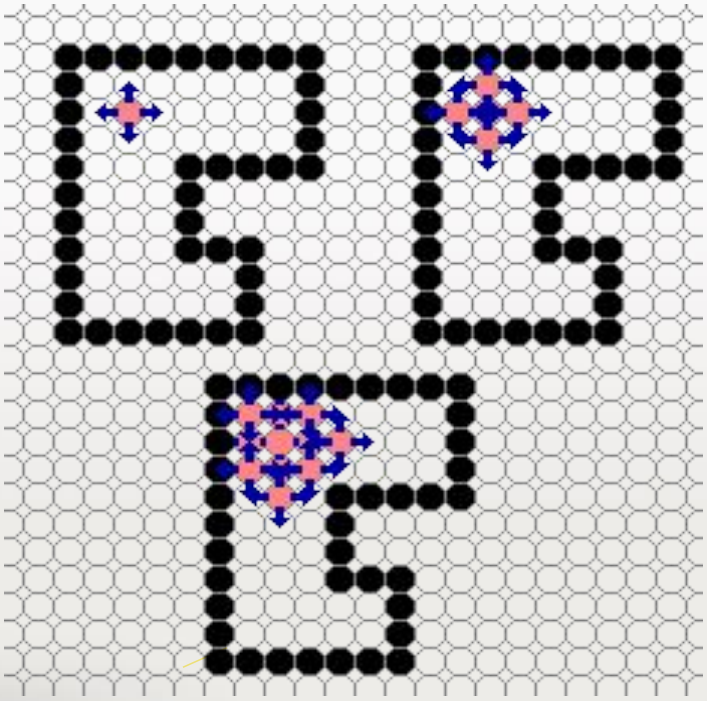
- These algorithms assume that at least one pixel interior to a polygon or region is known
  -assume that the boundary is defined by Bresenham's



Flood Fill – 4 connect

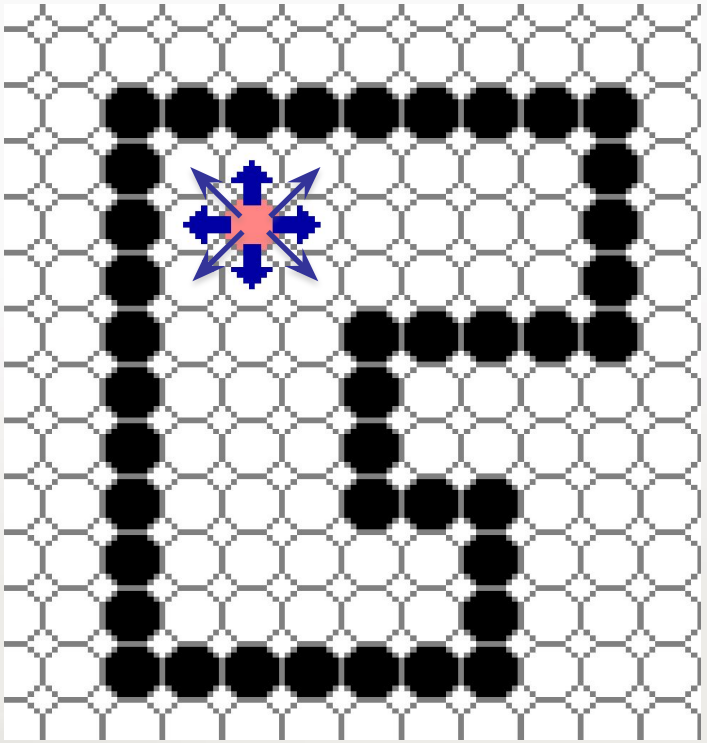
- Fill can be done recursively if we know a seed point (x,y) located inside (WHITE)
- Scan convert edges into buffer in edge/inside color (BLACK)

```
flood_fill(int x, int y) {
    if(read_pixel(x,y)= =
WHITE) {

write_pixel(x,y,BLACK);
        flood_fill(x-1, y);
        flood_fill(x+1, y);
        flood_fill(x, y+1);
        flood_fill(x, y-1);
}
```



Flood Fill – 8 connect

```
flood_fill(int x, int y) {
    if(read_pixel(x,y)= = WHITE) {
        write_pixel(x,y,BLACK);
        flood_fill(x-1, y);
        flood_fill(x+1, y);
        flood_fill(x, y+1);
        flood_fill(x, y-1);
      flood_fill(x+1, y+1);
      flood_fill(x+1, y-1);
      flood_fill(x-1, y+1);
      flood_fill(x-1, y-1);

}
```



Flood Fill Algorithm

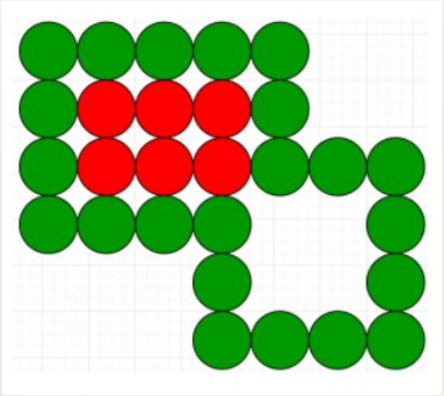
- we replace all the associated pixels of the selected color with a fill color
- We also check the pixels for a particular interior color,

Advantages of Flood-fill algorithm

- It provides an easy way to fill color in graphics.
- The Flood-fill algorithm colors the whole area through interconnected pixels by a single color.
- The algorithm fills the same color inside the boundary.

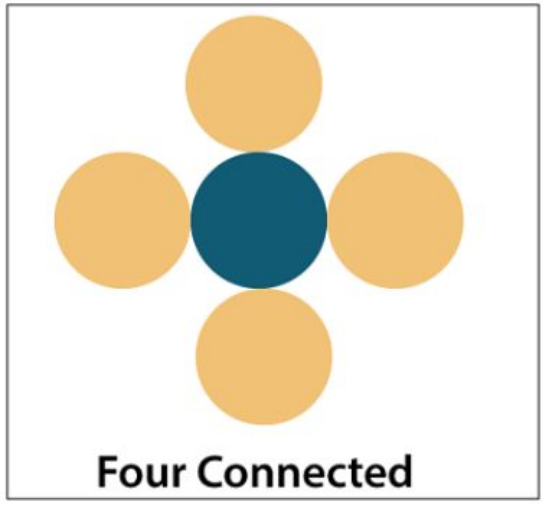
Disadvantages of Flood-fill Algorithm

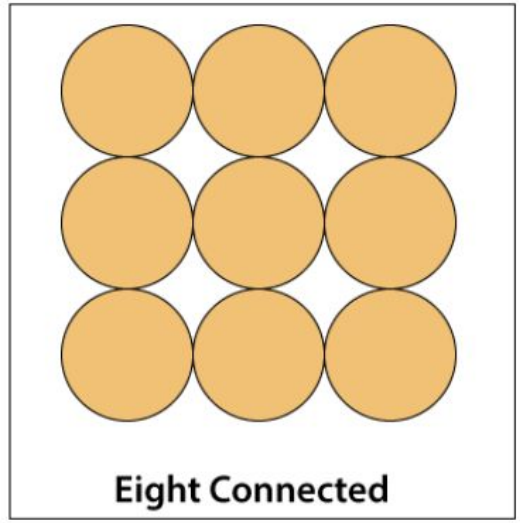
- It is a more time-consuming algorithm.
- Sometimes it does not work on large polygons.
- not for boundary color.



Boundary-fill Algorithm

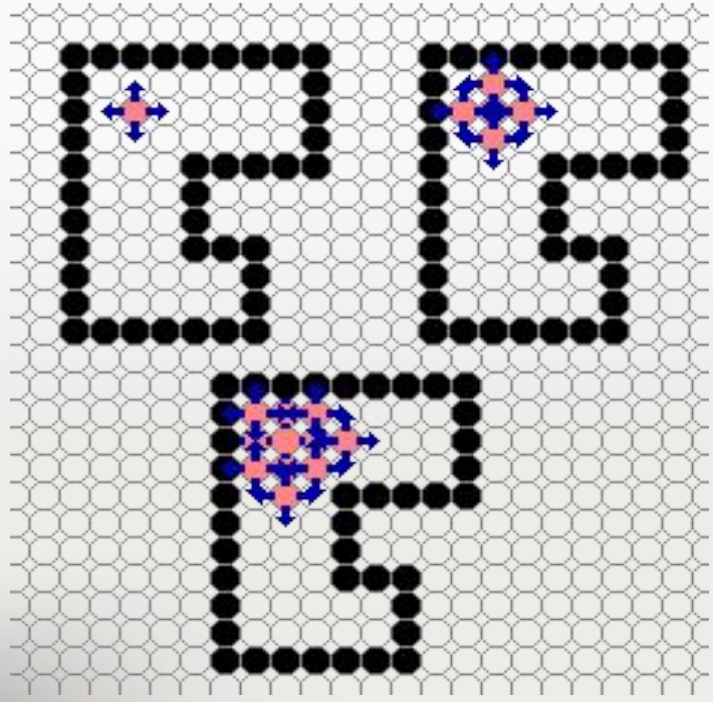
- It is also known as the "**Edge-fill algorithm.**"
- It is used for area filling
- The object has a particular boundary in a single color, then the algorithm travels each pixel until it reaches the boundary.
- In the Boundary-fill algorithm, there are **4-connected** and **8-connected** methods.





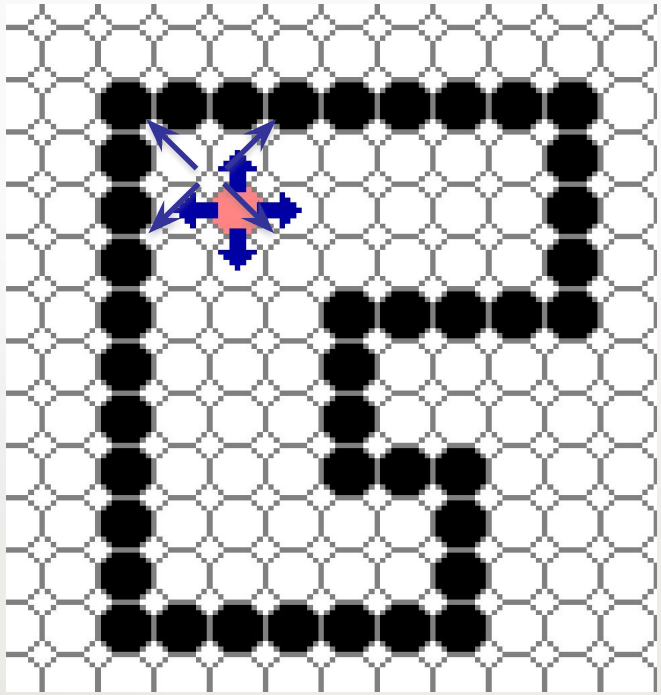
Boundary-fill – 4 connect


```
void boundaryFill4(int x, int y, int fill_color,int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
       getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);
        boundaryFill4(x + 1, y, fill_color, boundary_color);
        boundaryFill4(x, y + 1, fill_color, boundary_color);
        boundaryFill4(x - 1, y, fill_color, boundary_color);
        boundaryFill4(x, y - 1, fill_color, boundary_color);
    }
}
```



Boundary-fill – 8 connect

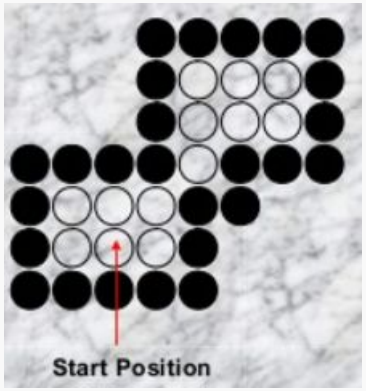

```
void boundaryFill8(int x, int y, int fill_color,int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
       getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);
        boundaryFill8(x + 1, y, fill_color, boundary_color);
        boundaryFill8(x, y + 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y, fill_color, boundary_color);
        boundaryFill8(x, y - 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y - 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y + 1, fill_color, boundary_color);
        boundaryFill8(x + 1, y - 1, fill_color, boundary_color);
        boundaryFill8(x + 1, y + 1, fill_color, boundary_color);
    }
```

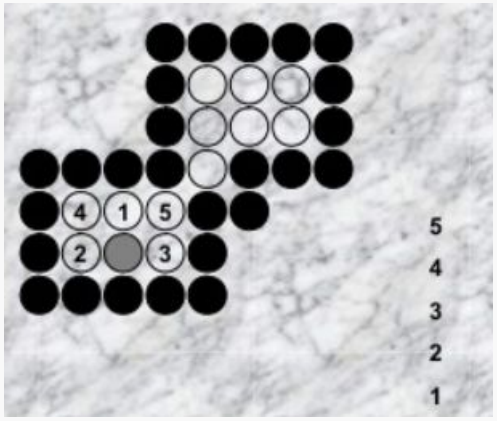


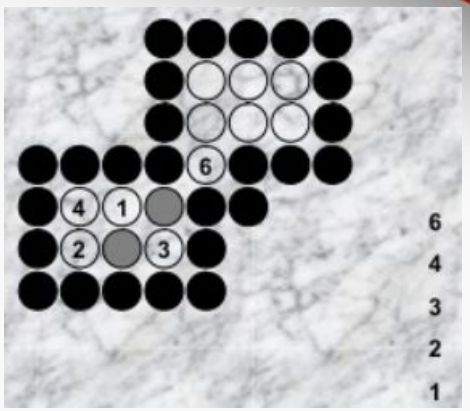
FLOOD-FILL ALGORITHM	BOUNDARY-FILL ALGORITHM
It can process the image containing more than one boundary colours.	It can only process the image containing single boundary colour.
Flood-fill algorithm is comparatively slower than the Boundary-fill algorithm.	Boundary-fill algorithm is faster than the Flood-fill algorithm.
In Flood-fill algorithm a random colour can be used to paint the interior portion then the old one is replaced with a new one.	In Boundary-fill algorithm Interior points are painted by continuously searching for the boundary colour.
It requires huge amount of memory.	Memory consumption is relatively low in Boundary-fill algorithm.
Flood-fill algorithms are simple and efficient.	The complexity of Bounndary-fill algorithm is high.

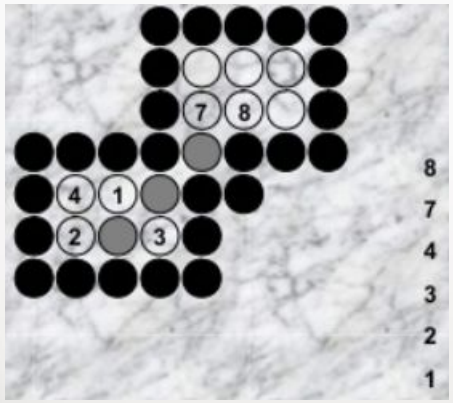
Flood Fill – 8 connect

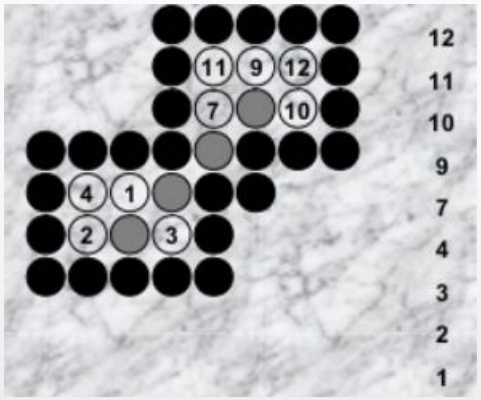
```
flood_fill(int x, int y) {
    if(read_pixel(x,y)= = WHITE…) {
 write_pixel(x,y,BLACK);
 flood_fill(x, y+1);
 flood_fill(x-1, y);
 flood_fill(x+1, y);
 flood_fill(x, y-1);
 flood_fill(x-1, y+1);
 flood_fill(x-1, y-1);
 flood_fill(x+1, y+1);
 flood_fill(x+1, y-1);
   }
}
```

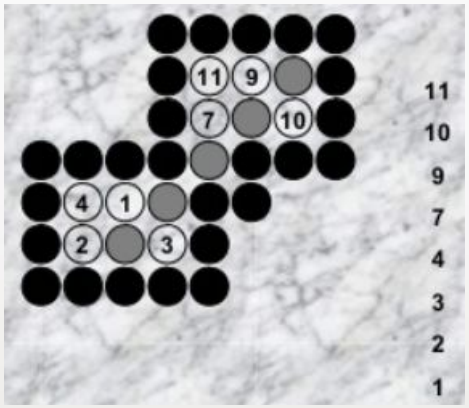

Start Position

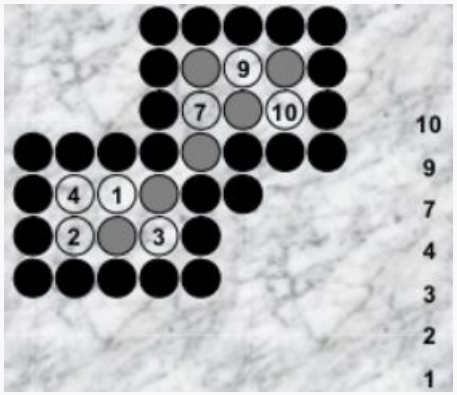

4 1 5
2 3
5
4
3
2
1

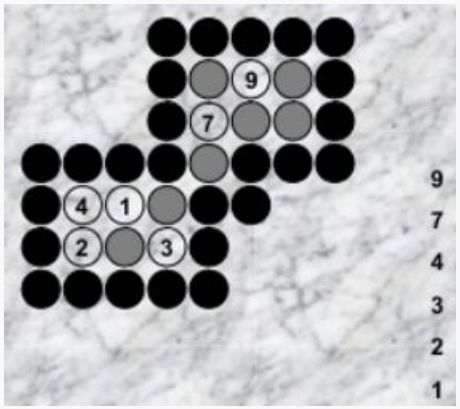

6
4 1
2 3
6
4
3
2
1

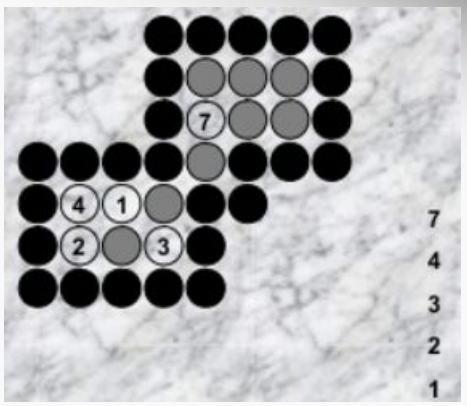

7 8
4 1
2 3
8
7
4
3
2
1

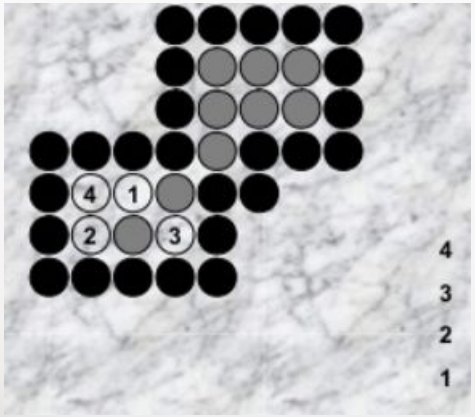

11 9 12
7 10
4 1
2 3
12
11
10
9
7
4
3
2
1

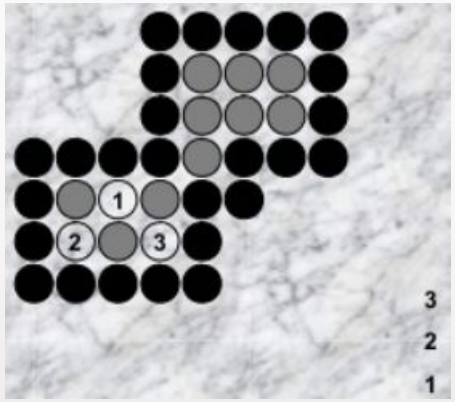

11 9
7 10
4 1
2 3
11
10
9
7
4
3
2
1

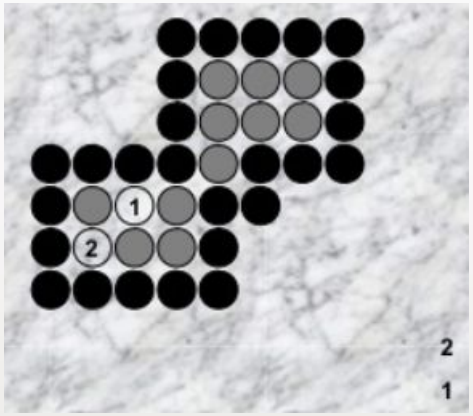

9
7
10
4
1
2
3
10
9
7
4
3
2
1

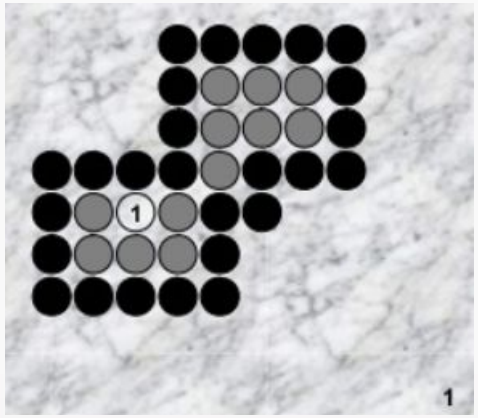

9
7
4
1
2
3
9
7
4
3
2
1

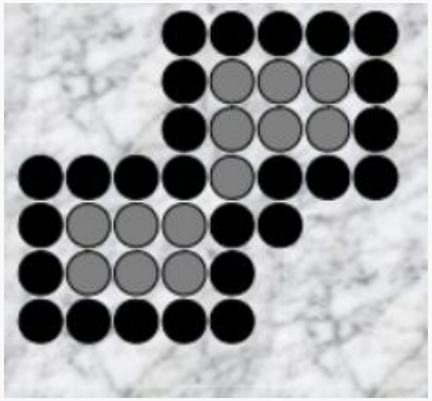

7
4
1
2
3
7
4
3
2
1


4
1
2
3
4
3
2
1


1
2
3
3
2
1

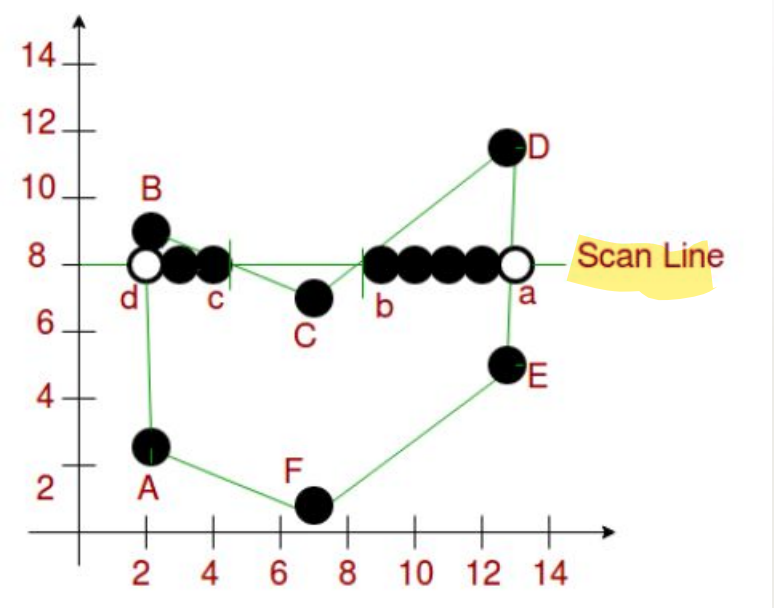

1
2
2
1





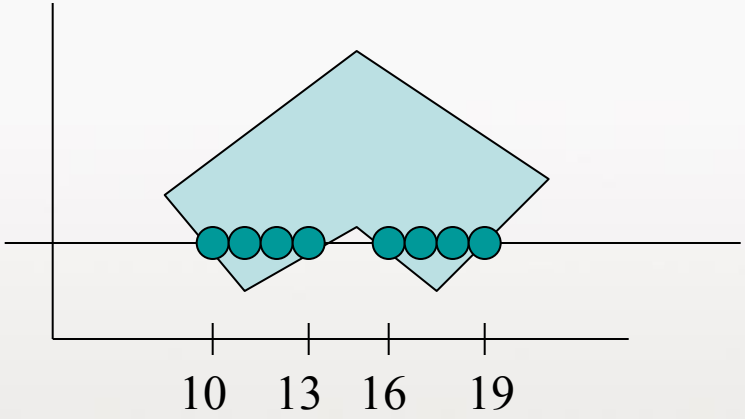
Scan Line Polygon Fill

- Scanline filling is basically filling up of polygons using horizontal lines or scanlines.
- Image drawn by a single pen starting from bottom left, continuing to the right, plotting only points where there is a point present in the image, and when the line is complete, start from the next line and continue.
- This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections.

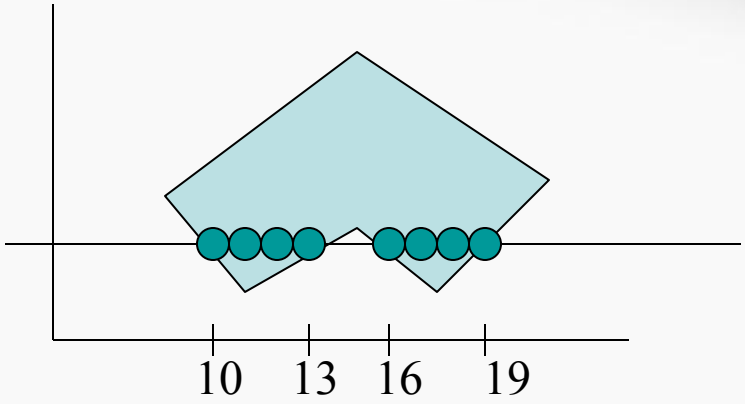


Scan Line Polygon Fill

- Determine overlap Intervals for scan lines that cross that area.
  - Requires determining intersection positions of the edges of the polygon with the scan lines
  - Fill colors are applied to each section of the scanline that lies within the interior of the region.
  - Interior regions determined as in odd-even test



Interior pixels along a scan line passing through a polygon area

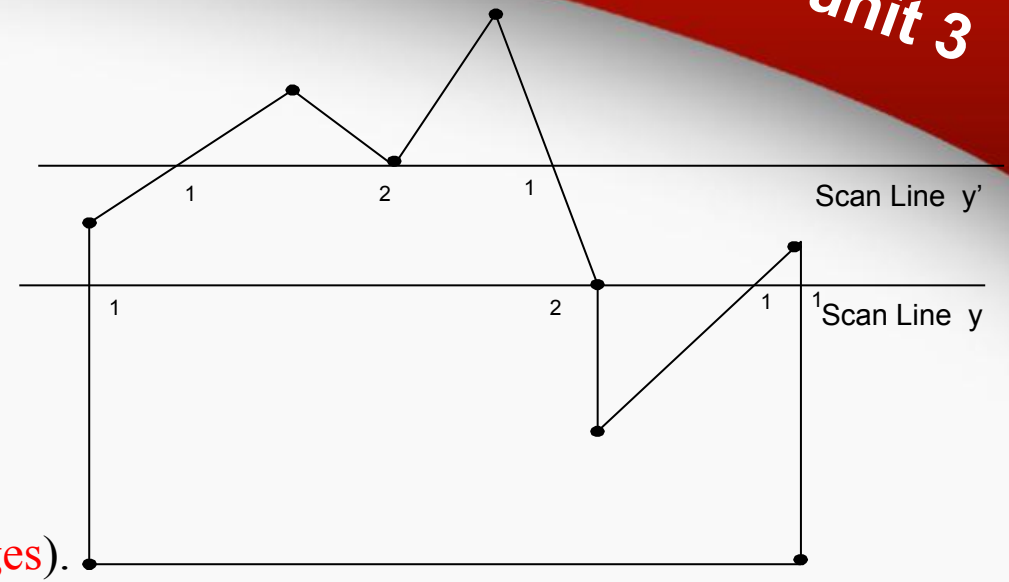


- In the given example, four pixel intersections are at x=10, x=13, x=16 and x=19

- These intersection points are then sorted from left to right , and the corresponding frame buffer positions between each intersection pair are set to specified color
  - 10 to 13, and 16 to 19

Scan Line Polygon Fill

- Some scan line intersections at polygon vertices require special handling:
  - A scan line passing through a vertex intersects two polygon edges at that position



Intersection points along the scan lines that intersect polygon vertices.

line y' properties:

- Scan line y' generates an even number of intersections (crosses even number of edges).
- The two intersecting edges are both above the scan line
- We can pair the intersections to identify correctly the interior pixel spans by counting the vertex intersection two points.

line y properties:

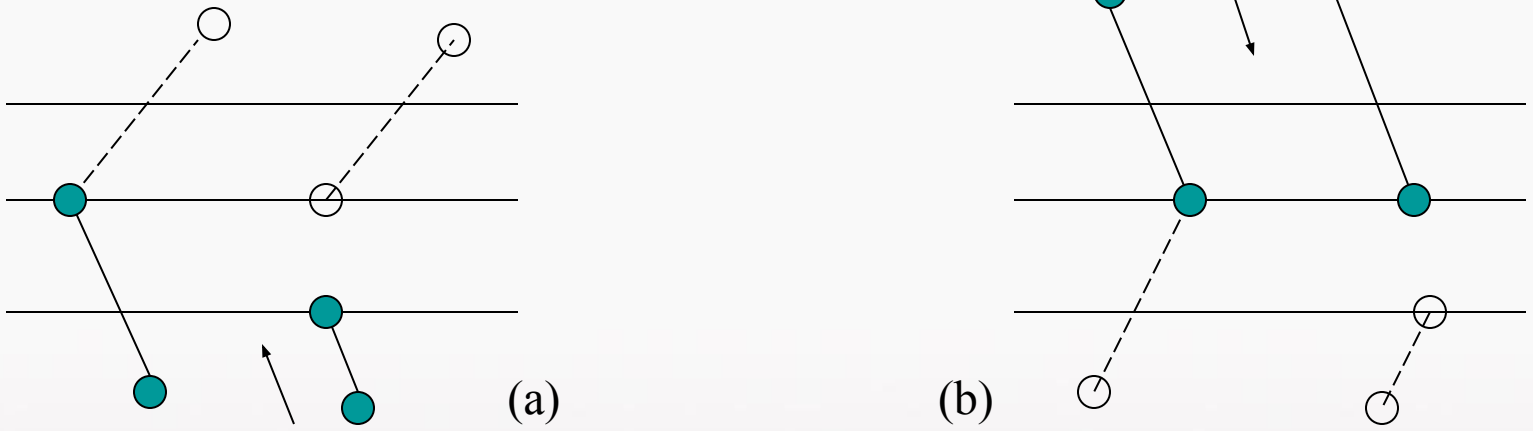
- Scan line y crosses odd number of edges.
- The two intersecting edges sharing a vertex are on opposite sides of the scan line
- To identify the interior pixels for scan line y, we must count the vertex intersection as only one point.

Scan Line Polygon Fill

- We can distinguish these cases by tracing around the polygon boundary either in clockwise or counterclockwise order and observing the relative changes in vertex y coordinates as we move from one edge to the next.

- Let (y1, y2) and (y2, y3) be the endpoint y values of two consecutive edges. If y1, y2, y3 monotonically increase or decrease, we need to count the middle vertex as a single intersection point for any scan line passing through that vertex.

Scan Line Polygon Fill Algorithm



Adjusting endpoint values for a polygon, as we process edges in order around the polygon perimeter. The edge currently being processed is indicated as a solid line. In (a), the y coordinate of the upper endpoint of the current edge is decreased by 1. In (b), the y coordinate of the upper end point of the next edge is decreased by 1

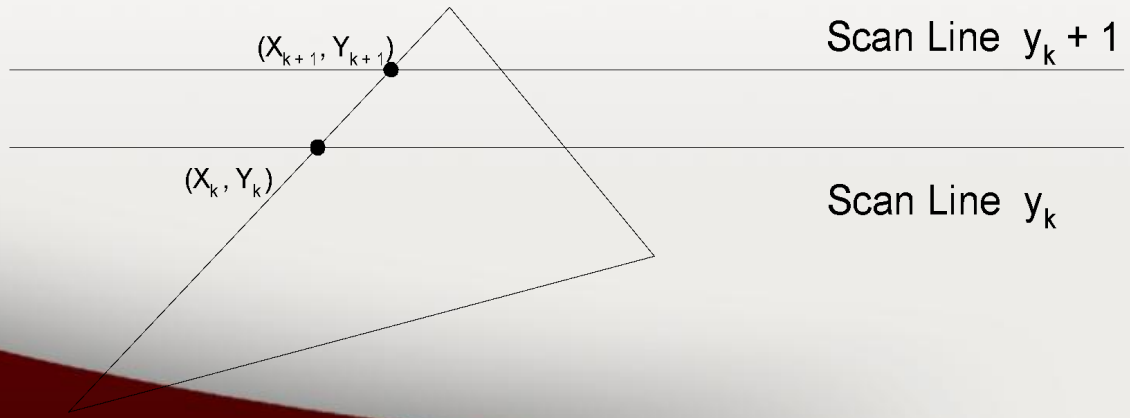
The scan conversion algorithm works as follows

i. Intersect each scanline with all edges
ii. Sort intersections in x
iii. Calculate parity of intersections to determine in/out
iv. Fill the "in" pixels

Special cases to be handled:

v. Horizontal edges should be excluded
vi. Vertices lying on scanlines handled by shortening of edges,

- Coherence between scanlines tells us that
  - Edges that intersect scanline y are likely to intersect $y + 1$
  - X changes predictably from scanline y to $y + 1$ *(Incremental Calculation Possible)*



- The slope of the edge is constant from one scan line to the next:
  - let m denote the slope of the edge.

$$y_{k+1} - y_k = 1$$

$$x_{k+1} = x_k + \frac{1}{m}$$

- Each successive x is computed by adding the inverse of the slope and rounding to the nearest integer

Integer operations

- Recall that slope is the ratio of two integers:

$$m = \frac{\Delta y}{\Delta x}$$

- So, incremental calculation of x can be expressed as

$$x_{k+1} = x_k + \frac{\Delta y}{\Delta x}$$

LAB Program-4: Open GL Program for Polygon Fill using Scan Line Algorithm

```
void display()
{
x1=200.0;y1=200.0;x2=100.0;y2=300.0;x3=200.0;y3=400.0;x4=300.0;y4=30
0.0;
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(0.0, 0.0, 1.0);
glBegin(GL_LINE_LOOP);
      glVertex2f(x1,y1);
      glVertex2f(x2,y2);
      glVertex2f(x3,y3);
      glVertex2f(x4,y4);
glEnd();
scanfill(x1,y1,x2,y2,x3,y3,x4,y4);
glFlush();
}
```

```
void scanfill(float x1,float y1,float x2,float y2,float x3,float y3,float
x4,float y4)
{
    int le[500],re[500],i,y;
    for(i=0;i<500;i++)
    {
        le[i]=500;
        re[i]=0;
    }
    edgedetect(x1,y1,x2,y2,le,re);
    edgedetect(x2,y2,x3,y3,le,re);
    edgedetect(x3,y3,x4,y4,le,re);
    edgedetect(x4,y4,x1,y1,le,re);
    for(y=0;y<500;y++)
    {
        if(le[y]<=re[y])
        for(i=(int)le[y];i<(int)re[y];i++)
        draw_pixel(i,y);
    }
```

initialized to rightmost point

initialized to leftmost point

Iterates through each edge of the polygon.
For every scanline y it crosses, it calculates the x-intersection poin
and updates the minimum value in le[y] and the maximum value i
re[y]

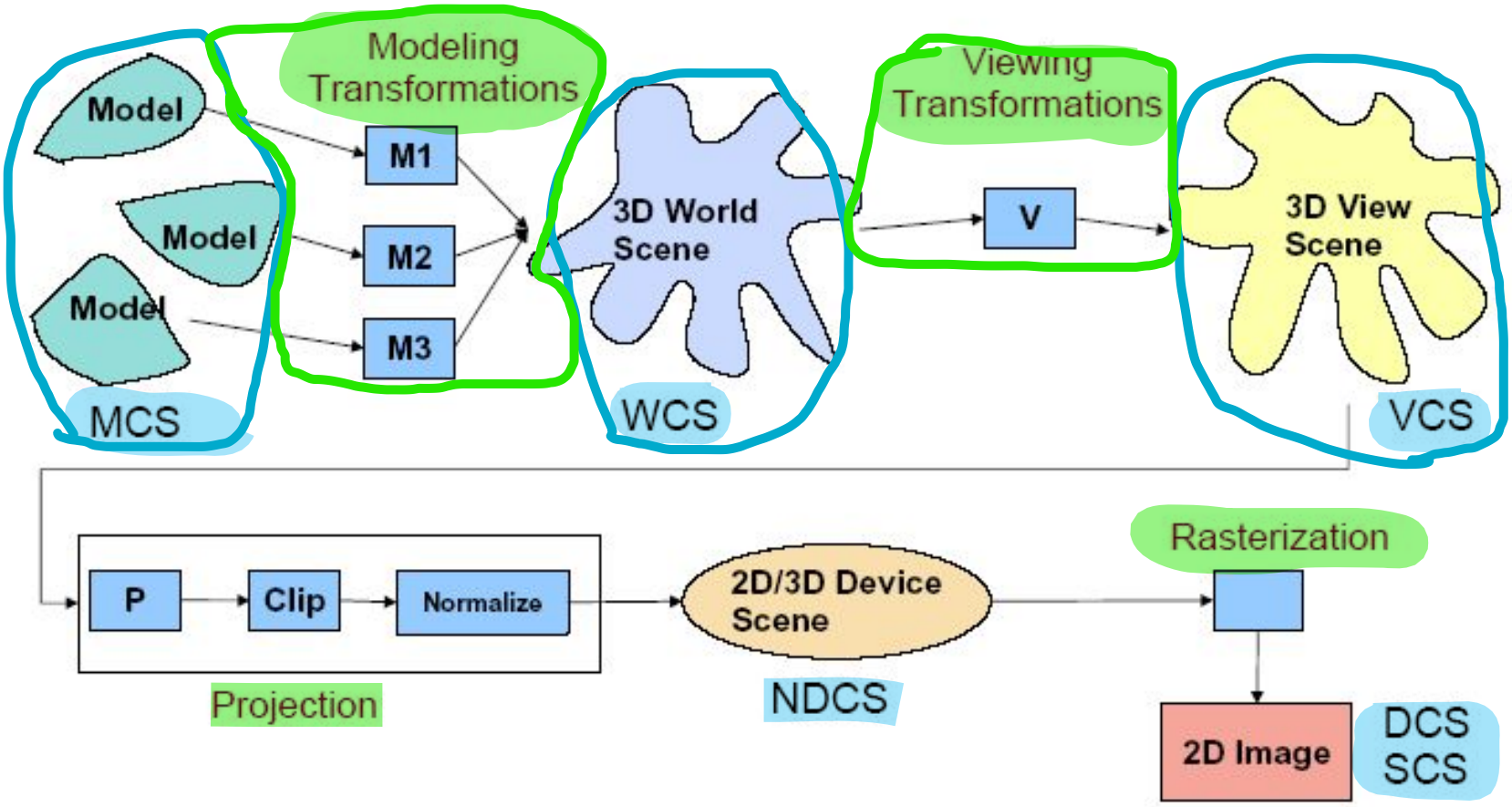
```
void edgedetect(float x1,float y1,float x2,float y2,int *le,int *re)
{
    float mx,x,temp;
    int i;
    if((y2-y1)<0)
    {
        temp=y1;y1=y2;y2=temp;
        temp=x1;x1=x2;x2=temp;
    }

    if((y2-y1)!=0)
    mx=(x2-x1)/(y2-y1);
    else  mx=x2-x1;

    x=x1;
    for(i=y1;i<=y2;i++)
    {
        if(x<(float)le[i])
        le[i]=(int)x;
        if(x>(float)re[i])
        re[i]=(int)x;
        x+=mx;
    }
```

edge is always processed from lower y-coordinate to the higher y-coordinate .

SWAP

```
void draw_pixel(int x,int y)
{
    glColor3f(1.0,0.0,0.0);
    Sleep(1); // To set the delay time
    glBegin(GL_POINTS);
    glVertex2i(x,y);
    glEnd();
    glFlush();
}
```

Viewing Pipeline Revisited



- **Model coordinates to World coordinates:**
  Modelling transformations

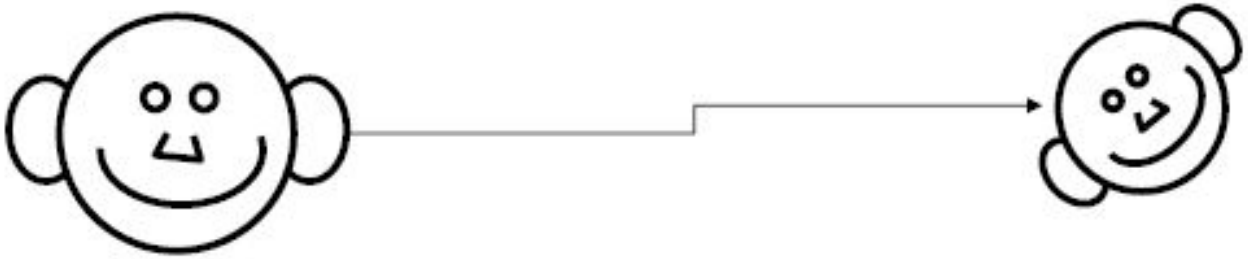
Model coordinates:
1 circle (head),
2 circles (eyes),
1 line group (nose),
1 arc (mouth),
2 arcs (ears).
With their relative coordinates and sizes

World coordinates:
All shapes with their absolute coordinates and sizes.
circle(0,0,2)
circle(-.6,.8,.3) circle(.6,.8,.3)
lines[(-.4,0),(-.5,-.3),(.5,.3),(.4,0)]
arc(-.6,0,.6,0,1.8,180,360)
arc(-2.2,.2,-2.2,-.2,.8,45,315)
arc(2.2,.2,2.2,-.2,.8,225,135)

- World coordinates to Viewing coordinates: Viewing transformations

World coordinates

Viewing coordinates:
Viewers position and view angle. i.e. rotated/translated



- Projection: 3D to 2D. Clipping depends on viewing frame/volume. Normalization: device independent coordinates

Viewing coordinates:

Device Independent Coordinates:
Invisible shapes deleted, others reduced to visible parts.
3 arcs, 1 circle, 1 line group

2-D viewing transformation pipeline

Model coordinates

Construct world coordinate scene using modeling coordinate transformations →

World coordinates

Convert world coordinates to viewing coordinates →

Viewing coordinates

Transform viewing coordinates to normalised coordinates →

Normalized coordinates

Map normalized coordinates to device coordinates →

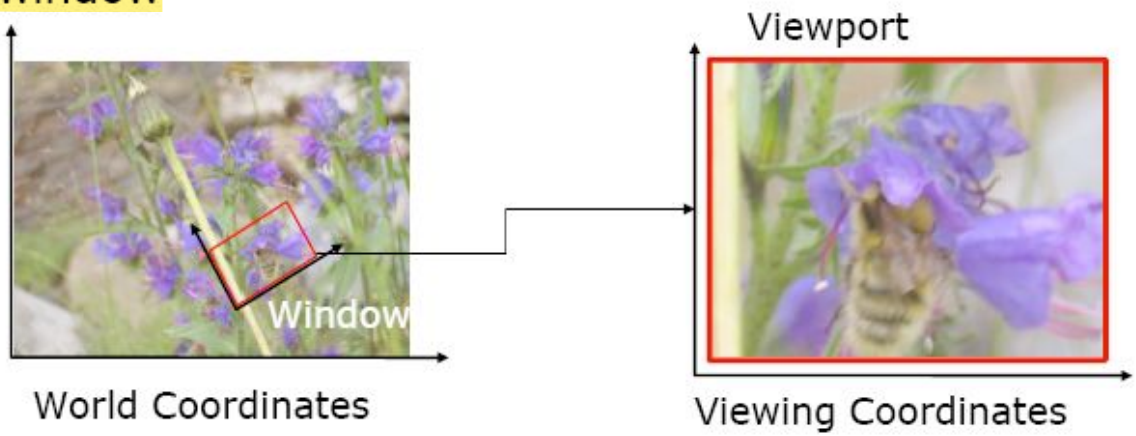
Device Coordinates

2D Viewing

- World coordinates to Viewing coordinates
- Window to Viewport.

Window: A region of the scene selected for viewing (also called *clipping window*)

Viewport: A region on display device for mapping to window



Graphics packages commonly allow only rectangular clipping windows aligned with the x- and y-axes

Must implement own clipping and coordinate transformations for other form of clipping

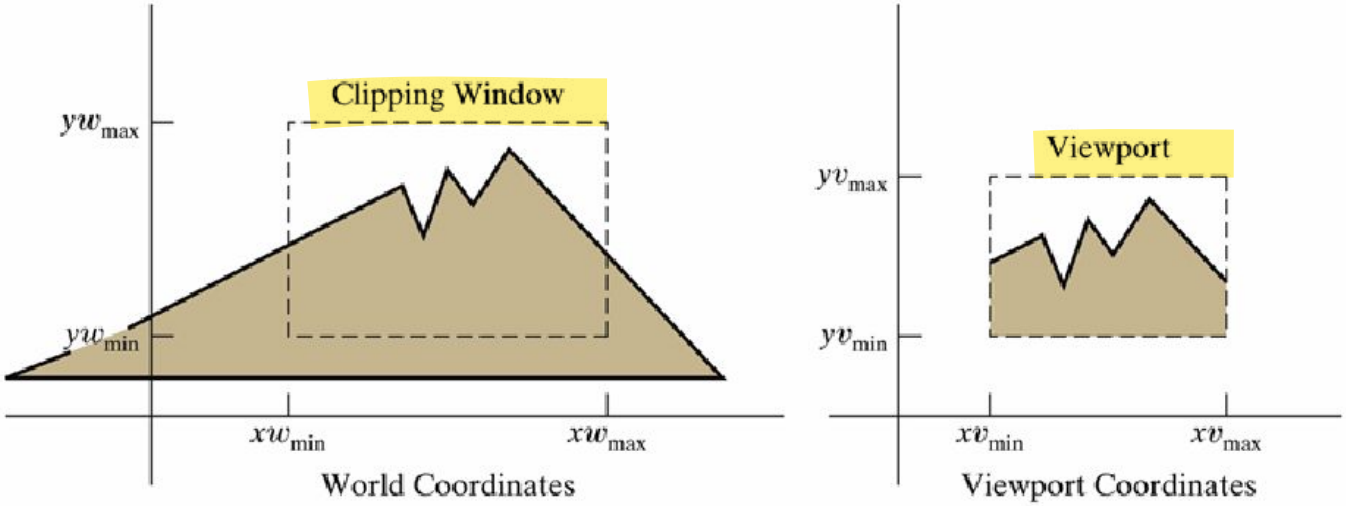
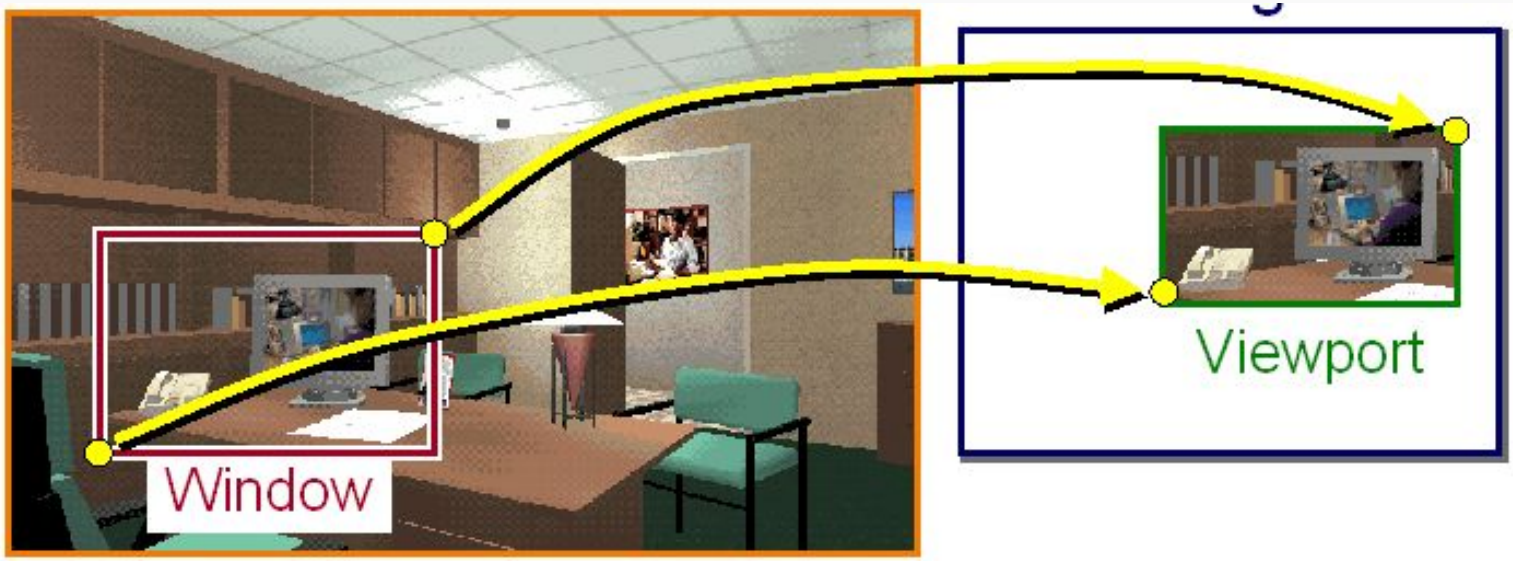


Figure 6-2

A clipping window and associated viewport, specified as rectangles aligned with the coordinate axes.

Clipping Window vs. Viewport

- The clipping window selects what we want to see in our virtual 2D world.
- The *viewport* indicates where it is to be viewed on the output device (or within the display window)
- By default the *viewport* has the same location and dimensions as the GLUT display window you create
  - But it can be modified so that only a part of the display window is used for OpenGL display



- Zooming
  - is successive mapping of different sized clipping windows to the viewport
    - reducing clip window : zoom in on part of scene
    - increase clip window : zoom out
- Panning
  - moving a fixed-size clipping window across the scene

Window to viewport transformation

Normalization

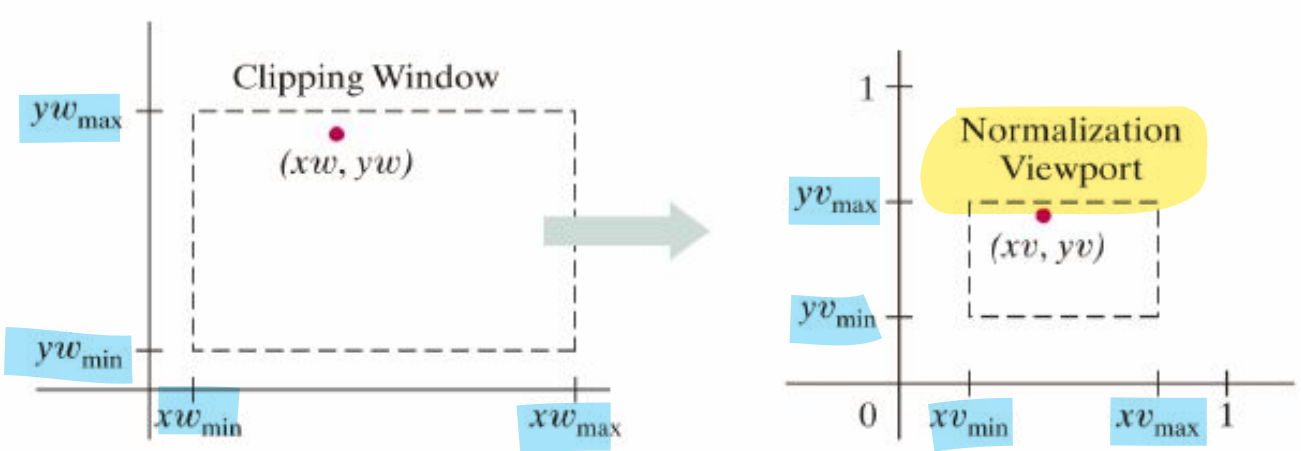
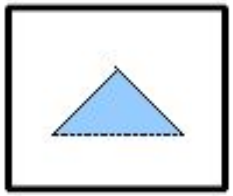


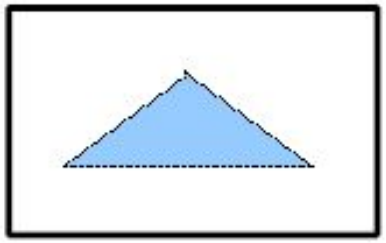
Figure 6-7

A point (xw, yw) in a world-coordinate clipping window is mapped to viewport coordinates (xv, yv), within a unit square, so that the relative positions of the two points in their respective rectangles are the same.

Maintaining relative position of points within the two rectangles

- Coordinate transformation: Different sizes and/or height width ratios?





- For any point:

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

should hold.

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

$$xv = xv_{min} + (xw - xw_{min}) \frac{(xv_{max} - xv_{min})}{xw_{max} - xw_{min}}$$

$$yv = yv_{min} + (yw - yw_{min}) \frac{(yv_{max} - yv_{min})}{yw_{max} - yw_{min}}$$

- This can also be accomplished in 2 steps:

1. Scale over the fixed point:

$$\mathbf{S}(xw_{min}, yw_{min}, s_x, s_y)$$

2. Translate lower-left corner of the clipping window to the lower-left corner of the viewport

$$\mathbf{T}(xv_{min} - xw_{min}, yv_{min} - yw_{min})$$

Aspect ratio

- Relative proportions of objects are maintained only if the aspect ratio of the viewport is the same as the aspect ratio of the clipping window – i.e. only if the scaling factors *sx* and *sy* are the same
  - Otherwise world objects will be stretched or contracted in x or y directions when displayed

Clipping

- In computer graphics our screen act as a 2-D coordinate system
- It is not necessary that each and every point can be viewed on our viewing pane(i.e. our computer screen)
- **line clipping** is the process of removing lines or portions of lines outside an area of interest.
- There are two common algorithms for line clipping: Cohen–Sutherland and Liang–Barsky.
- Liang–Barsky is a simplified form of Cyrus–Beck algorithm

Program 6

Program to implement Cohen-Sutherland Line Clipping Algorithm. Make provision to specify the input line, window for clipping and view port for displaying the clipped image.

Cohen-Sutherland Line-Clipping:

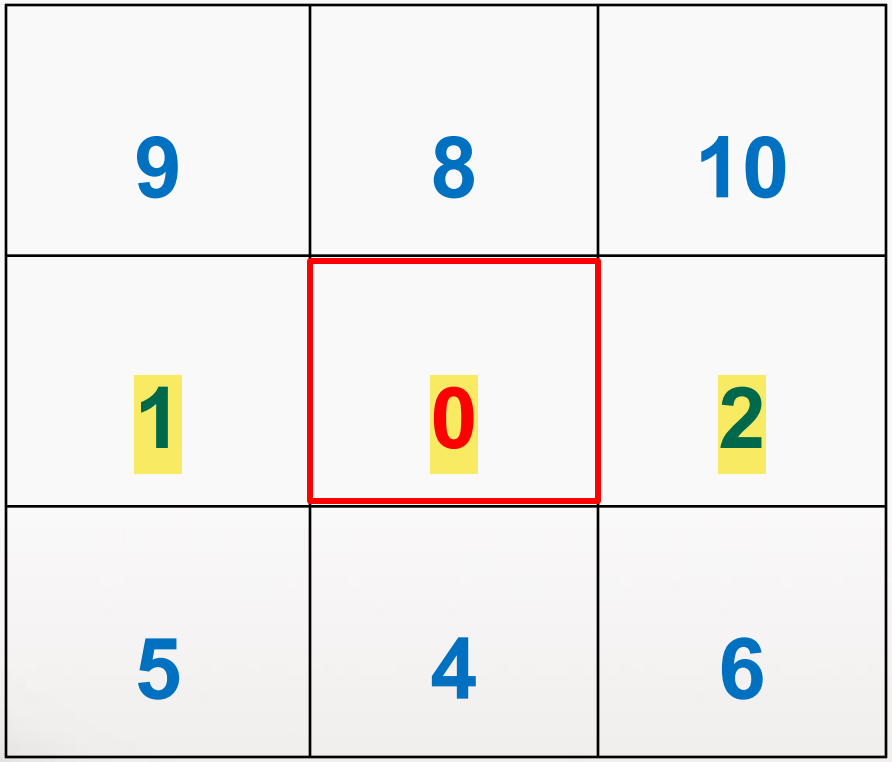
- It is a very good algorithm for clipping pictures that are larger than the screen.
- This algorithm divides a 2D space into 9 parts, of which only the middle part (view port) is visible.
- The 9 regions can be uniquely identified using a 4 bit code. This 4 bit code is called ***outcode***.

Cohen-Sutherland Line-Clipping Outcodes

1001	1000	1010
0001	0000	0010
0101	0100	0110

We use the order **Top, Bottom, Right, Left** for these 4 bits.

Cohen-Sutherland Line-Clipping Outcode



We use the order **Top, Bottom, Right, Left** for these 4 bits.

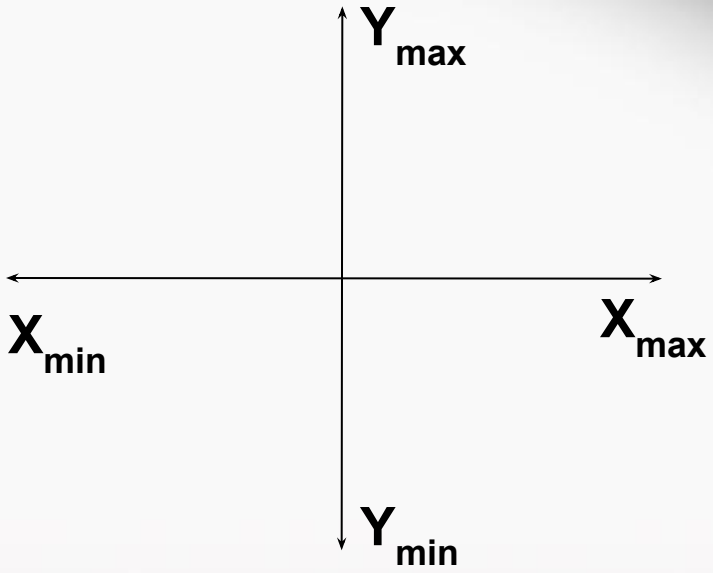
Cohen-Sutherland Line-Clipping Region Outcodes

1001	1000	1010
0001	0000	0010
0101	0100	0110

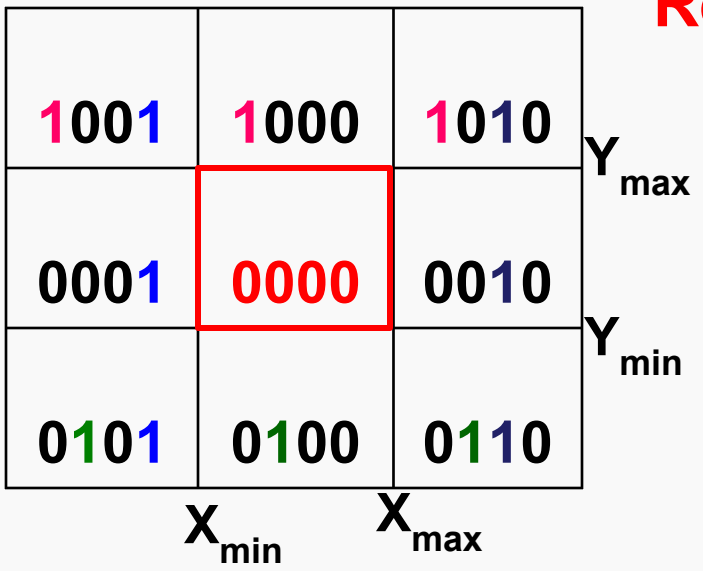
Y_{max}

Y_{min}

X_{min} X_{max}



Cohen-Sutherland Line-Clipping Region Outcodes



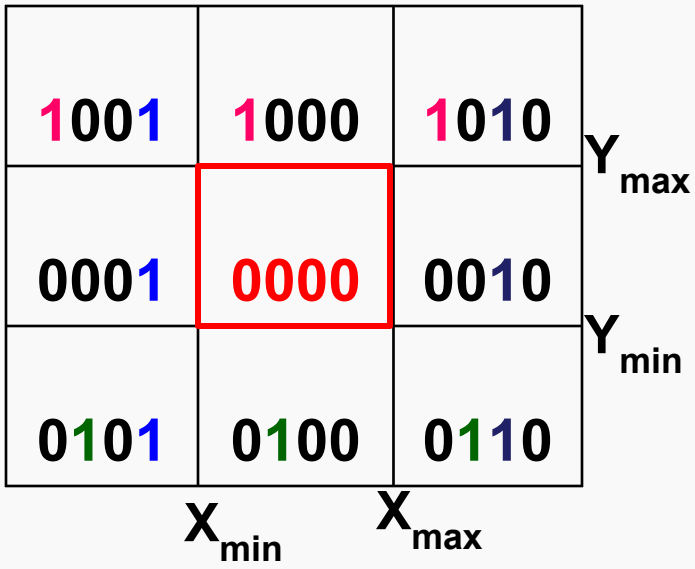
Bit Number	1	0
First (MSB)	Above top edge $Y>Y_{max}$	Below top edge $Y<Y_{max}$
Second	Below bottom edge $Y<Y_{min}$	Above bottom edge $Y>Y_{min}$
Third	Right of right edge $X>X_{max}$	Left of right edge $X<X_{max}$
4th (LSB)	Left of left edge $X<X_{min}$	Right of left edge $X>X_{min}$

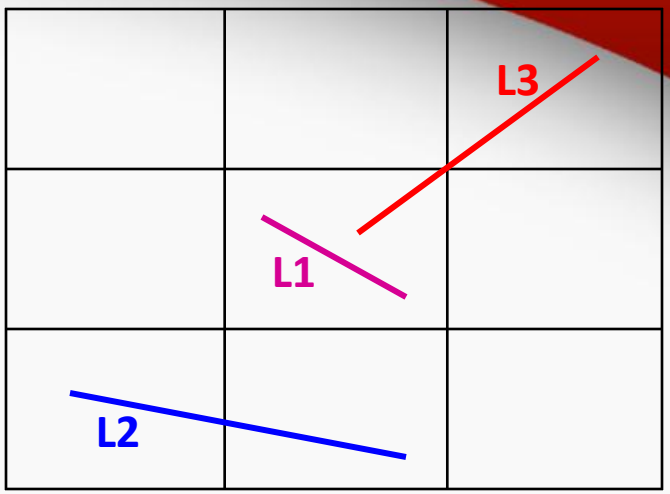
Line-Clipping Algorithm

Assume two endpoints are p0 and p1

1. If code(p0) **OR** code(p1) is **0000**,
   - the line can be **trivially accepted.**
   - the line is drawn.
2. If code(p0) **AND** code(p1) is **NOT 0000**,
   - the line can be **trivially rejected.**
   - the line is not drawn at all.
3. Otherwise, compute the **intersection points** of the line segment and window boundary lines (make sure to check all the boundary lines)

Line-Clipping





Line L1

code1 = 0000, code2 = 0000

Is code1|code2==0? Trivial accept

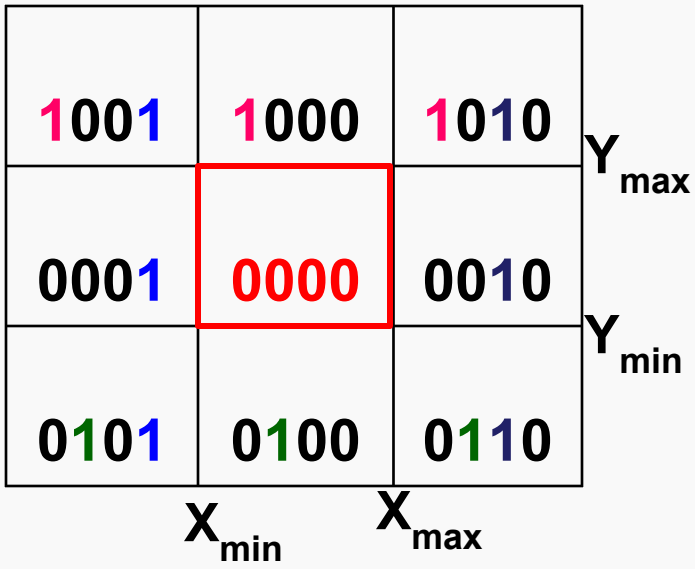
Is code1 & code2!=0? Trivial reject

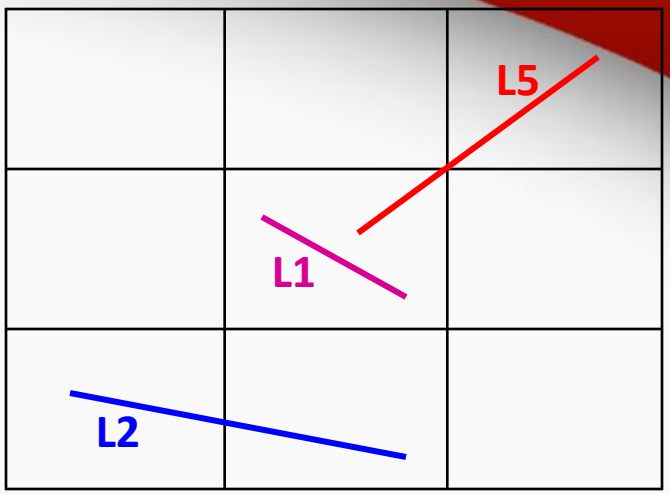
Else clip the line until one code becomes 0

code1 | code2 = 0000 | 0000 = 0000

Trivial accept completely – no need to clip.

Line-Clipping





Line L2

code1 = 0101, code2 = 0100

Is code1|code2==0? Trivial accept

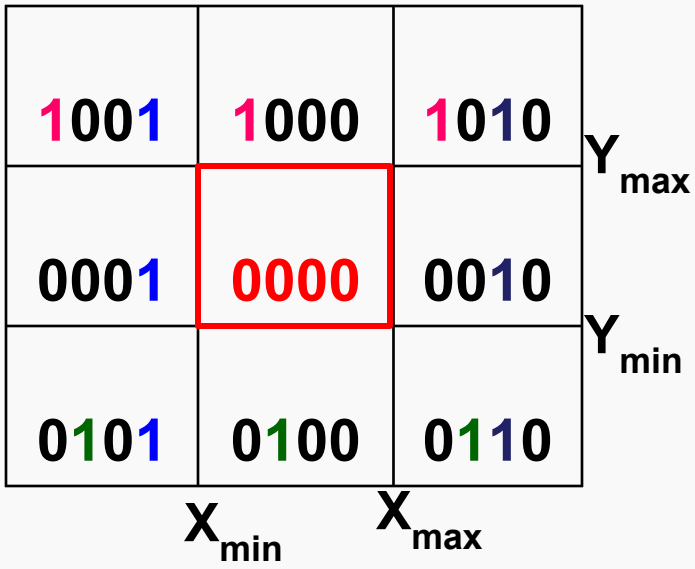
Is code1 & code2!=0? Trivial reject

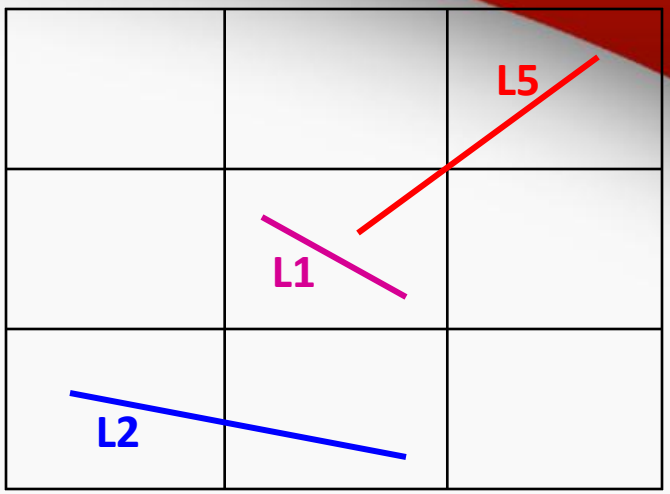
Else clip the line until one code becomes 0

code1 | code2 = 0101 | 0100 = 0101 (NOT 0000) : No Trivial accept

Code1 & code2 = 0101 & 0100 = 0100 (NOT 0000) : Trivial reject

Line-Clipping





Line L5

code1 = 0000, code2 = 1010

Is code1|code2==0? Trivial accept

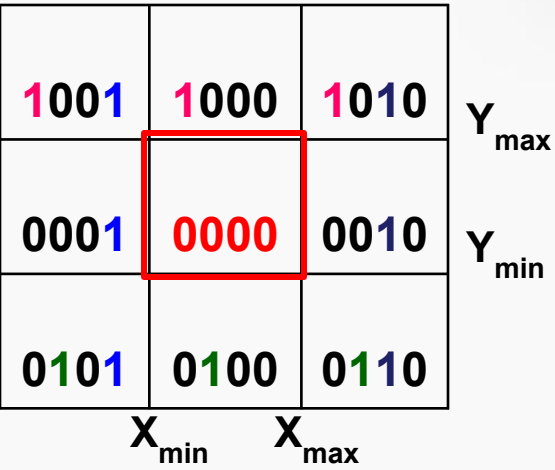
Is code1 & code2!=0? Trivial reject

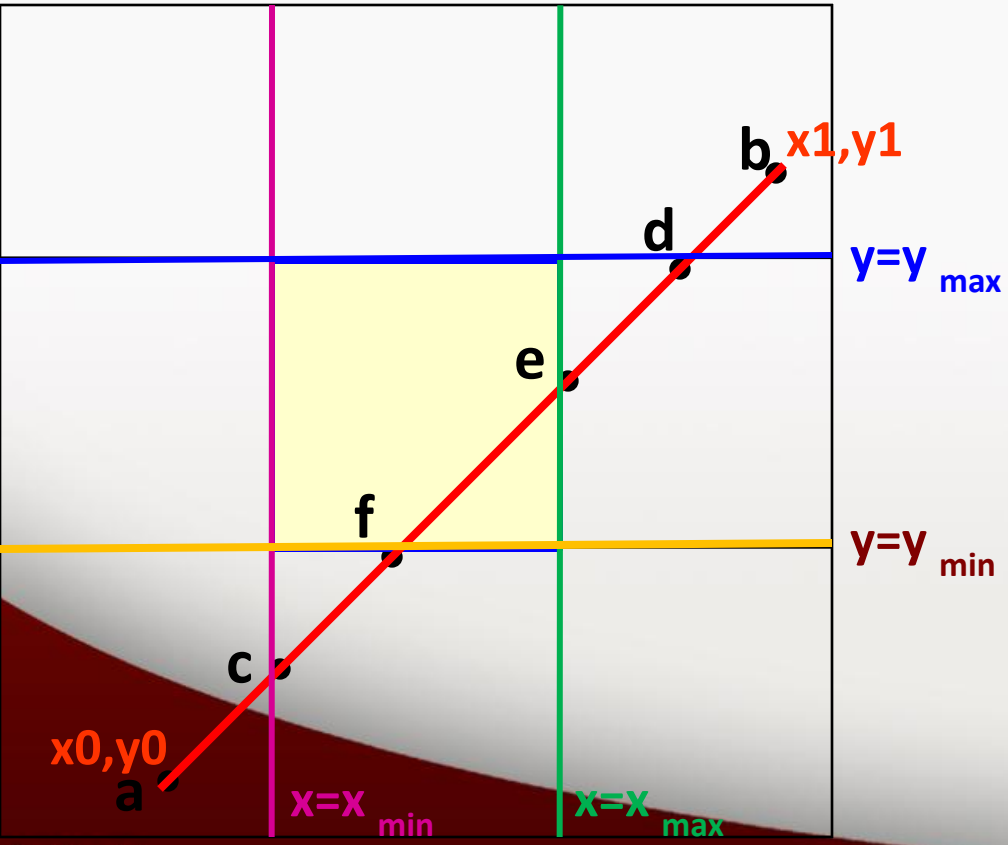
Else clip the line until code becomes 0

code1 | code2 = 0000 | 1010 = 1010 (NOT 0000) : No Trivial accept

& code2 = 0000 & 1010 = 0000 : No Trivial reject

Line-Clipping Example





Intersection computation

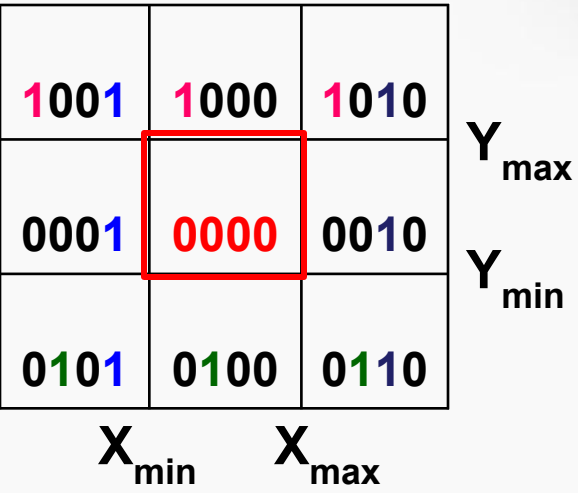
Line equation

$y = y0 + m(x - x0)$

where

$$m = \frac{y1 - y0}{x1 - x0}$$

Line-Clipping Example



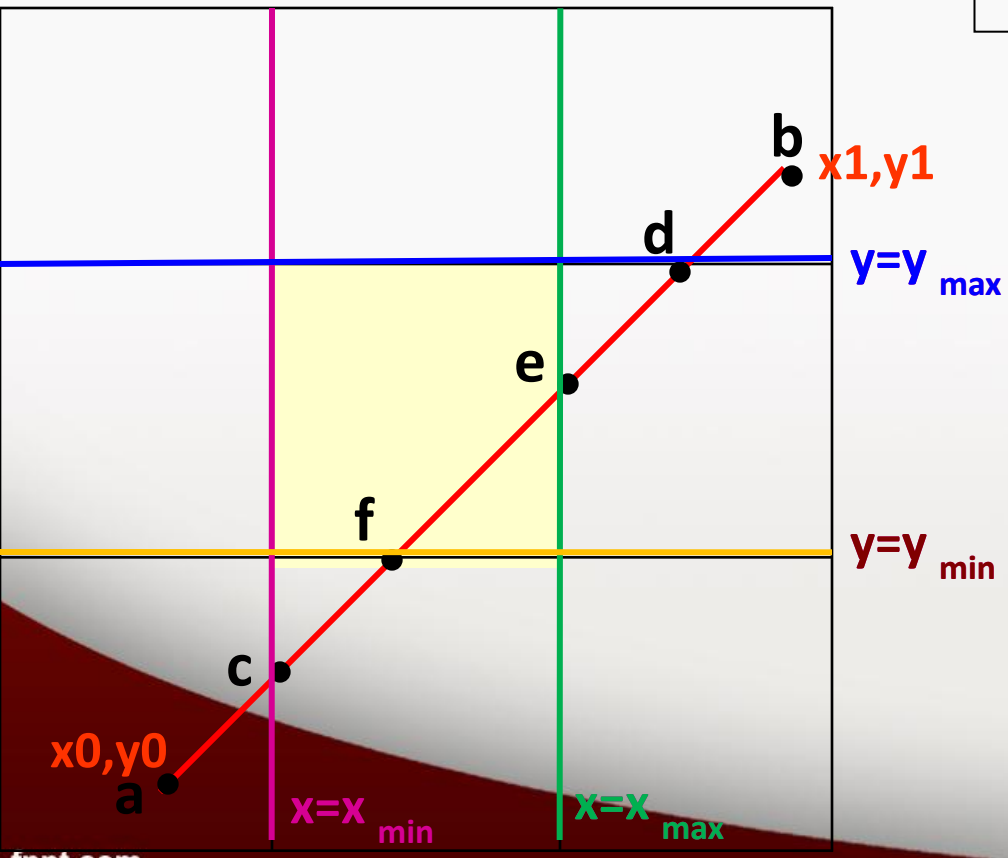
Intersection computation

Line equation

$y = y0 + m(x - x0)$

where

$m = \frac{y1 - y0}{x1 - x0}$



Line intersection with the left vertical boundary

Assume the intersection is **c**

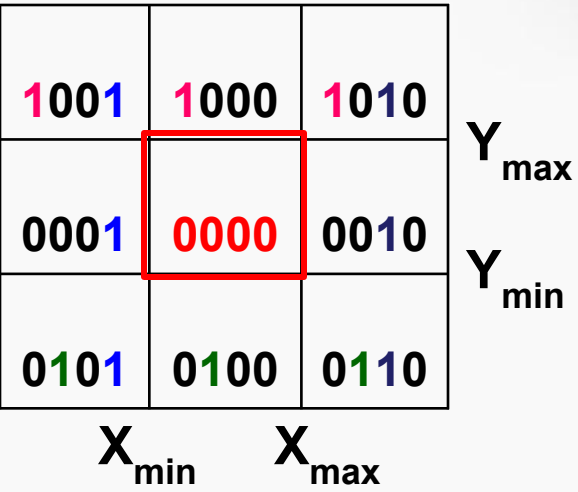
$x = x_{min}$

$y = y0 + m(x_{min} - x0)$

Line ab is clipped w.r.t. $x = x_{min}$

now **ab becomes cb**

Line-Clipping Example



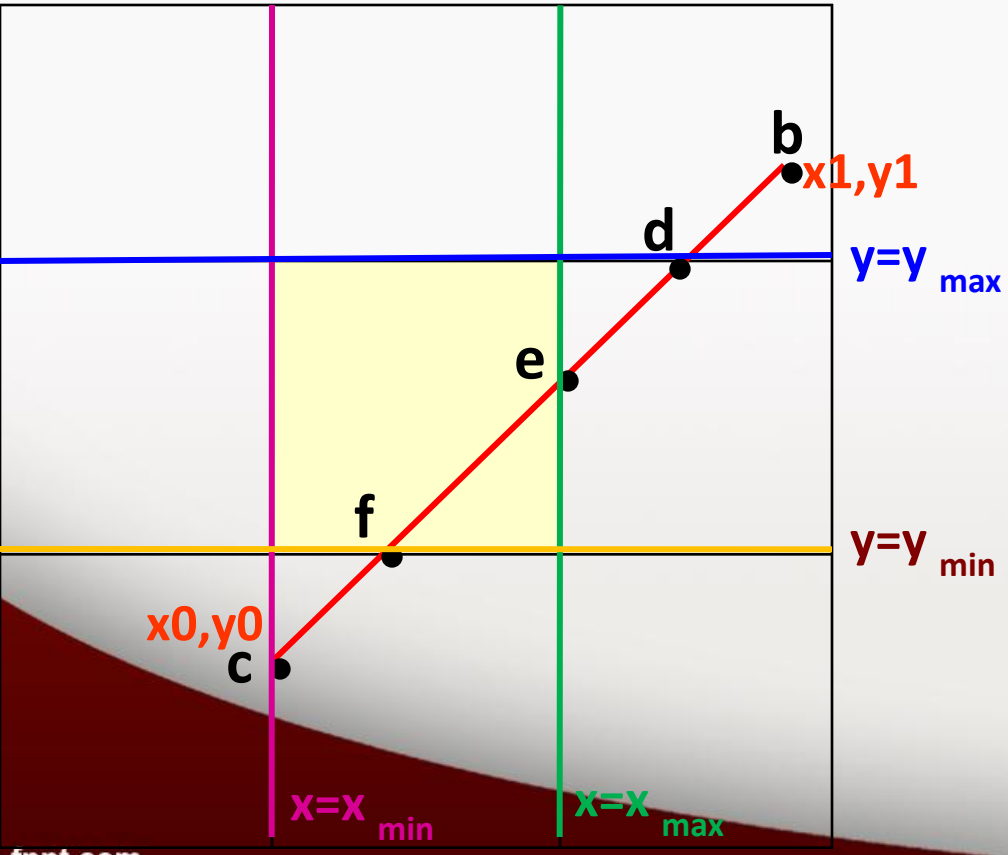
Intersection computation

Line equation

$y = y0 + m(x - x0)$

where

$m = \frac{y1 - y0}{x1 - x0}$



Line intersection with the bottom boundary

Assume the intersection is **f**

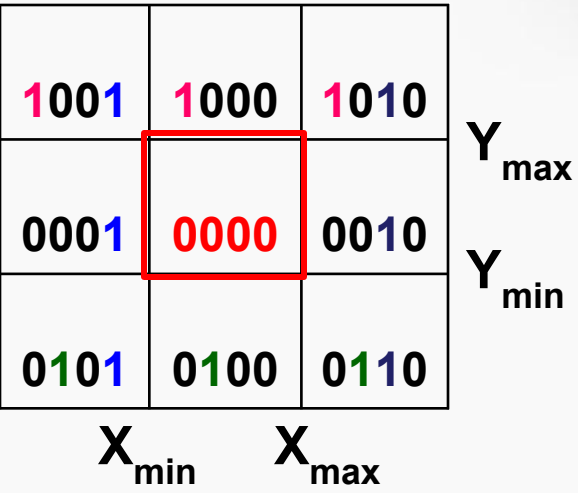
$y = y_{min}$

$x = (1/m)(y_{min} - y0) + x0$

Line cb is clipped w.r.t. $y = y_{max}$

Line **cb becomes fb**

Line-Clipping Example



Intersection computation

unit 3

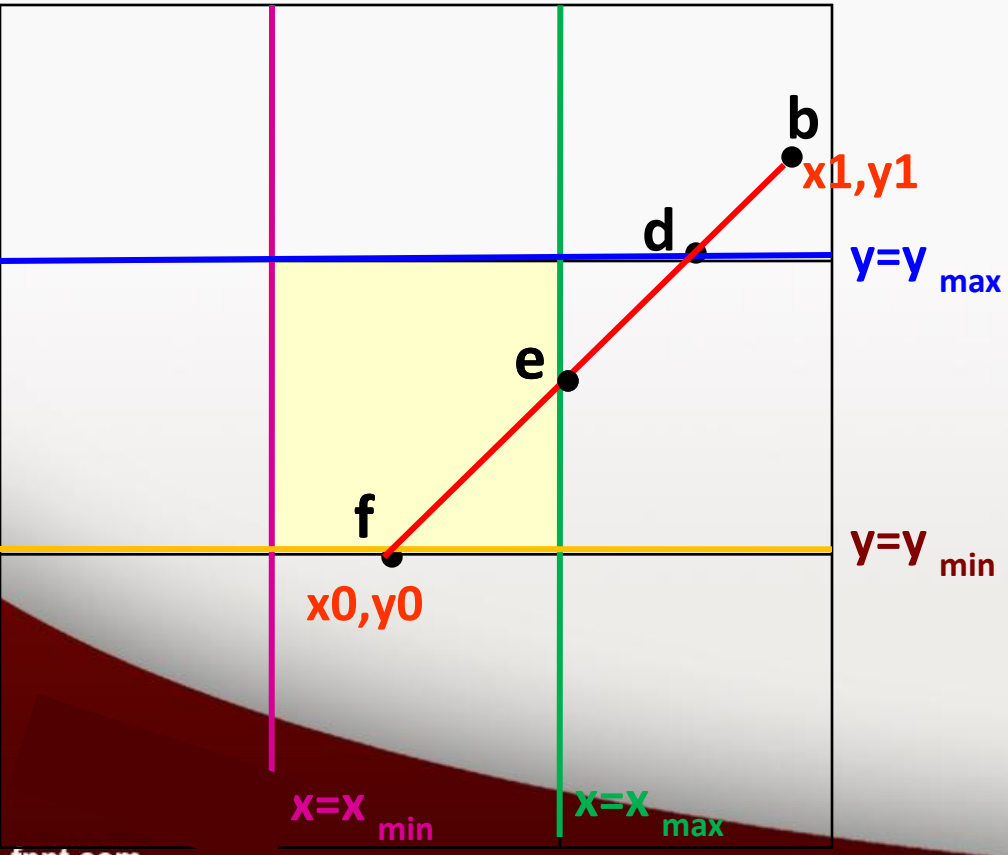
Line equation

$y = y0 + m(x - x0)$

where

$m = \frac{y1 - y0}{x1 - x0}$

Line intersection with the top boundary



Assume the intersection is **d**

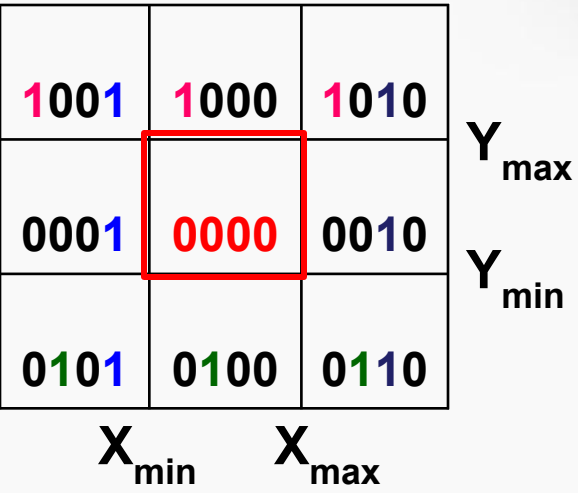
$y = y_{max}$

$x = 1/m\ (y_{max} - y0) + x0$

Line fb is clipped w.r.t. $y=y_{max}$

line fb becomes fd

Line-Clipping Example



Intersection computation

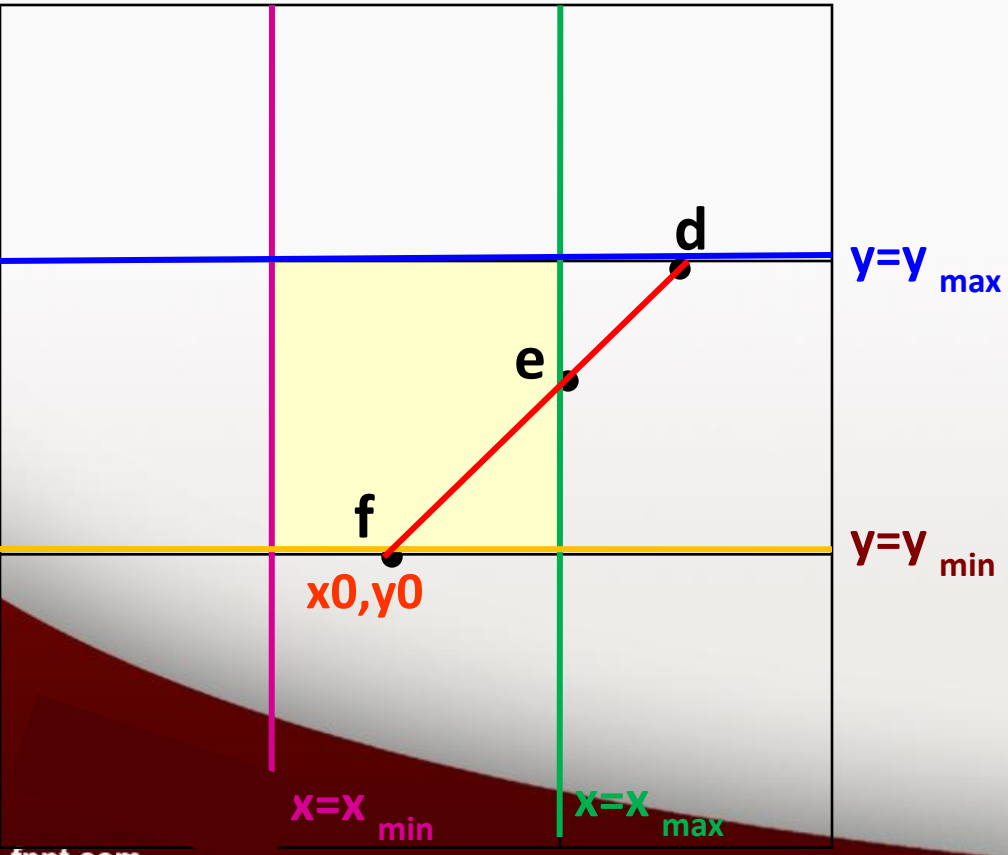
unit 3

Line equation

$y = y0 + m(x - x0)$

where

$m = \frac{y1 - y0}{x1 - x0}$



Line intersection with the right boundary

Assume the intersection is **e**

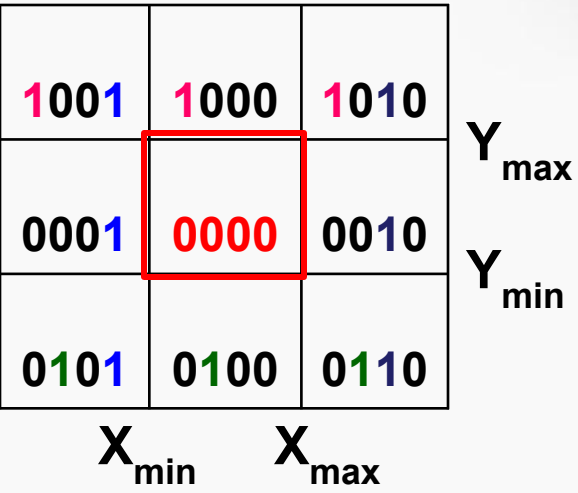
$x = x_{max}$

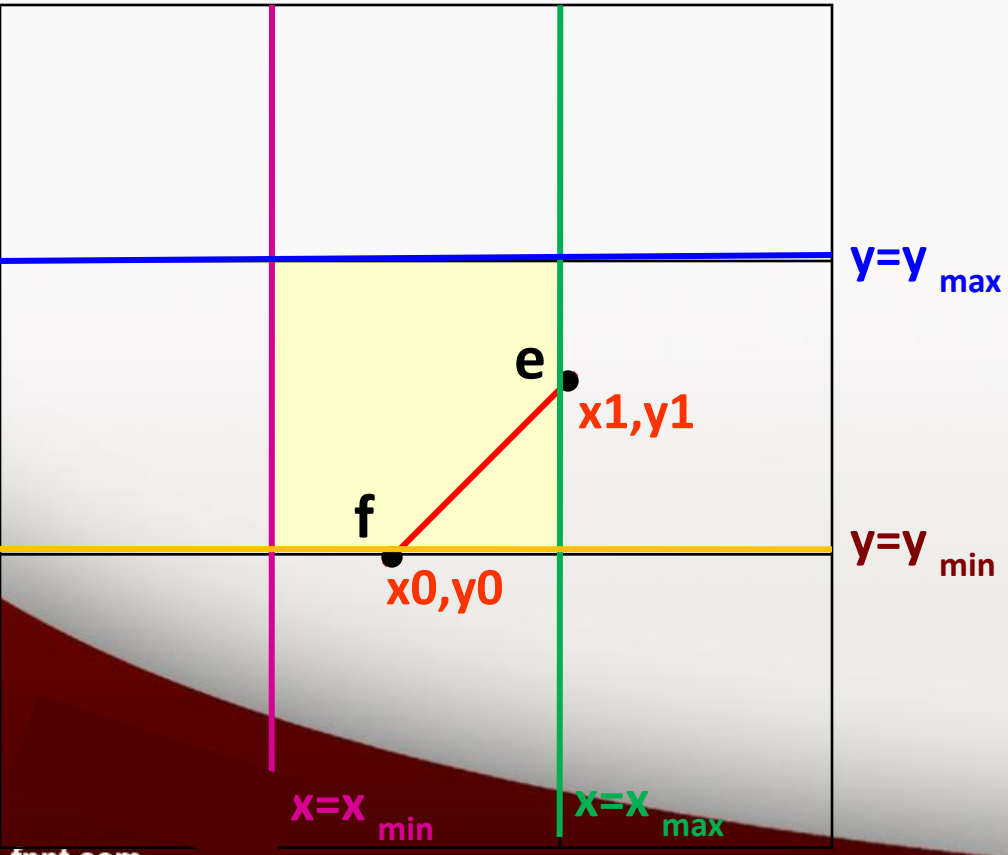
$y = y0 + m(x_{max} - x0)$

Line fd is clipped w.r.t. $x=x_{max}$

line fd becomes fe

Line-Clipping Example





FINALLY…….

….

ab -----> cb

cb -----> fb

fb -----> fd

fd -----> fe

```
#include<GL/glut.h>
#define outcode int
double xmin=50,ymin=50, xmax=100,ymax=100;
// Window boundaries
double xvmin=200,yvmin=200,xvmax=300,yvmax=300;
// Viewport boundaries

double m;

//bit codes for the right, left, top, & bottom
const int LEFT = 1;
const int RIGHT = 2;
const int BOTTOM = 4;
const int TOP = 8;

//used to compute bit codes of a point
outcode ComputeOutCode (double x, double y);

//Cohen-Sutherland clipping algorithm clips a line from
//P0 = (x0, y0) to P1 = (x1, y1) against a rectangle with
//diagonal from (xmin, ymin) to (xmax, ymax).
```

```
void CohenSutherlandLineClipAndDraw(double x0, double y0,double x1, double y1)
{
    //Outcodes for P0, P1, and whatever point lies outside the clip rectangle
    outcode outcode0, outcode1, outcodeOut;
    bool accept = false, done = false;


    outcode0 = ComputeOutCode (x0, y0);
    outcode1 = ComputeOutCode (x1, y1);
     m= (y1-y0)/(x1-x0);
    do
    {
        if (!(outcode0 | outcode1)) //logical or is 0 Trivially accept & exit
        {
            accept = true;
            done = true;
        }
        else if (outcode0 & outcode1)
         //logical and is not 0.Trivially reject &exit
        done = true;
```

```
else
{
/* failed both tests, so calculate the line segment to clip from an outside
point to an intersection with clip edge */

double x, y;
outcodeOut = outcode0? outcode0: outcode1;

    if (outcodeOut& TOP) //point is above the clip rectangle
    {
        x = x0 + (1/m) * (ymax - y0);
        y = ymax;
    }

    else if (outcodeOut& BOTTOM)//point is below the clip rectangle
    {
        x = x0 + (1/m) * (ymin - y0);
        y = ymin;
    }
```

```
else if(outcodeOut& RIGHT)//point is to right of clip rectangle
{
      y = y0 +m*(xmax - x0);
      x = xmax;
}

else   //point is to the left of clip rectangle
{
      y = y0 +m*(xmin - x0);
      x = xmin;
}
```

```
//Now we move outside point to intersection point to clip
//and get ready for next pass.
      if (outcodeOut == outcode0)
      {
            x0 = x;
            y0 = y;
            outcode0 = ComputeOutCode (x0, y0);
      }
      else
      {
            x1 = x;
            y1 = y;
            outcode1= ComputeOutCode (x1, y1);
      }
       } //end of if
}while (!done);
```

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

$$xv = xv_{min} + (xw - xw_{min})\frac{(xv_{max} - xv_{min})}{xw_{max} - xw_{min}}$$

$$yv = yv_{min} + (yw - yw_{min})\frac{(yv_{max} - yv_{min})}{yw_{max} - yw_{min}}$$

- This can also be accomplished in 2 steps:

1. Scale over the fixed point:

$$\mathbf{S}(xw_{min}, yw_{min}, s_x, s_y)$$

2. Translate lower-left corner of the clipping window to the lower-left corner of the viewport

$$\mathbf{T}(xv_{min} - xw_{min}, yv_{min} - yw_{min})$$

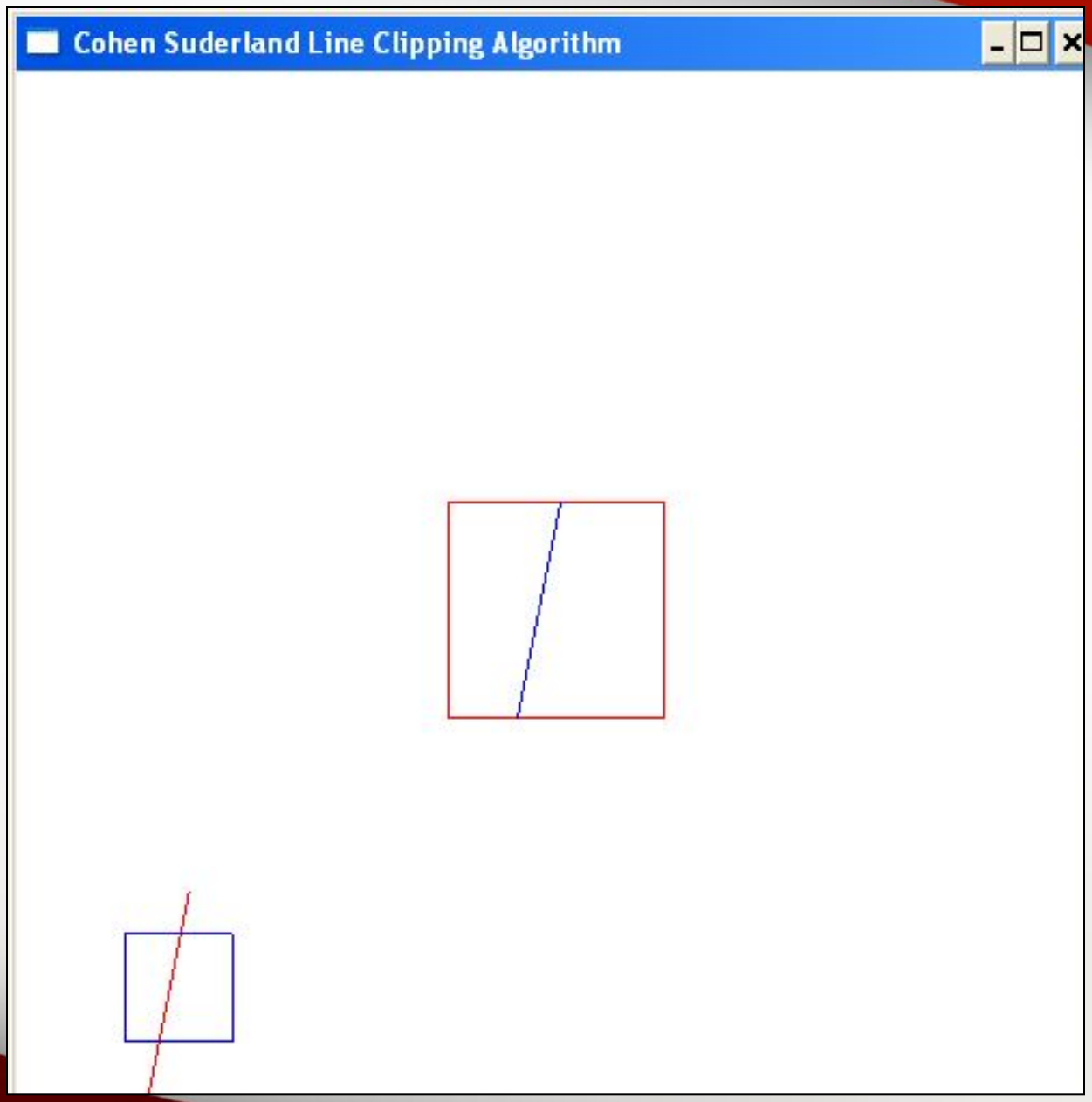
```
if (accept)
    {           double sx=(xvmax-xvmin)/(xmax-xmin);
        double sy=(yvmax-yvmin)/(ymax-ymin);
        double vx0=xvmin+(x0-xmin)*sx;
        double vy0=yvmin+(y0-ymin)*sy;
        double vx1=xvmin+(x1-xmin)*sx;
        double vy1=yvmin+(y1-ymin)*sy;
        glColor3f(1.0, 0.0, 0.0);  // new view port in red color
        glBegin(GL_LINE_LOOP);
            glVertex2f(xvmin, yvmin);
            glVertex2f(xvmax, yvmin);
            glVertex2f(xvmax, yvmax);
            glVertex2f(xvmin, yvmax);
        glEnd();
        glColor3f(0.0,0.0,1.0);   // clipped line in blue color
        glBegin(GL_LINES);
            glVertex2d (vx0, vy0);
            glVertex2d (vx1, vy1);
        glEnd();
    }
}
```

```
outcode ComputeOutCode (double x, double y)
{
    outcode code = 0;
    if (y >ymax)     //above the clip window
    code |= TOP;
    else if (y <ymin)     //below the clip window
    code |= BOTTOM;
    if (x >xmax)     //to the right of clip window
    code |= RIGHT;
    else if (x <xmin)     //to the left of clip window
    code |= LEFT;
    return code;
}
```

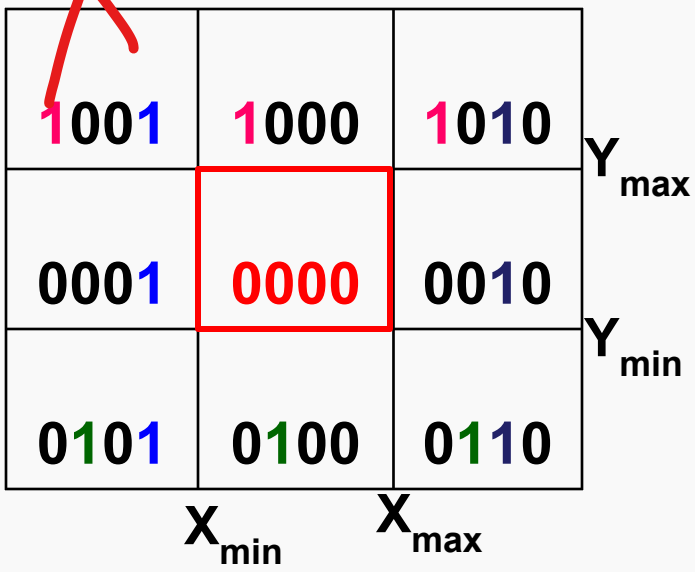
Bit Number	1	0
First (MSB)	Above top edge $Y>Y_{max}$	Below top edge $Y<Y_{max}$
Second	Below bottom edge $Y<Y_{min}$	Above bottom edge $Y>Y_{min}$
Third	Right of right edge $X>X_{max}$	Left of right edge $X<X_{max}$
Fourth (LSB)	Left of left edge $X<X_{min}$	Right of left edge $X>X_{min}$

```
void display()
{
	double x0=60,y0=20,x1=80,y1=120;
	glClear(GL_COLOR_BUFFER_BIT);
	glColor3f(1.0,0.0,0.0); //draw the line with red color
	glBegin(GL_LINES);
		glVertex2d (x0, y0);
		glVertex2d (x1, y1);
	glEnd();
	glColor3f(0.0, 0.0, 1.0); //draw a blue colored window
	glBegin(GL_LINE_LOOP);
		glVertex2f(xmin, ymin);
		glVertex2f(xmax, ymin);
		glVertex2f(xmax, ymax);
		glVertex2f(xmin, ymax);
	glEnd();
	CohenSutherlandLineClipAndDraw(x0,y0,x1,y1);
	glFlush();
}
```

```
void myinit()
{
	glClearColor(1.0,1.0,1.0,1.0);
	glColor3f(1.0,0.0,0.0);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(0.0,499.0,0.0,499.0);
}

void main(int argc, char** argv)
{
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
	glutInitWindowSize(500,500);
	glutCreateWindow("Cohen Suderland Line Clipping Algorithm");
	glutDisplayFunc(display);
	myinit();
	glutMainLoop();
}
```



Cohen-Sutherland Line-Clipping



Algorithm

Is code1|code2==0? **Trivial accept**

Is code1 & code2!=0? **Trivial reject**

Else clip the line until one code becomes 0

Intersection computation

Line equation

$y = y0 + m(x - x0)$

where

$$m = \frac{y1 - y0}{x1 - x0}$$

Line intersection with the left vertical boundary

$x = x_{min}$

$y = y0 + m(x_{min} - x0)$

Line intersection with the right boundary

$x = x_{max}$

$y = y0 + m(x_{max} - x0)$

Line intersection with the bottom boundary

$y = y_{min}$

$x = (1/m)\ (y_{min} - y0) + x0$

Line intersection with the top boundary

$y = y_{max}$

$x = 1/m\ (y_{max} - y0) + x0$

PROGRAM 7

Write a program to implement the Liang-Barsky line clipping algorithm. Make provision to specify the input for multiple lines, window for clipping and viewport for displaying the clipped image.

Liang-Barsky Line Clipping Algorithm

Parametric Line Equation

$x = tx1 + (1-t)x0$

$x = x0 + t(x1 - x0)$ where $0<=t<=1$

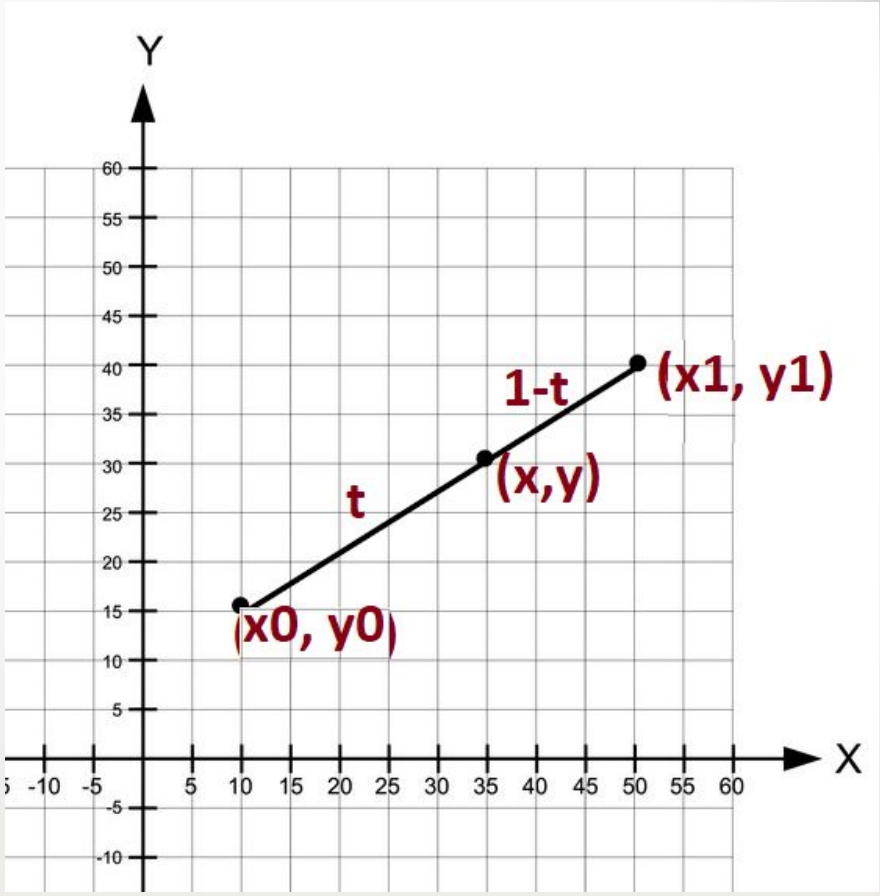
$x = x0 + tdx$ where $0<=t<=1$

Similarly,

$y = y0 + t(y1 - y0)$ where $0<=t<=1$

$y = y0 + tdx$ where $0<=t<=1$

t=0 starting point
t=1 ending point



Liang-Barsky Line Clipping Algorithm

$$xmin <= x <= xmax$$

$$ymin <= y <= ymax$$

$$xmin <= x0 + tdx <= xmax$$

$$ymin <= y0 + tdy <= ymax$$

$$x0 + tdx >= xmin$$

$$x0 + tdx <= xmax$$

$$y0 + tdy >= ymin$$

$$y0 + tdy <= ymax$$

Liang-Barsky Line Clipping Algorithm

t.p <= q

tdx >= xmin – x0

tdx <= xmax – x0

tdy >= ymin – y0

tdy <= ymax – y0

-tdx <= x0 - xmin

tdx <= xmax – x0

-tdy <= y0 - ymin

tdy <= ymax – y0

Liang-Barsky Line Clipping Algorithm

Edge	p	q	t
Left	p = -dx	q = x0 – xmin	t = q/p
Right	p = dx	q = xmax – x0	t = q/p
Bottom	p = -dy	q = y0 – ymin	t = q/p
Top	p = dy	q=ymax – y0	t = q/p

Liang-Barsky Line Clipping Algorithm

For any edge,

T{entry}(t1): The maximum parameter value where the line enters the clipping window.
Initialized to 0.0

T{leaving}(t2): The minimum parameter value where the line leaves the clipping window
Initialized to 1.0

- If p< 0 // entry point, update t_{entry} (t1)
  - if t > t1 set t1 = t
  - if t > t2 return 0 // line portion is outside
- If p > 0 // leaving point, update $t_{leaving}$ (t2)
  - if t < t2 set t2 = t
  - if t < t1 return 0 // line portion is outside
- If p == 0 // line parallel to edge
  if(q < 0.0) return 0 // outside the edge

Liang-Barsky Line Clipping Algorithm

if($t_{entry} > 0.0$) calculate new x0, y0

$x0_{new} = x0 + t_{entry} * dx$

$y0_{new} = y0 + t_{entry} * dy$

if($t_{leaving} < 1.0$) calculate new x1, y1

$x1_{new} = x0 + t_{leaving} * dx$

$y1_{new} = y0 + t_{leaving} * dy$

Draw clipped line with the points

$A(x0_{new}, y0_{new})$ and $B(x1_{new}, y1_{new})$

Liang-Barsky Line Clipping Algorithm

dx = P1x - P0x // *Horizontal diff between P0 and P1.*

dx = x1 – x0 = 280-30 = 250

dy = P1y - P0y // *Vertical diff between P0 and P1.*

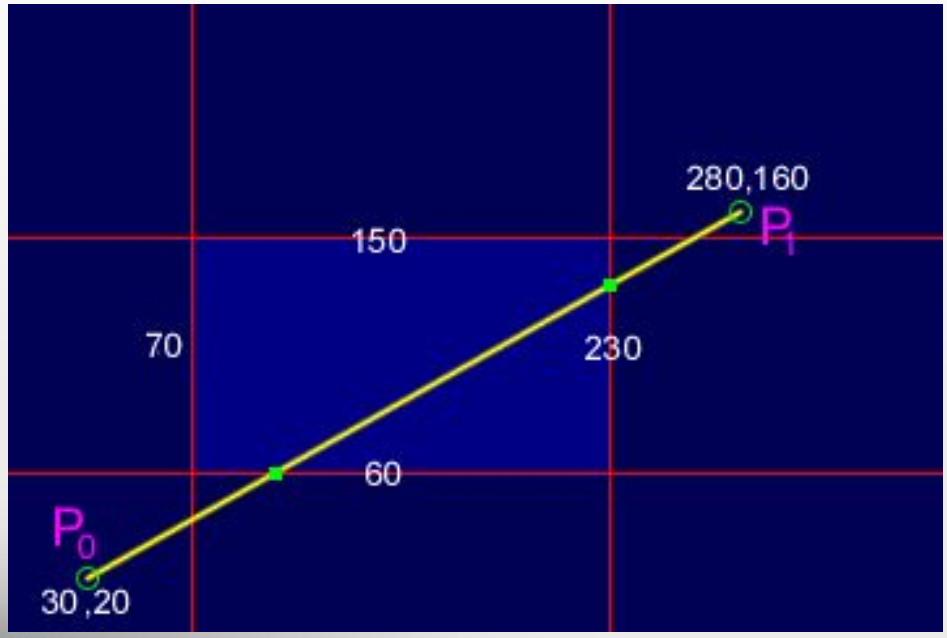
dy = y1 – y0 = 160-20 = 140

dx = 250 , dy = 140

Initially

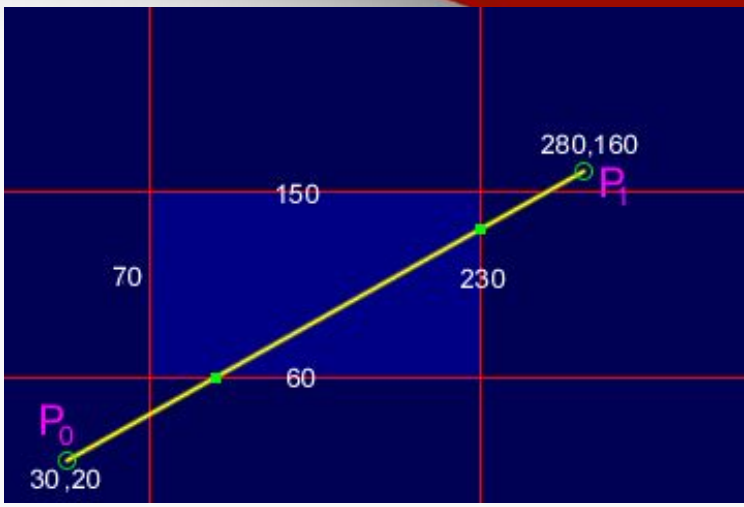
t_{entry} (t1) = 0.0

$t_{leaving}$ (t2) = 1.0



dx = 250
dy = 140

Liang-Barsky Line Clipping - Example



Left edge check:

p = -dx = -250

q = x0 - xmin = 30-70 = -40

t = q/p = 0.16

- **If p< 0** *// entry point, update t_{entry} (t1)*
  if t > t1 set t1 = t

-250 < 0 TRUE

check if t > t1 -----> 0.16 > 0.0 TRU

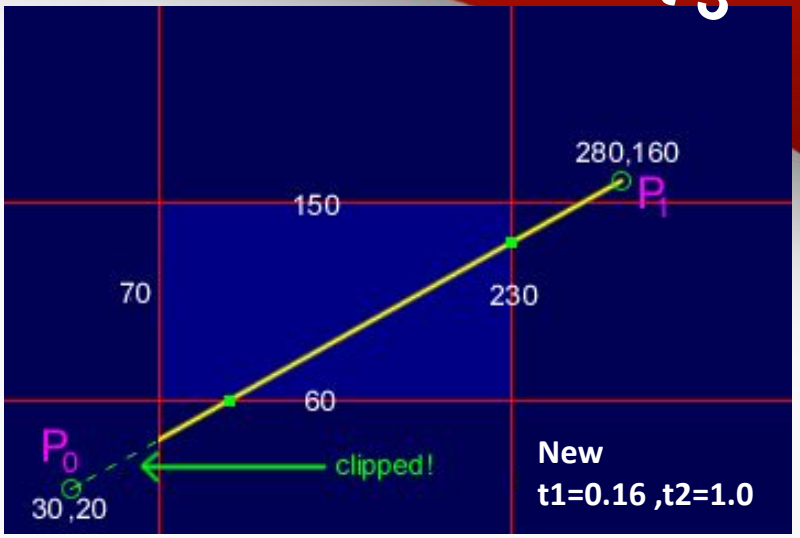
t1 = t

t1 = 0.16

~ 0.16 t2 = 1.0

dx = 250
dy = 140

Liang-Barsky Line Clipping - Example



Right edge check:

p = dx = 250

q = xmax – x0 = 230-30 = 200

t = q/p = 0.8

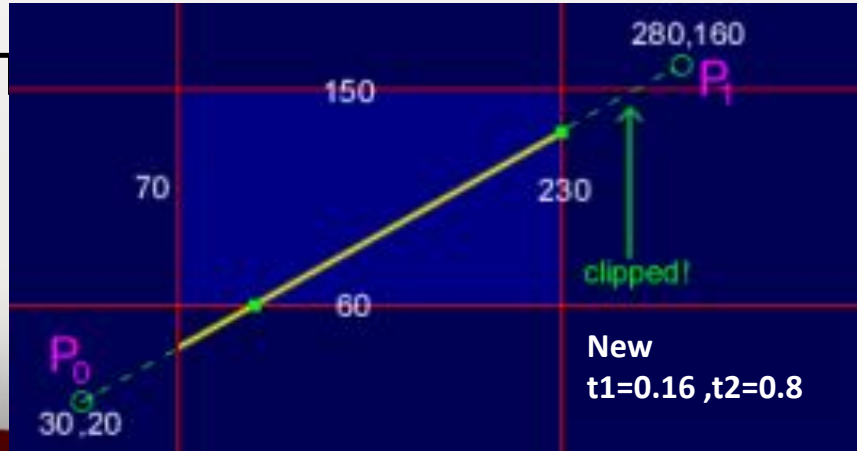
- **If p< 0 FALSE**
- **If p > 0** *// leaving point, update $t_{leaving}$ (t2)*
  if t < t2 set t2 = t

  250 > 0 TRUE

  check if t < t2 -----> 0.8 < 1.0 T

  t2 = t = 0.8

t1 = 0.16 t2 = 0.8



dx = 250
dy = 140

Liang-Barsky Line Clipping - Example

Bottom edge check:

p = -dy = -140

q = y0 - ymin = 20-60 = -40

t = q/p = 0.2857

- **If p< 0** *// entry point, update t_{entry} (t1)*
  if t > t1 set t1 = t

-140 < 0 TRUE

check if t > t1 ----->

0.2857 > 0.16 TRUE

t1 = t----> t1 = 0.2857

t1 = 0.2857 t2 = 0.8

dx = 250
dy = 140

Liang-Barsky Line Clipping - Example

Top edge check:

p = dy = 140

q = ymax – y0 = 150 - 20 = 130

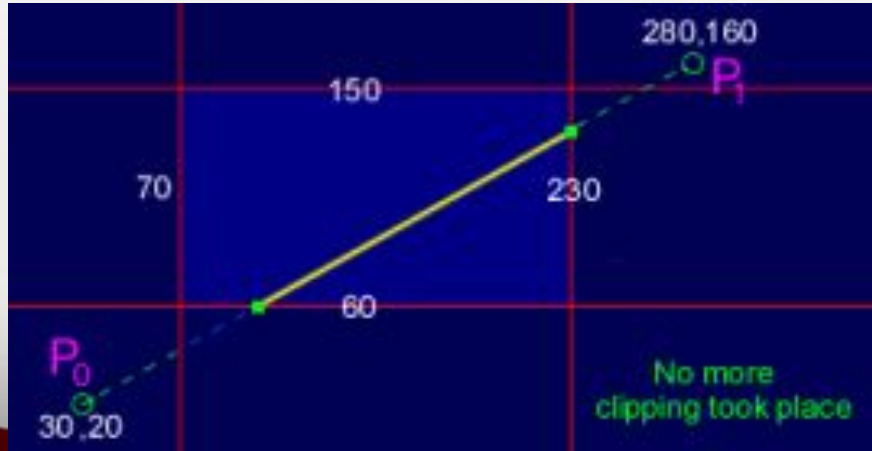
t = q/p = 0.928

- **If p< 0 FALSE**
- **If p > 0** *// leaving point, update $t_{leaving}$ (t2)*
  if t < t2 set t2 = t

  140 > 0 TRUE

  check if t < t2 ----->

  0.928 < 0.8 FALSE

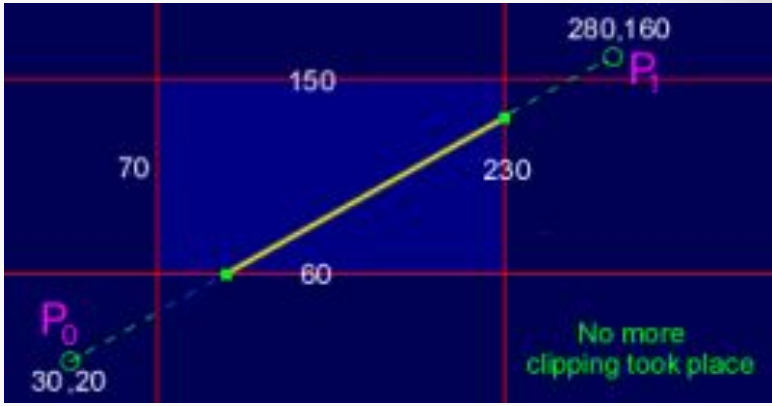


t_1 = 0.2857 t2 = 0.8

Liang-Barsky Line Clipping - Example

t1 = 0.2857 t2 = 0.8

dx = 250 dy = 140



if(t_{entry} > 0.0) calculate new x0, y0

0.2857 > 0.0 **TRUE**

$x0_{new} = x0 + t_{entry} * dx$

$\mathbf{x0_{new}} = 30 + 0.2857 * 250 = \mathbf{101.425}$

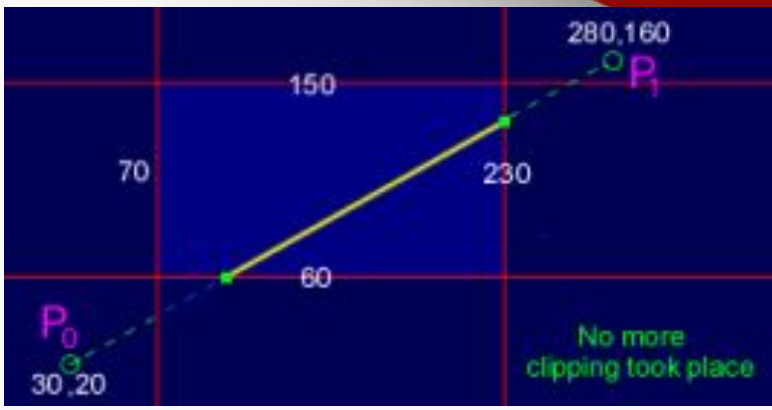
$y0_{new} = y0 + t_{entry} * dy$

$\mathbf{y0_{new}} = 20 + 0.2857 * 140 = \mathbf{59.99}$

Liang-Barsky Line Clipping - Example

t1 = 0.2857 t2 = 0.8

dx = 250 dy = 140



if($t_{leaving}$ < 1.0) calculate new x1, y1

0.8 > 0.0 **TRUE**

$x1_{new} = x0 + t_{leaving} * dx$

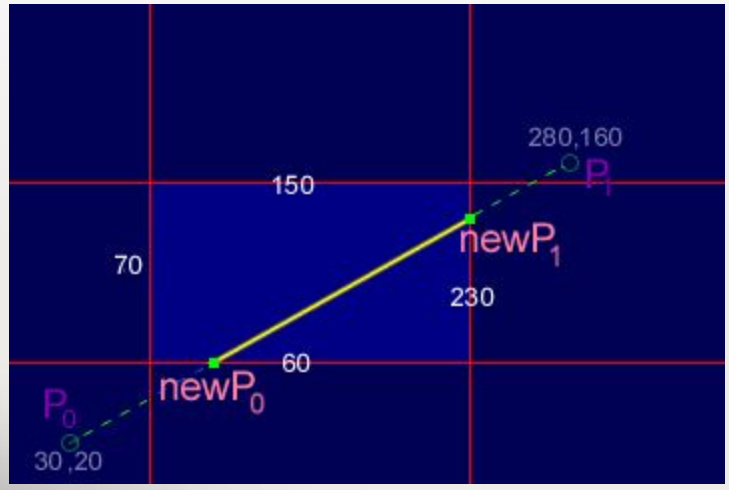
$\mathbf{x1_{new}} = 30 + 0.8 * 250 = \mathbf{230}$

$y1_{new} = y0 + t_{leaving} * dy$

$\mathbf{y1_{new}} = 20 + 0.8 * 140 = \mathbf{132}$

Draw clipped line with the points

A($x0_{new}$, $y0_{new}$) and B($x1_{new}$, $y1_{new}$)



Liang-Barsky Line Clipping - Algorithm


```
#include<stdio.h>
#include<GL/glut.h>
 double xmin=50,ymin=50, xmax=100,ymax=100;
// Window boundaries

double xvmin=200,yvmin=200,xvmax=300,yvmax=300;
// Viewport boundaries

int cliptest(double p, double q, double *t1, double *t2)
{
      double t=q/p;
      if(p < 0.0)    // potentially entry point, update te
      {
            if( t > *t1) *t1=t;
            if( t > *t2) return(false); // line portion is outside
      }
}
```

```
    else if(p > 0.0)    //  Potentially leaving point, update tl
    {
        if( t < *t2) *t2=t;
        if( t < *t1) return(false); // line portion is outside
    }
    else if(p == 0.0)
    {
        if( q < 0.0) return(false); // line parallel to edge but outside
     }
    return(true);
}
```

```
void LiangBarskyLineClipAndDraw(double x0,double y0,double x1,double y1)
{
    double dx=x1-x0, dy=y1-y0, te=0.0, tl=1.0;
    if(cliptest(-dx,x0-xmin,&te,&tl)) // inside test wrt left edge
    if(cliptest(dx,xmax-x0,&te,&tl)) // inside test wrt right edge
    if(cliptest(-dy,y0-ymin,&te,&tl)) // inside test wrt bottom edge
    if(cliptest(dy,ymax-y0,&te,&tl)) // inside test wrt top edge
    {
        if( tl< 1.0 )
        {
            x1 = x0 + tl*dx;
            y1 = y0 + tl*dy;
        }
        if( te> 0.0 )
        {   x0 = x0 + te*dx;
            y0 = y0 + te*dy;
        }
```

```
// Window to viewport mappings
double sx=(xvmax-xvmin)/(xmax-xmin); // Scale parameters
double sy=(yvmax-yvmin)/(ymax-ymin);
double vx0=xvmin+(x0-xmin)*sx;
double vy0=yvmin+(y0-ymin)*sy;
double vx1=xvmin+(x1-xmin)*sx;
double vy1=yvmin+(y1-ymin)*sy;

//draw a red colored viewport
glColor3f(1.0, 0.0, 0.0);
glBegin(GL_LINE_LOOP);
glVertex2f(xvmin, yvmin);
glVertex2f(xvmax, yvmin);
glVertex2f(xvmax, yvmax);
glVertex2f(xvmin, yvmax);
glEnd();
```

```
        glColor3f(0.0,0.0,1.0); // draw blue colored clipped line
        glBegin(GL_LINES);
        glVertex2d (vx0, vy0);
        glVertex2d (vx1, vy1);
        glEnd();
    }
}
```

```
void display()
{
    double x0=60,y0=20,x1=80,y1=120;
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,0.0,0.0);    //draw the line with red color
    glBegin(GL_LINES);
    glVertex2d (x0, y0);
    glVertex2d (x1, y1);
    glEnd();
    glColor3f(0.0, 0.0, 1.0); //draw a blue colored window
    glBegin(GL_LINE_LOOP);
    glVertex2f(xmin, ymin);
        glVertex2f(xmax, ymin);
        glVertex2f(xmax, ymax);
        glVertex2f(xmin, ymax);
    glEnd();
    LiangBarskyLineClipAndDraw(x0,y0,x1,y1);
    glFlush();
}
```

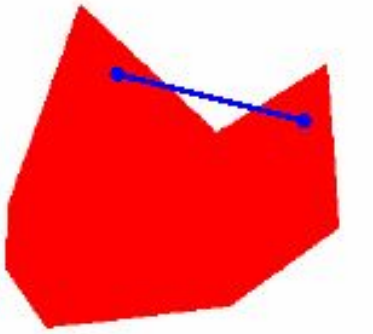
```
void myinit()
{
    glClearColor(1.0,1.0,1.0,1.0);
    glColor3f(1.0,0.0,0.0);
    glPointSize(1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0,499.0,0.0,499.0);
}
```

```
void main(int argc, char** argv)
{

    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500,500);
    glutInitWindowPosition(0,0);
    glutCreateWindow("Liang Barsky Line Clipping Algorithm");
    glutDisplayFunc(display);
    myinit();
    glutMainLoop();
}
```

Polygon types

- Simple polygons are either ***convex*** or ***concave***:
  - Convex polygon: All interior angles < 180°, or any line segment combining two points in the interior is also in the interior

convex polygon



concave polygon

can be split into a number of convex polygons

Identifying a concave polygon

- has at least one interior angle >180 degrees
- extensions of some edges will intersect other edges

One test:

Express each polygon edge as a vector, with a consistent orientation.

Can then calculate cross-products of adjacent edges

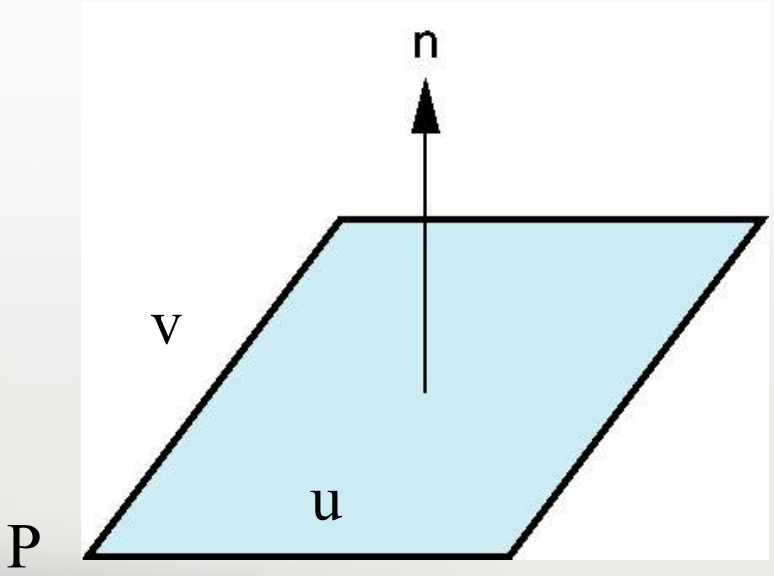
Identifying a concave polygon

- When polygon edges are oriented with an anti-clockwise sense
  - cross product at convex vertex has positive sign
  - concave vertex gives negative sign

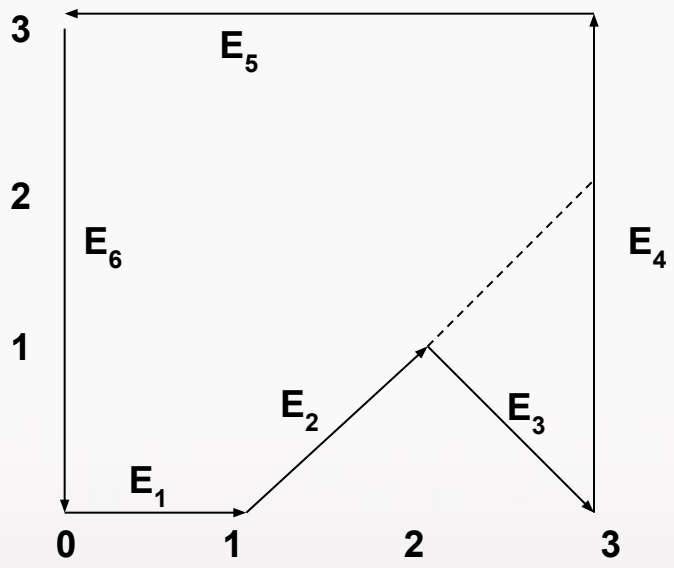
Normals

- Every plane has a vector **n** normal (perpendicular, orthogonal) to it
- **n** = **u** x **v** (vector cross product)

$$\overline{u} \times \overline{v} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix}$$



Vector method for splitting concave polygons

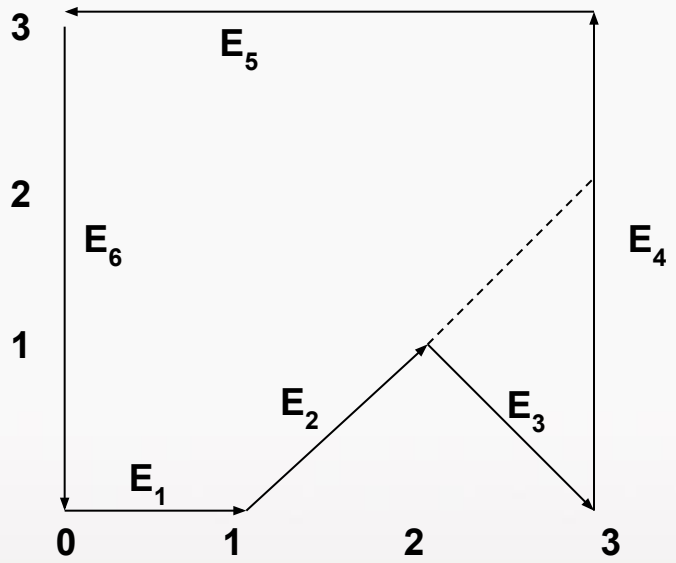


- E_1=(1,0,0) E_2=(1,1,0)
- E_3=(1,-1,0) E_4=(0,3,0)
- E_5=(-3,0,0) E_6=(0,-3,0)

- All z components have 0 value.
- Cross product of two vectors $E_j x E_k$ is perpendicular to them with z component

$$E_{jx}E_{ky}-E_{kx}E_{jy}$$

Example continued

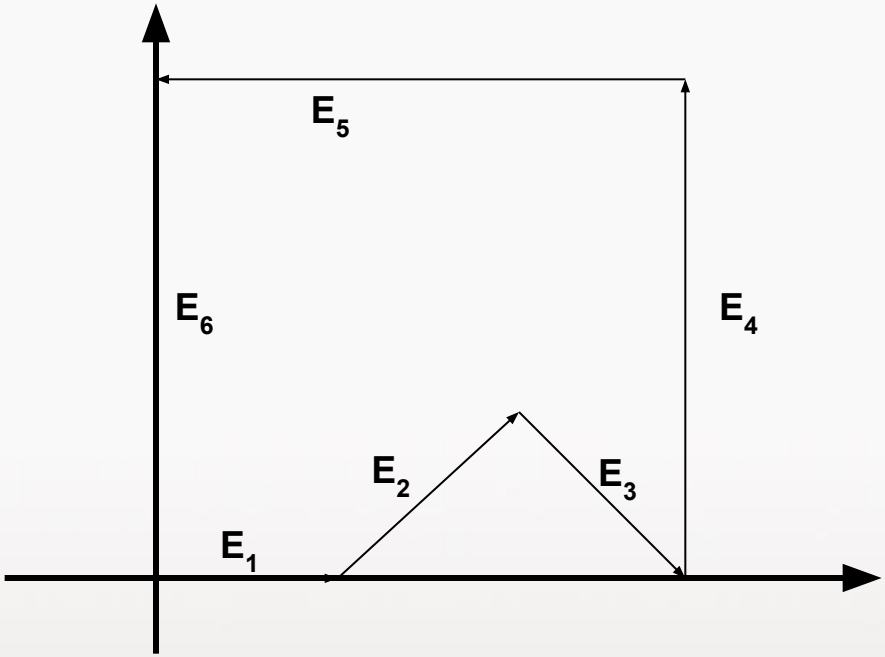


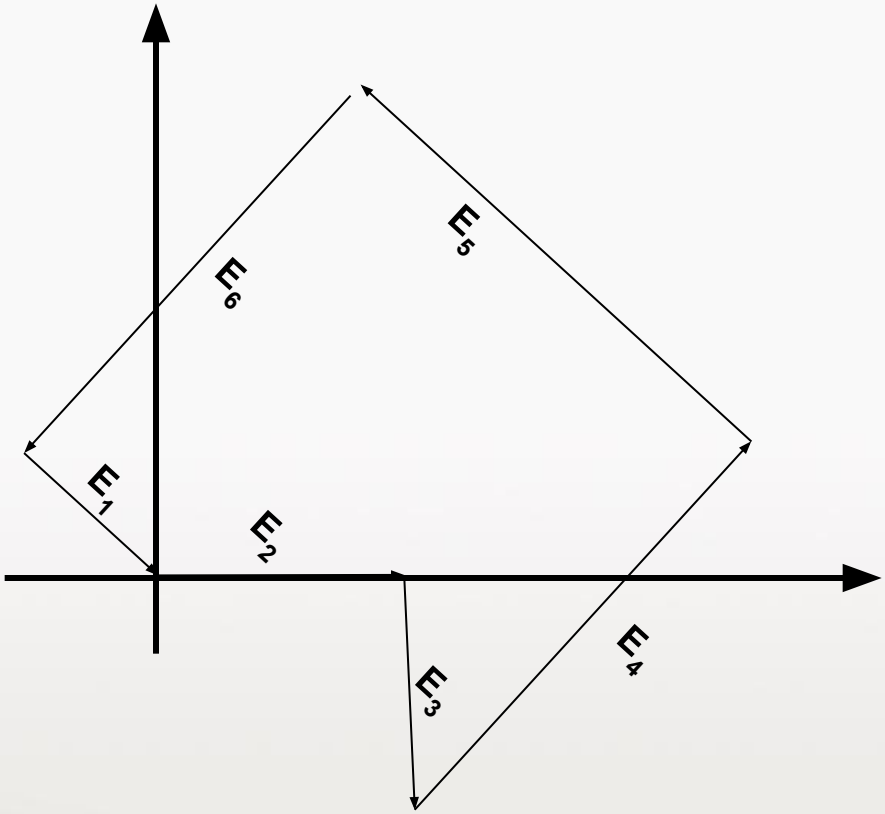
- $E_1 x E_2 = (0,0,1)$
  $E_2 x E_3 = (0,0,-2)$
  $E_3 x E_4 = \ldots$
  $E_4 x E_5 = \ldots$
- $E_5 x E_6 = \ldots$
  $E_6 x E_1 = \ldots$

- Since $E_2 x E_3$ has negative sign, split the polygon along the line of vector E_2

Rotational method

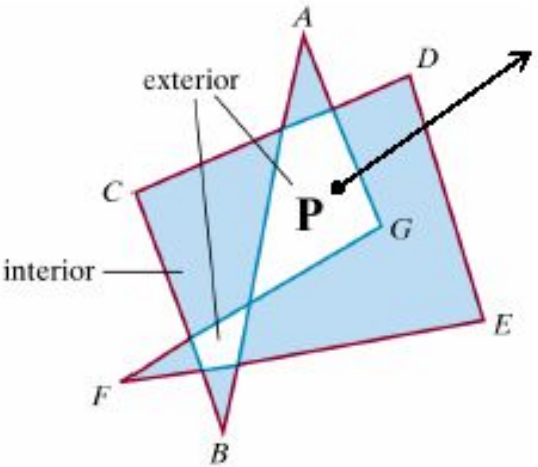
- Rotate the polygon so that each vertex in turn is at coordinate origin.
- If following vertex is below the x axis, polygon is concave.
- Split the polygon by x axis.





Inside-Outside Tests

- Identifying the interior of a polygon (simple or complex) is important to identify the region to be filled
- Odd-even rule: To determine whether point **P** is inside or not. Draw a line starting from P to a distant position. Count the number of edges that cross this line. If the count is **odd** then the point is **inside**, otherwise it is outside.



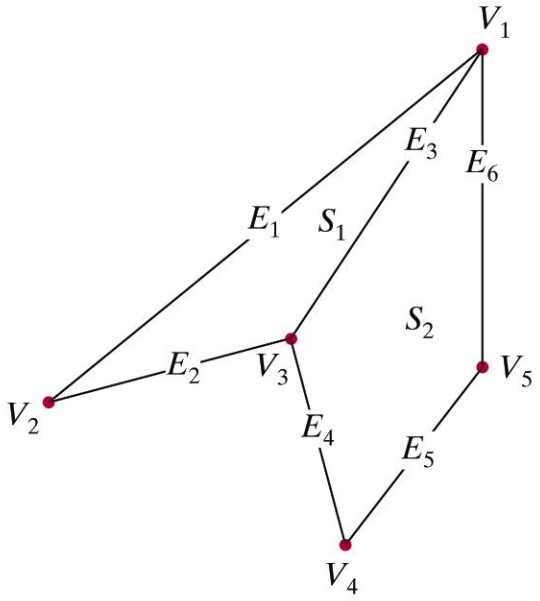
Odd-Even Rule

Polygon tables

- store descriptions of polygon geometry and topology, and surface parameters: colour, transparency, light-reflection
- organise in 2 groups
  - geometric data
  - attribute data

Polygon Tables:

Geometric data



VERTEX TABLE	
V_1:	x_1, y_1, z_1
V_2:	x_2, y_2, z_2
V_3:	x_3, y_3, z_3
V_4:	x_4, y_4, z_4
V_5:	x_5, y_5, z_5

EDGE TABLE	
E_1:	V_1, V_2
E_2:	V_2, V_3
E_3:	V_3, V_1
E_4:	V_3, V_4
E_5:	V_4, V_5
E_6:	V_5, V_1

SURFACE-FACET TABLE	
S_1:	E_1, E_2, E_3
S_2:	E_3, E_4, E_5, E_6

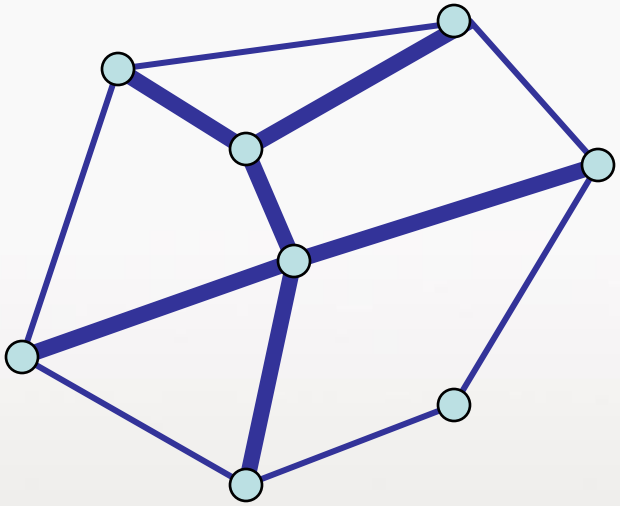
Figure 3-50

Geometric data-table representation for two adjacent polygon surface facets, formed with six edges and five vertices.

- Data can be used for consistency checking
- Additional geometric data stored: slopes, bounding boxes

Shared Edges

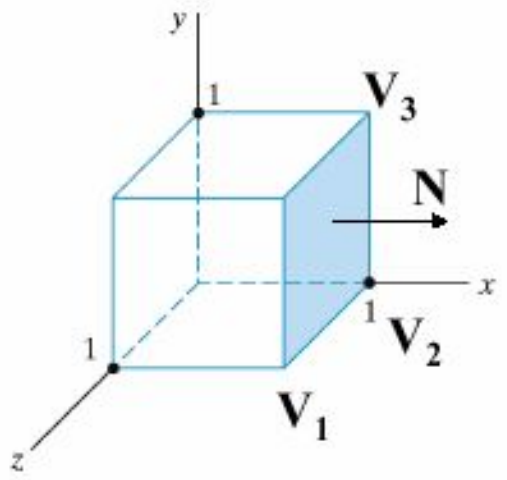
- Vertex lists will draw filled polygons correctly but if we draw the polygon by its edges, shared edges are drawn twice



- Can store mesh by *edge list*

Front and Back Face of a Polygon

- The normal vector points in a direction from the back face of the polygon to the front face
- Normal vector is the cross product of the two edges of the polygon in counter-clockwise direction

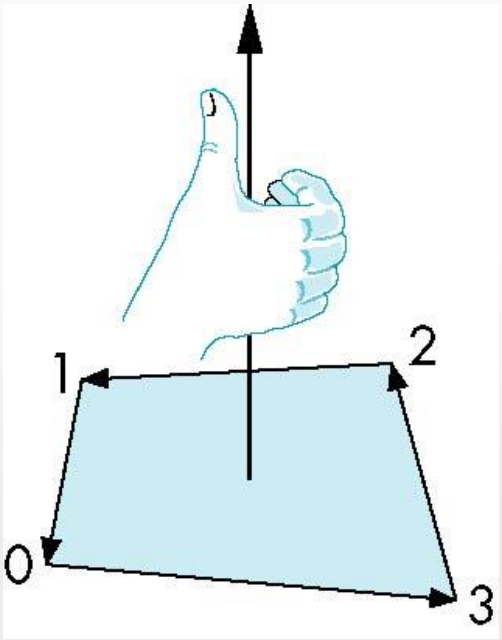


$$\mathbf{N} = (\mathbf{V}_2 - \mathbf{V}_1) \times (\mathbf{V}_3 - \mathbf{V}_2)$$

Inward and Outward Facing Polygons

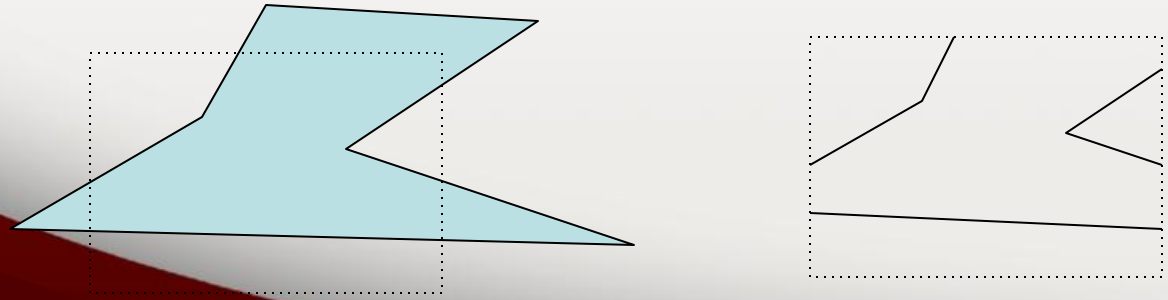
Use the *right-hand rule* =
counter-clockwise encirclement
of outward-pointing normal

OpenGL can treat inward and
outward facing polygons differently



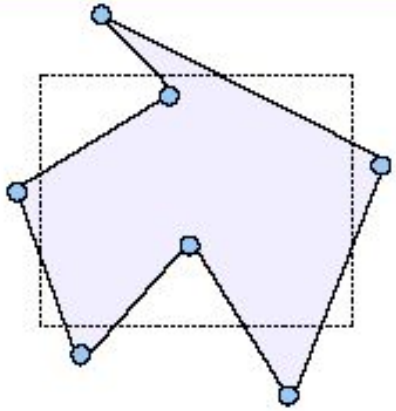
Polygon clipping

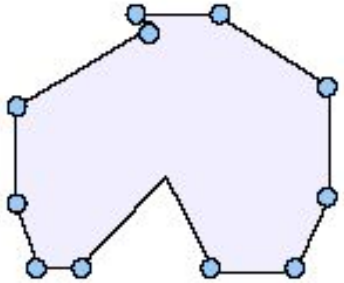
- Clipping a polygon fill area needs more than line-clipping of the polygon edges
  - would produce and unconnected set of lines
- Must generate one or more closed polylines, which can be filled with the assigned colour or pattern



Polygon Clipping

- Find the vertices of the new polygon(s) inside the window.
- Sutherland-Hodgeman Polygon Clipping:
  Check each edge of the polygon against all window boundaries. Modify the vertices based on transitions. Transfer the new edges to the next clipping boundary.





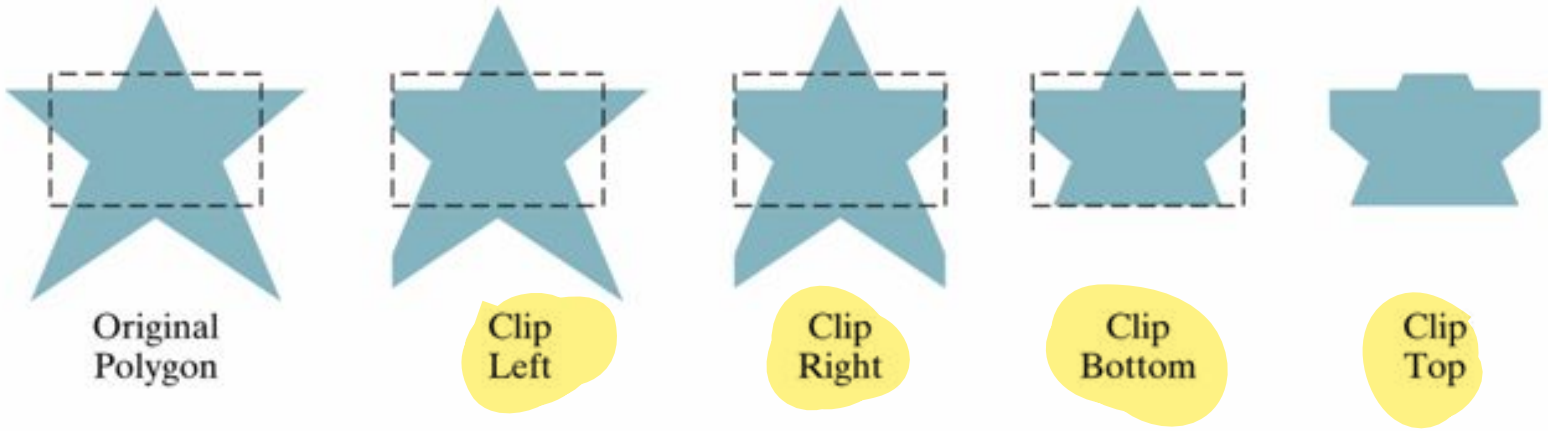
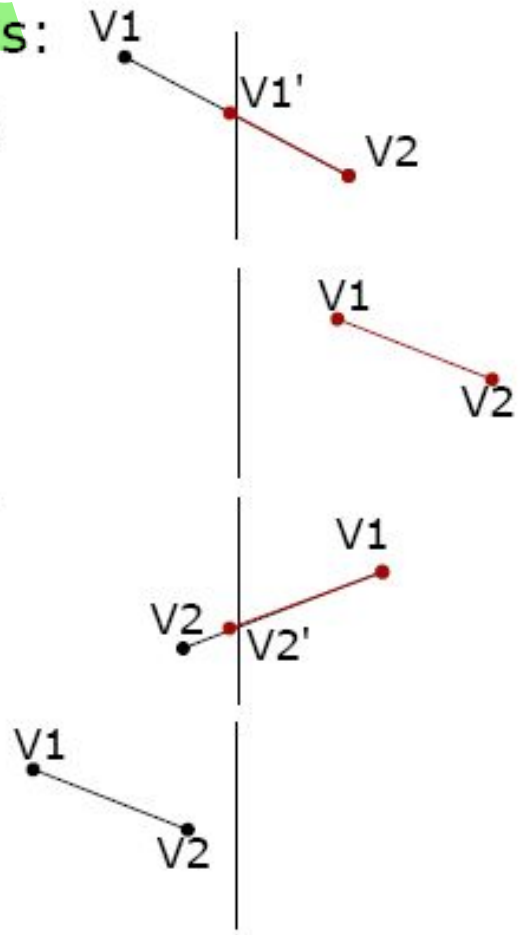


Figure 6-23

Processing a polygon fill area against successive clipping-window boundaries.

Sutherland-Hodgeman Polygon Clipping

- Traverse edges for borders; 4 cases:
  - V1 outside, V2 inside: take V1' and V2
  - V1 inside, V2 inside: take V1 and V2
  - V1 inside, V2 outside: take V1 and V2'
  - V1 outside, V2 outside: take none



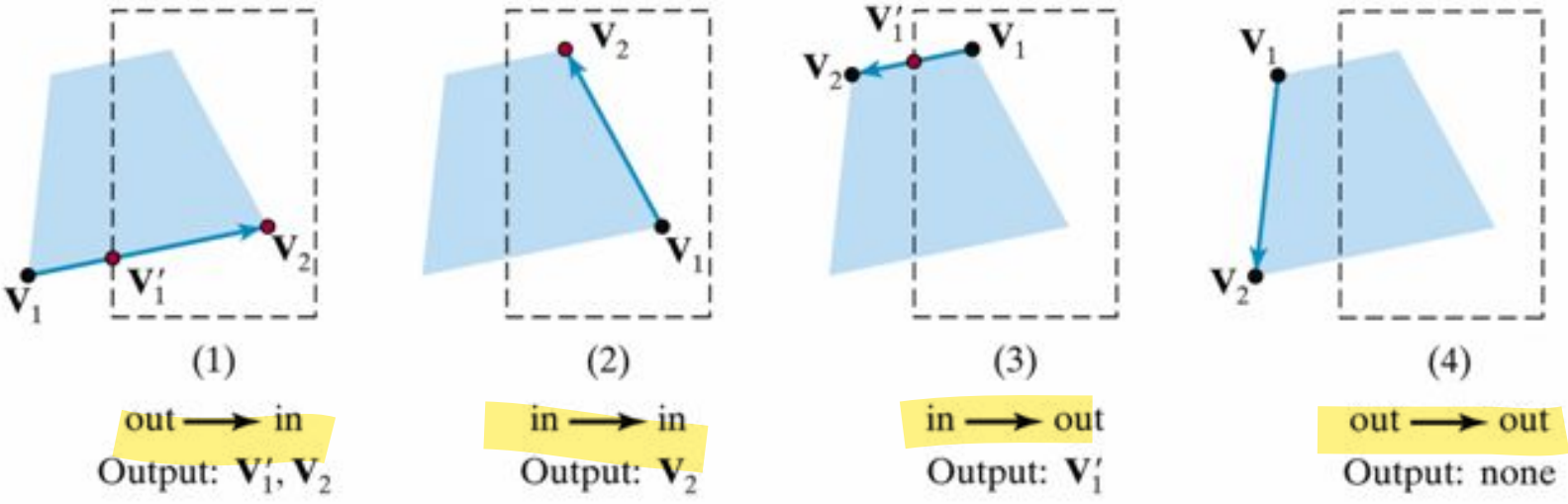
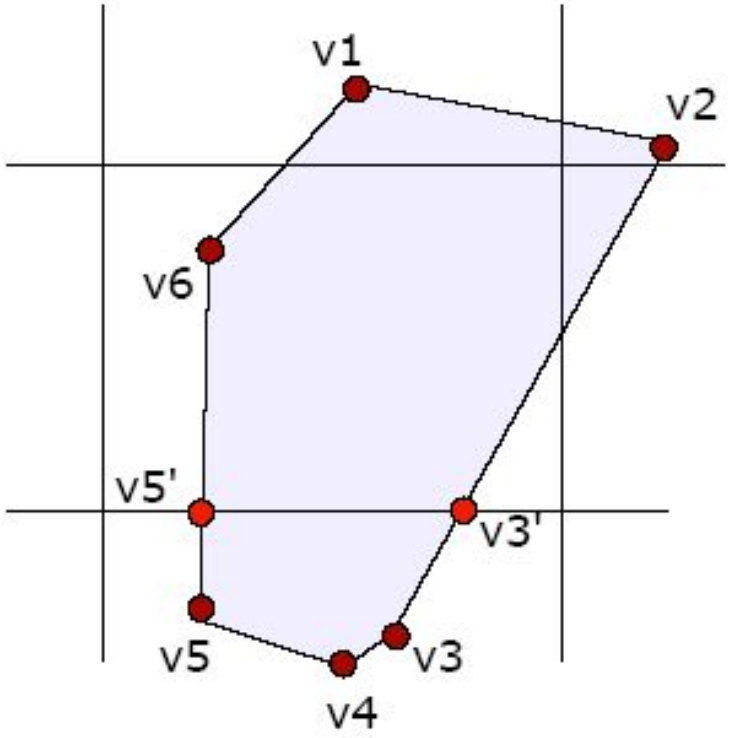


Figure 6-26

The four possible outputs generated by the left clipper, depending on the position of a pair of endpoints relative to the left boundary of the clipping window.



- Left border:

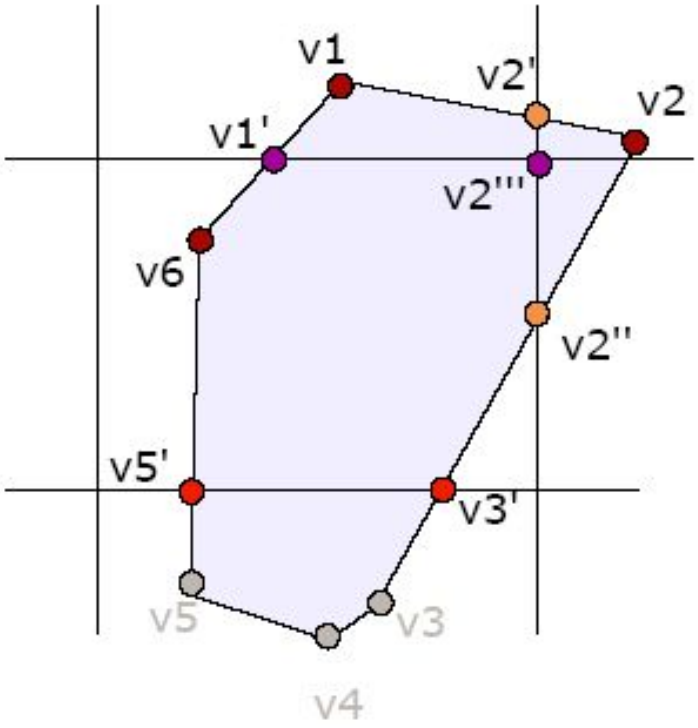
v1 v2	both inside	v1 v2
v2 v3	both inside	v2 v3
.....	" "	.....

v1,v2,v3,v4v5,v6,v1

- Bottom Border:

v1 v2	both inside	v1 v2
v2 v3	v2 i, v3 o	v2 v3'
v3 v4	both outside	none
v4 v5	both outside	none
v5 v6	v5 o, v6 i	v5' v6
v6 v1	both inside	v6 v1

v1,v2,v3',v5',v6,v1



- v1,v2,v3',v5',v6,v1
- Right border:

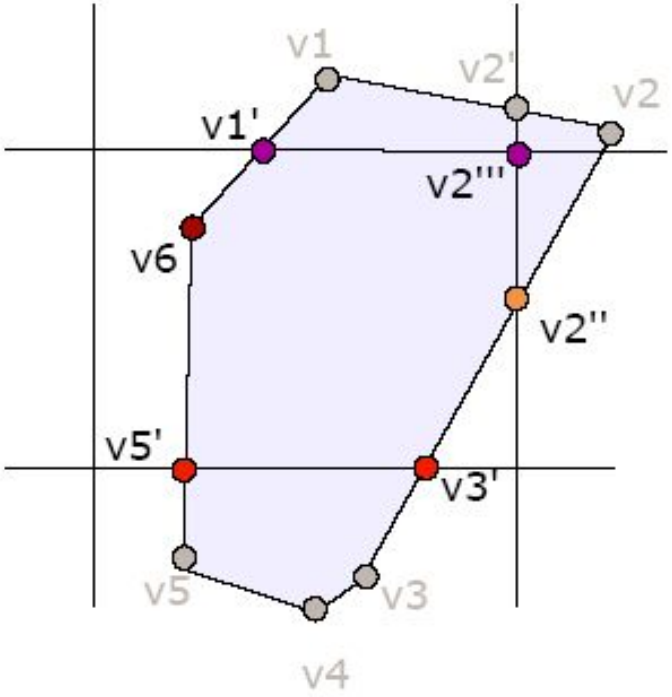
v1 v2	v1 i, v2 o	v1 v2'
v2 v3'	v2 o, v3'i	v2'' v3'
v3' v5'	both inside	v3' v5'
v5' v6	both inside	v5' v6
v6 v1	both inside	v6 v1

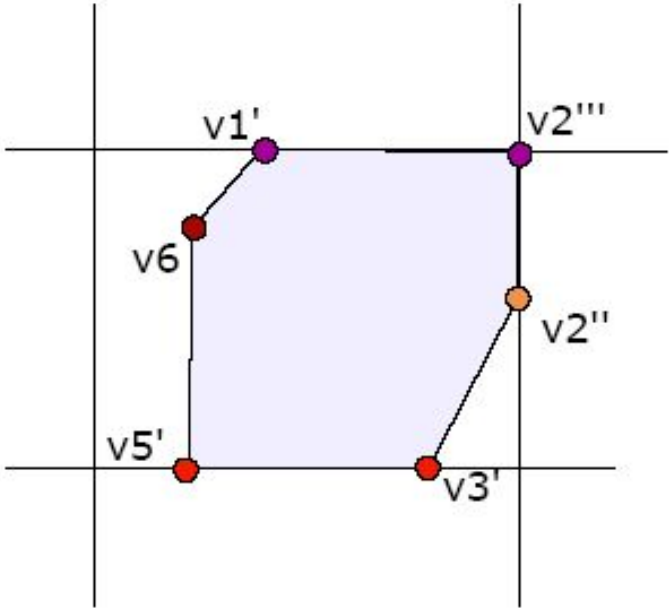
v1,v2',v2'',v3',v5',v6,v1

- Top Border:

v1 v2'	both outside	none
v2' v2''	v2' o, v2'' i	v2''' v2''
v2'' v3'	both inside	v2'' v3'
v3' v5'	both inside	v3' v5'
v5' v6	both inside	v5' v6
v6 v1	v6 i, v1 o	v6 v1'

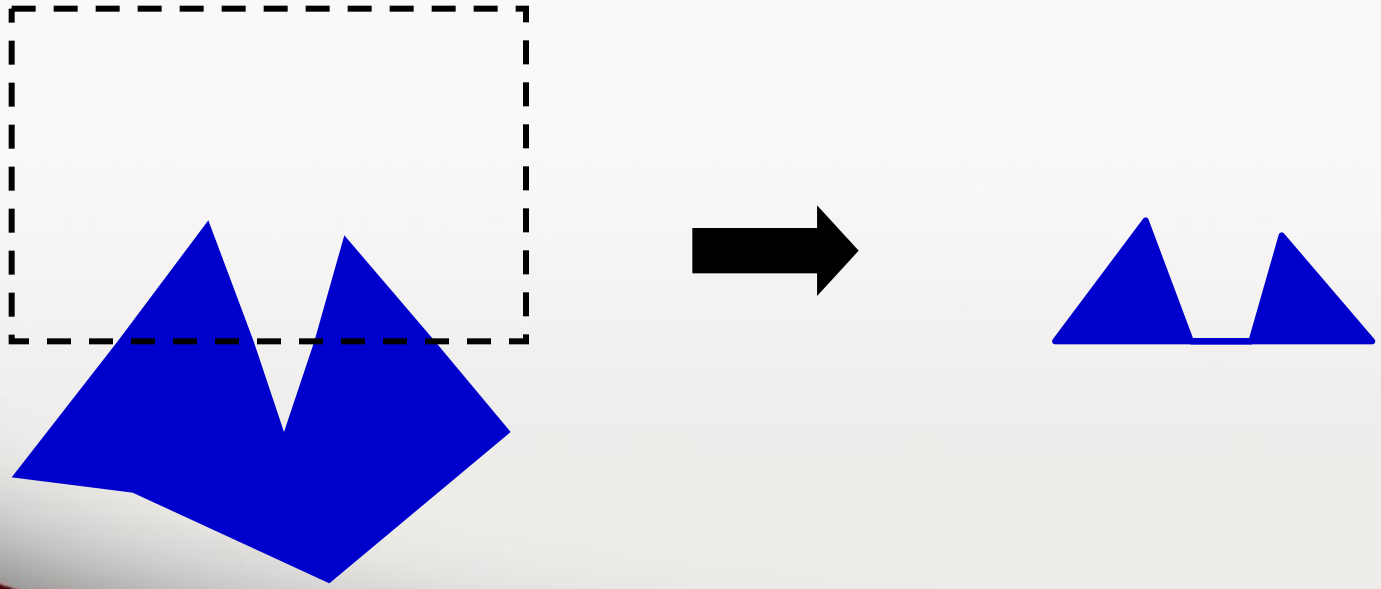
v2''',v2'',v3',v5',v6,v1'





Sutherland-Hodgman Polygon Clipping

- The algorithm correctly clips convex polygons, but may display extraneous lines for concave polygons



Other Issues in Clipping

- Problem in Sutherland-Hodges. Weiler-Atherton has a solution
- Clipping other shapes: Circle, Ellipse, Curves.
- Clipping a shape against another shape
- Clipping the exteriors.